

IASI Quarto Technical Guide

Framework for Engineering Documentation with Quarto

IASI Project

Table of contents

1	Introducción	11
1.1	iasi.quarto y Quarto	11
1.2	¿Qué encontrarás en este libro?	11
1.3	Requisitos	12
2	Quarto como base	13
2.1	Qué hace Quarto	14
2.2	Qué añade iasi.quarto	15
2.3	Separación de responsabilidades	16
2.4	Un proyecto sigue siendo un proyecto Quarto	17
2.5	Cuándo resulta útil	18
3	Qué es iasi.quarto	19
3.1	Una convención sobre Quarto	19
3.2	El problema que resuelve	19
3.3	Qué aporta	20
3.4	Qué no pretende ser	20
3.5	Una herramienta para proyectos que crecen	21
4	Fundamentos	22
4.1	La publicación IASI Quarto	22
4.1.1	La firma de una publicación	22
4.1.2	Una publicación no es necesariamente un libro	23
4.1.3	Publicación individual	24
4.1.4	Multiproyecto	24
4.1.5	La publicación actual tiene prioridad	25
4.1.6	Publicación y contenido	26
4.1.7	Archivos fuente y archivos derivados	27
4.1.8	Reconocer no significa construir	28
4.1.9	La publicación como unidad de construcción	28
4.2	Estructura de un proyecto	29
4.3	Configuración general y configuración por formato	30
4.4	Configuración de iasi.quarto	31
4.5	Directorio de contenido	32
4.6	Documentos de raíz	32

4.7	Carpetas de contenido	33
4.8	Estructura regular	33
4.9	Estructura structured	35
4.10	Archivos fuente y archivos derivados	36
4.11	La estructura preparada	36
4.12	Publicaciones múltiples	37
4.13	Una estructura explícita	37
4.14	El archivo <code>iasi.yml</code>	38
4.15	Valores predeterminados	38
4.16	<code>strategy</code>	39
4.16.1	<code>regular</code>	39
4.16.2	<code>structured</code>	40
4.16.3	<code>direct</code>	40
4.17	<code>content-dir</code>	41
4.18	<code>numbered</code>	41
4.18.1	<code>numbered: false</code>	42
4.19	Configuración parcial	43
4.20	Configuración válida completa	44
4.21	YAML mal formado	44
4.22	Valores semánticamente inválidos	45
4.23	Propiedades desconocidas	46
4.24	Separación de responsabilidades	46
4.25	Configuración y ciclo de construcción	47
4.26	Un contrato pequeño	48
4.27	Estrategias de publicación	48
4.28	Una misma estructura física, distintas publicaciones	49
4.29	Estrategia <code>regular</code>	50
4.29.1	Propiedad del <code>index.qmd</code>	51
4.29.2	<code>front-matter.quarto</code>	51
4.29.3	Estructura resultante	52
4.29.4	Cuándo utilizar <code>regular</code>	53
4.30	Estrategia <code>structured</code>	53
4.30.1	Propiedad del <code>index.qmd</code>	54
4.30.2	Estructura resultante	55
4.30.3	<code>front-matter.quarto</code> no participa	55
4.30.4	Cuándo utilizar <code>structured</code>	56
4.31	Estrategia <code>direct</code>	56
4.31.1	Qué no hace <code>direct</code>	57
4.31.2	Cuándo utilizar <code>direct</code>	57
4.32	Comparación	58
4.33	Estrategia y numeración	58
4.34	Estrategia y formatos de salida	59
4.35	Elección de estrategia	60

4.36	Una decisión editorial explícita	60
4.37	Ciclo de construcción	61
4.38	<code>build()</code> como orquestador	61
4.39	Primera etapa: <code>validate()</code>	62
4.39.1	Publicación actual	63
4.39.2	Multiproyecto	64
4.40	Selección de publicaciones	64
4.41	Segunda etapa: <code>.discover()</code>	65
4.41.1	Aplicación de valores predeterminados	66
4.42	Tercera etapa: <code>.check()</code>	67
4.42.1	Resultado de la comprobación	67
4.43	Cuarta etapa: <code>.prepare()</code>	68
4.43.1	Preparación <code>regular</code>	68
4.43.2	Preparación <code>structured</code>	69
4.43.3	Preparación <code>direct</code>	69
4.43.4	Artefactos	69
4.44	Idempotencia de la preparación	69
4.45	Resolución de formatos	70
4.46	Quinta etapa: <code>.render()</code>	71
4.46.1	Delegación en Quarto	72
4.46.2	Resultado	72
4.47	Fallar antes de renderizar	72
4.48	Ciclo de una publicación individual	73
4.49	Ciclo de un multiproyecto	74
4.50	Estado acumulado	75
4.51	Un pipeline deliberado	75
4.52	API pública	76
4.53	<code>validate()</code>	76
4.54	Resultado de <code>validate()</code>	77
4.54.1	<code>plan\$path</code>	78
4.54.2	<code>plan\$current</code>	78
4.54.3	<code>plan\$books</code>	78
4.55	Ruta no reconocida	78
4.56	Qué hace <code>validate()</code>	79
4.57	Ejemplo con una publicación	79
4.58	Ejemplo con un multiproyecto	80
4.59	<code>build()</code>	80
4.60	Construcción predeterminada	81
4.61	Argumento <code>path</code>	82
4.62	Argumento <code>format</code>	82
4.63	Argumento <code>book</code>	84
4.63.1	Nombre completo	84
4.63.2	Nombre sin prefijo numérico	84

4.63.3	Prefijo numérico	84
4.64	<code>book = NULL</code> y <code>book = "all"</code>	85
4.65	Selecciones no encontradas	85
4.66	Selección antes del descubrimiento	85
4.67	Ejemplos de uso	86
4.67.1	Construir la publicación actual	86
4.67.2	Construir solo HTML	86
4.67.3	Construir solo PDF	86
4.67.4	Construir un multiproyecto completo	86
4.67.5	Construir una publicación del multiproyecto	87
4.67.6	Construir una publicación y un formato	87
4.67.7	Construir varias publicaciones	87
4.68	Resultado de <code>build()</code>	87
4.69	Mensajes de construcción	88
4.70	API pública y funciones privadas	89
4.71	Por qué no se expone cada etapa	90
4.72	<code>validate()</code> frente a <code>build()</code>	90
4.73	Llamadas con argumentos nombrados	91
4.74	Contrato estable, implementación evolutiva	92
5	Instalación	93
5.1	Instalación	93
5.2	Requisitos	94
5.3	R	94
5.4	Quarto	95
5.5	HTML	96
5.6	PDF	96
5.7	Git	97
5.8	Acceso al repositorio	97
5.9	Permisos de escritura	98
5.10	Red y servicios externos	98
5.11	Comprobación mínima	98
5.12	Instalar <code>iasi.quarto</code>	99
5.13	Instalar <code>remotes</code>	100
5.14	Instalar la versión actual	100
5.15	Instalar una versión concreta	101
5.16	Actualizar el paquete	101
5.17	Comprobar la versión instalada	102
5.18	Biblioteca de instalación	103
5.19	Dependencias	104
5.20	Instalación desde una copia local	104
5.21	Cargar el paquete	105
5.22	Instalación reproducible	105

5.23	Instalación mínima	106
5.24	Verificar la instalación	106
5.25	Comprobar el paquete	106
5.26	Comprobar Quarto	107
5.27	Situarse en una publicación	108
5.28	Validar la publicación	108
5.28.1	Publicación actual	109
5.28.2	Multiproyecto	109
5.29	Directorio no reconocido	110
5.30	Construir HTML	111
5.31	Inspeccionar el resultado	111
5.32	Construir PDF	112
5.33	Verificación completa	113
5.34	Diagnóstico rápido	114
5.35	Resultado esperado	114
6	PlantUML	116
6.1	Instalación	116
6.1.1	¿Por qué PlantUML?	117
6.1.2	Estilos compartidos	119
6.1.3	Servidor independiente	120
6.1.4	Una imagen común para HTML y PDF	121
6.1.5	Caché antes del renderizado	121
6.1.6	Integración con la arquitectura de extensiones	122
6.1.7	Comparación resumida	122
6.1.8	Criterio de elección	123
6.2	Servidor PlantUML	124
6.3	Alternativas	124
6.3.1	Servidor público	124
6.3.2	Servidor local	125
6.3.3	Servidor compartido	125
6.4	Instalación recomendada con Docker	125
6.5	Instalación con Docker Compose	126
6.6	Comprobar el servidor	127
6.7	Utilizarlo desde otra máquina	127
6.8	Configuración local recomendada	128
6.9	Configuración para un equipo	129
6.10	Actualizar el servidor	129
6.11	Seguridad y privacidad	130
6.12	Disponibilidad y caché	130
6.13	La instalación de referencia	130
6.14	Configurar la extensión	131
6.15	Ruta del filtro	132

6.16	<code>enabled</code>	132
6.17	<code>server</code>	133
6.18	<code>format</code>	134
6.19	<code>cache</code>	134
6.20	<code>styles</code>	135
6.21	Configuración mínima	136
6.22	Configuración recomendada	137
6.23	Configuración por entorno	138
6.24	Opciones de bloque	138
6.25	Configuración HTML y PDF	139
6.26	Comprobar la configuración	139
6.27	Errores de configuración habituales	140
6.27.1	El filtro no existe	140
6.27.2	El servidor incluye <code>/png</code>	140
6.27.3	El estilo no puede leerse	141
6.27.4	<code>localhost</code> apunta al lugar equivocado	141
6.27.5	La configuración declara SVG	141
6.28	Contrato de configuración	142
6.29	Escribir diagramas	142
6.30	Estructura mínima	143
6.31	Primer diagrama	143
6.32	Diagramas de secuencia	145
6.33	Diagramas de componentes	146
6.34	Diagramas de actividad	147
6.35	Diagramas de despliegue	149
6.36	Orientación	151
6.37	Alias y nombres visibles	152
6.38	Etiquetas en las relaciones	153
6.39	Notas	154
6.40	Separar significado y apariencia	154
6.41	Un diagrama por bloque	155
6.42	Ubicación dentro del documento	156
6.43	Tamaño y complejidad	156
6.44	Nombres estables	157
6.45	Caracteres y codificación	157
6.46	Opciones por bloque	158
6.47	Prueba mínima	159
6.48	Criterio práctico	159
6.49	Estilos compartidos	160
6.50	Estructura recomendada	160
6.51	Qué pertenece al estilo	161
6.52	Qué debe permanecer en el diagrama	161
6.53	Cómo se aplica el estilo	162

6.54	Varios archivos de estilo	163
6.55	Un único archivo o varios	164
6.56	Estilo base recomendado	164
6.57	Estilos por tipo de elemento	164
6.57.1	Rectángulos	165
6.57.2	Componentes	165
6.57.3	Secuencias	165
6.57.4	Actividades	165
6.58	Colores semánticos	165
6.59	Macros y convenciones reutilizables	166
6.60	Estereotipos	167
6.61	Fuentes	168
6.62	Transparencia y fondo	168
6.63	Cambiar el estilo	169
6.64	Estilo y caché	169
6.65	Probar el estilo	170
6.66	Criterio de diseño	170
6.67	Caché de diagramas	171
6.68	Por qué existe	172
6.69	Ubicación	172
6.70	Identidad de un diagrama	173
6.71	Flujo de lectura	174
6.72	Acierto de caché	174
6.73	Fallo de caché	175
6.74	Caché activada	175
6.75	Caché desactivada	176
6.76	Desactivar la caché en un bloque	176
6.77	Limpiar la caché	177
6.78	Cuándo limpiar	177
6.79	Cambios de estilo	178
6.80	Cambio de formato	178
6.81	Cambio de servidor	179
6.82	Versión de la extensión	180
6.83	Caché y disponibilidad del servidor	180
6.84	Caché en integración continua	181
6.84.1	Construcción limpia	181
6.84.2	Caché persistente	181
6.85	Versionado	182
6.86	Crecimiento de la caché	182
6.87	Diagnóstico	183
6.88	Prueba de funcionamiento	183
6.89	Contrato de la caché	184
6.90	Parámetros de los bloques PlantUML	185

6.91	Forma general	185
6.92	Parámetros de compilación	186
6.93	<code>server</code>	186
6.94	<code>format</code>	187
6.95	<code>cache</code>	187
6.96	Atributos de presentación	188
6.97	<code>width</code>	189
6.98	<code>height</code>	189
6.99	<code>fig-cap</code>	190
6.100	Identificador	190
6.101	<code>fig-alt</code>	191
6.102	<code>fig-align</code>	192
6.103	<code>caption</code>	192
6.104	<code>label</code>	193
6.105	Combinación recomendada	193
6.106	Qué parámetros modifican la caché	194
6.107	Parámetros globales y parámetros de bloque	195
6.108	Tabla de referencia	196
6.109	Criterio de uso	196
6.110	Resolución de problemas	197
6.111	Orden de diagnóstico	197
6.112	El filtro no se carga	198
6.113	Se está editando otra copia	198
6.114	Error de sintaxis Lua	199
6.115	El servidor no responde	200
6.116	<code>localhost</code> apunta al lugar equivocado	201
6.117	La URL contiene <code>/svg/</code>	201
6.118	Forzar temporalmente una comprobación	202
6.119	SVG y construcción PDF	202
6.120	La respuesta está vacía	203
6.121	Error de fuente PlantUML	203
6.122	El estilo no puede leerse	204
6.123	El estilo produce un error	205
6.124	El diagrama no cambia	205
6.125	El diagrama funciona en HTML pero no en PDF	206
6.126	El diagrama es demasiado ancho	206
6.127	La tipografía cambia	207
6.128	Caracteres incorrectos	207
6.129	Error al descargar la imagen	208
6.130	MIME inesperado	208
6.131	Caché dañada	209
6.132	Configuración no aplicada	209
6.133	Una publicación utiliza otro <code>_quarto.yml</code>	210

6.134Limpieza mínima	210
6.135Diagnóstico completo	211
6.136Tabla rápida	211
6.137Información útil al informar de un error	212

1 Introducción

Bienvenido a la guía de usuario de **iasi.quarto**.

Este libro está dirigido a quienes desean utilizar **iasi.quarto** para construir proyectos documentales basados en Quarto.

No es necesario conocer la arquitectura interna del framework ni los detalles de su implementación.

El objetivo de esta guía es ayudarte a comprender la filosofía de **iasi.quarto**, instalarlo, crear tu primer proyecto y aprovechar sus capacidades en situaciones reales.

1.1 iasi.quarto y Quarto

iasi.quarto vive sobre Quarto.

No pretende sustituirlo, modificar su modelo documental ni crear un sistema de publicación alternativo.

Quarto continúa siendo el motor responsable de procesar los documentos y generar los formatos finales. **iasi.quarto** proporciona una capa de organización, automatización y convenciones que facilita el desarrollo y mantenimiento de proyectos documentales de cualquier tamaño.

1.2 ¿Qué encontrarás en este libro?

A lo largo de esta guía aprenderás a:

- comprender el propósito de **iasi.quarto**;
- instalar el framework;
- crear un nuevo proyecto;
- organizar la documentación;
- generar libros en distintos formatos;
- configurar el comportamiento del framework;
- utilizar la API pública;
- resolver los problemas más habituales.

Cada capítulo desarrolla un aspecto concreto y puede consultarse de forma independiente cuando ya se conoce el funcionamiento básico del framework.

1.3 Requisitos

Se recomienda tener conocimientos básicos de:

- Quarto;
- Git;
- R (para utilizar el paquete `iasi.quarto`).

No obstante, los conceptos específicos del framework se explican desde el principio.

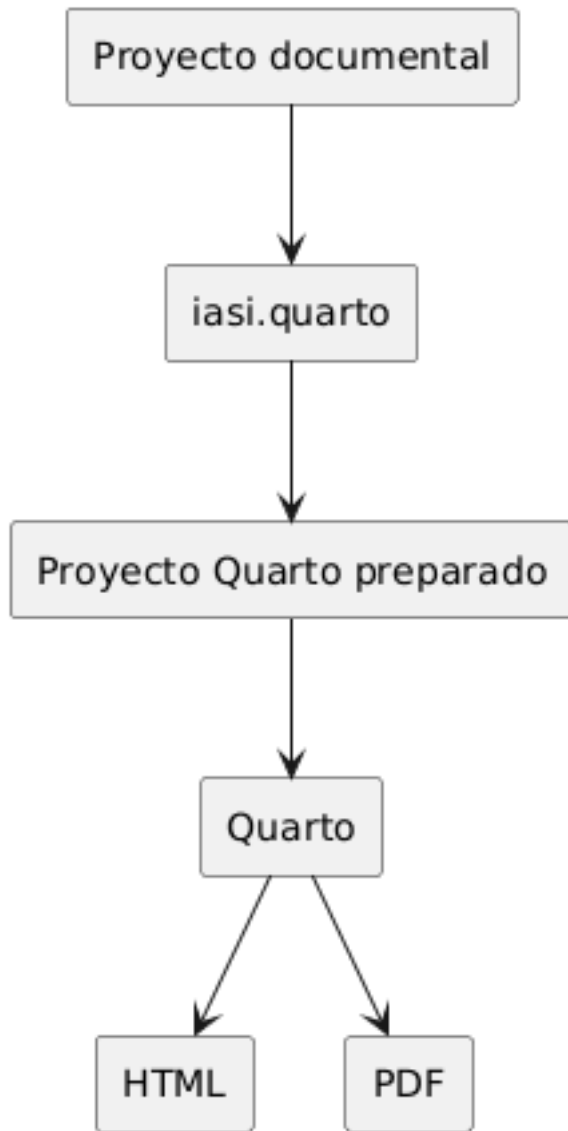
Comencemos entendiendo por qué nació el framework y qué problemas pretende resolver.

2 Quarto como base

`iasi.quarto` está construido sobre Quarto.

No introduce un nuevo formato documental ni sustituye el proceso de publicación de Quarto. Los archivos continúan siendo documentos `.qmd`, la configuración sigue expresándose mediante YAML y los formatos finales son generados por Quarto.

Esta relación permite utilizar `iasi.quarto` sin abandonar el ecosistema original.



La idea central es sencilla:

`iasi.quarto` organiza y prepara. Quarto renderiza.

2.1 Qué hace Quarto

Quarto es el motor de publicación.

Entre otras tareas, se encarga de:

- interpretar los documentos `.qmd`;
- aplicar la configuración definida en `_quarto.yml`;
- ejecutar el código incluido en los documentos;
- resolver referencias, citas, figuras y tablas;
- aplicar temas, plantillas y estilos;
- generar formatos como HTML y PDF.

Un proyecto utilizado por `iasi.quarto` continúa siendo un proyecto Quarto y puede renderizarse directamente con sus herramientas habituales:

```
quarto render
```

`iasi.quarto` no altera ese principio.

2.2 Qué añade `iasi.quarto`

En un libro Quarto, la estructura de la publicación se declara normalmente en `_quarto.yml`.

Por ejemplo:

```
book:
  chapters:
    - index.qmd
    - chapters/01-intro.qmd
    - chapters/02-installation.qmd
    - chapters/03-usage.qmd
```

Esta configuración es clara y suficiente para publicaciones pequeñas. Sin embargo, a medida que un proyecto crece, mantener manualmente la relación de capítulos, partes y documentos puede convertirse en una tarea repetitiva y propensa a errores.

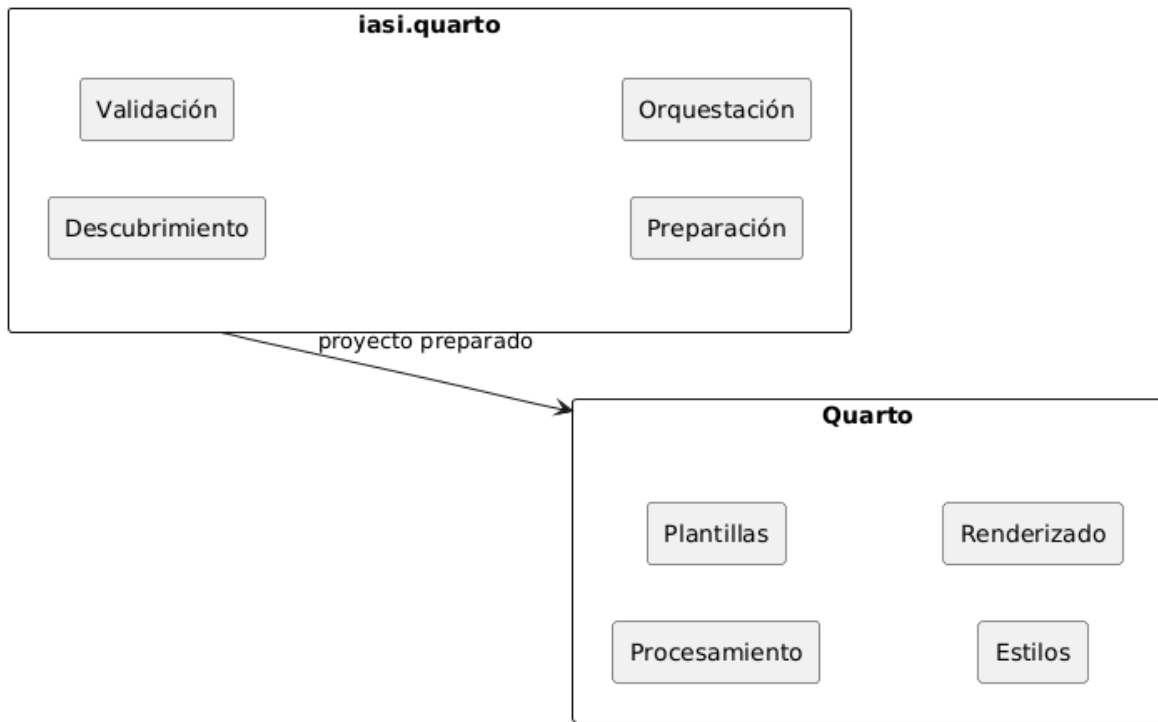
`iasi.quarto` añade una capa de convenciones y automatización que permite deducir parte de esa estructura a partir de la organización del proyecto.

Su trabajo consiste en:

- descubrir los documentos que forman la publicación;
- interpretar su organización;
- comprobar que la estructura sea coherente;
- preparar la configuración que necesita Quarto;
- coordinar la generación de uno o varios formatos.

2.3 Separación de responsabilidades

La separación entre ambas herramientas es deliberada.



Quarto controla:

- el modelo documental;
- los formatos de salida;
- los motores de ejecución;
- las plantillas;
- los estilos;
- el proceso de renderizado.

`iasi.quarto` controla:

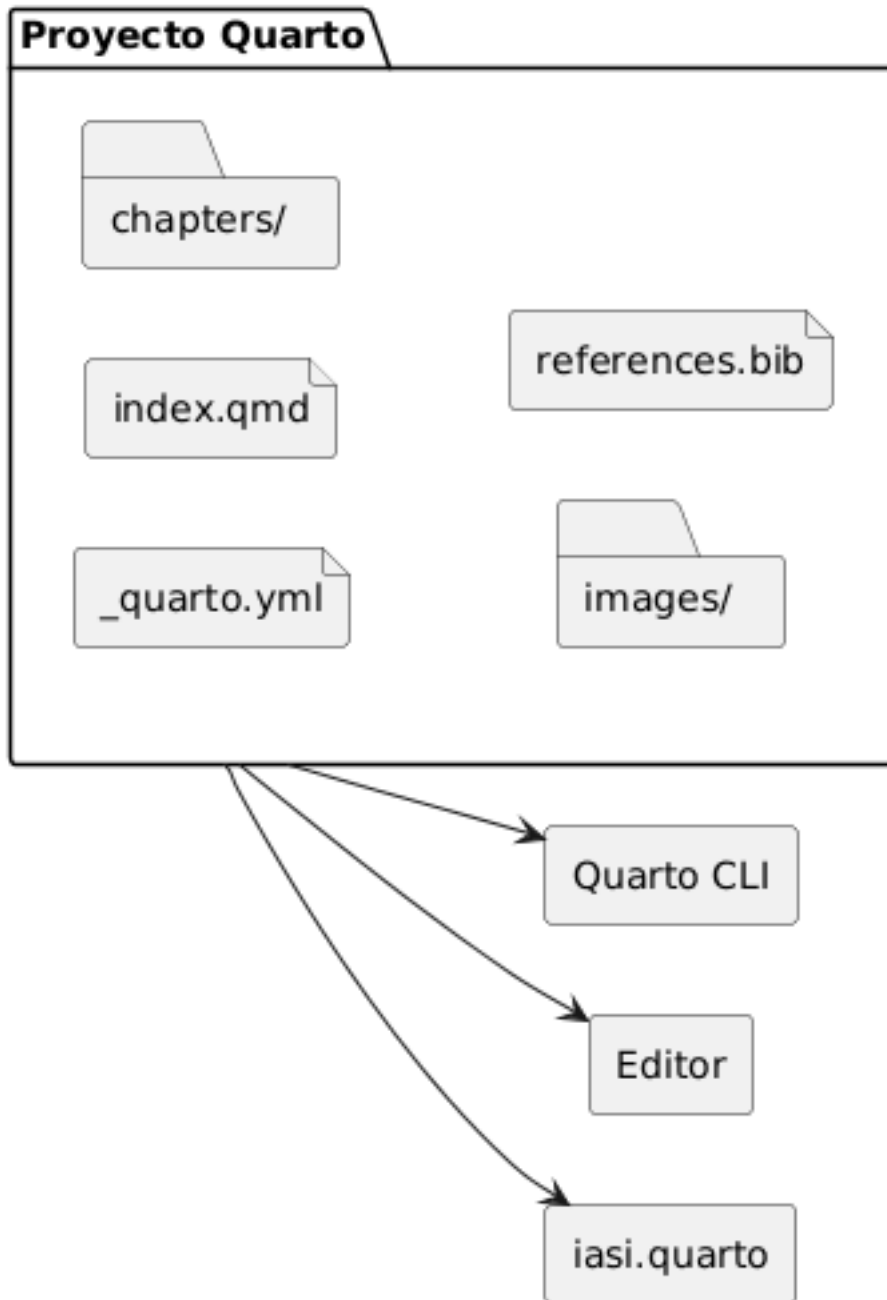
- las convenciones de organización;
- el descubrimiento de contenido;
- la validación de la estructura;
- la preparación de la publicación;
- la coordinación de libros y formatos.

Esta división evita duplicar capacidades que Quarto ya resuelve correctamente.

2.4 Un proyecto sigue siendo un proyecto Quarto

Adoptar `iasi.quarto` no encierra la publicación en un formato propietario.

El resultado preparado continúa utilizando archivos reconocibles por Quarto:



Esto permite:

- utilizar directamente la línea de comandos de Quarto;
- trabajar con las extensiones habituales del editor;
- conservar temas y configuraciones existentes;
- integrar otras extensiones de Quarto;
- dejar de utilizar `iasi.quarto` sin convertir los documentos.

`iasi.quarto` añade una capa sobre el proyecto, pero no se apropia de él.

2.5 Cuándo resulta útil

La automatización aporta más valor cuando:

- la publicación contiene muchos capítulos;
- los documentos cambian de posición con frecuencia;
- existen varios libros en un mismo repositorio;
- deben generarse varios formatos;
- se desea aplicar una organización uniforme;
- el proyecto debe poder crecer sin mantener manualmente listas extensas.

En publicaciones pequeñas, la ventaja principal es la sencillez.

En publicaciones grandes, la ventaja es mantener bajo control una estructura que, de otro modo, acabaría dispersa entre directorios, nombres de archivos y configuración YAML.

En el siguiente capítulo veremos cómo instalar `iasi.quarto` y comprobar que Quarto y R están correctamente disponibles.

3 Qué es `iasi.quarto`

`iasi.quarto` es una capa de organización y automatización construida sobre Quarto.

Su propósito no es sustituir el modelo de publicación de Quarto, sino facilitar el trabajo cuando un proyecto empieza a crecer: aparecen más capítulos, más formatos, distintas estructuras documentales y la necesidad de repetir tareas de preparación y renderizado de forma predecible.

3.1 Una convención sobre Quarto

Quarto ya sabe transformar documentos en libros, sitios web, artículos y otros formatos. `iasi.quarto` se ocupa de lo que ocurre alrededor de ese proceso:

- descubrir la estructura del proyecto;
- validar que esa estructura sea coherente;
- preparar los artefactos necesarios;
- renderizar uno o varios formatos;
- coordinar la construcción de uno o varios libros.

Quarto continúa siendo el motor de publicación. `iasi.quarto` actúa como capa de ingeniería.

3.2 El problema que resuelve

En un proyecto pequeño, mantener manualmente `_quarto.yml` puede ser suficiente. Cuando el número de documentos aumenta, esa configuración empieza a concentrar demasiado conocimiento:

- qué archivos forman parte del libro;
- en qué orden deben aparecer;
- cómo se organizan los capítulos;
- qué perfiles existen;
- qué formato debe generarse;
- dónde se escriben los resultados.

`iasi.quarto` convierte parte de ese conocimiento implícito en convenciones verificables.

La estructura del proyecto deja de ser una colección de decisiones manuales y pasa a ser información que el framework puede descubrir, validar y preparar.

3.3 Qué aporta

El flujo fundamental de `iasi.quarto` se divide en cuatro pasos:

```
project = discover()
validation = validate(project)
publication = prepare(validation)
render(publication)
```

En el uso habitual, `build()` coordina ese proceso completo.

Cada etapa tiene una responsabilidad concreta:

- `discover()` interpreta la estructura existente;
- `validate()` comprueba que el proyecto sea coherente;
- `prepare()` genera la configuración derivada necesaria;
- `render()` delega en Quarto la producción de los formatos finales.

Esta separación permite detectar los problemas antes de iniciar un renderizado y hace que cada transformación sea explícita.

3.4 Qué no pretende ser

`iasi.quarto` no es:

- un sustituto de Quarto;
- un nuevo lenguaje documental;
- un generador de contenido;
- una plantilla cerrada;
- una abstracción que oculte por completo la configuración de Quarto.

El proyecto sigue siendo un proyecto Quarto normal. Sus archivos pueden abrirse, editarse y renderizarse con las herramientas habituales.

La diferencia es que ciertas decisiones estructurales se expresan mediante convenciones que `iasi.quarto` puede comprender.

3.5 Una herramienta para proyectos que crecen

El valor de `iasi.quarto` aparece cuando la documentación deja de ser un conjunto de archivos aislados y empieza a comportarse como un sistema.

En ese momento importan el orden, la consistencia, la repetibilidad y la capacidad de modificar la estructura sin reconstruir manualmente toda la configuración.

`iasi.quarto` proporciona ese armazón sin quitarle a Quarto el papel central que le corresponde.

4 Fundamentos

4.1 La publicación IASI Quarto

Una **publicación IASI Quarto** es un proyecto Quarto cuya estructura, configuración y proceso de construcción siguen las convenciones definidas por `iasi.quarto`.

No es un tipo diferente de documento. Sigue siendo una publicación Quarto y puede utilizar sus formatos, extensiones y mecanismos habituales. `iasi.quarto` añade una capa de organización y automatización sobre ese proyecto:

- reconoce publicaciones;
- interpreta su estructura;
- comprueba su coherencia;
- genera los artefactos derivados;
- coordina su construcción.

La publicación continúa perteneciendo a Quarto. `iasi.quarto` no sustituye su motor, sino que organiza todo lo que ocurre antes de invocarlo.

4.1.1 La firma de una publicación

Una carpeta se reconoce como publicación IASI Quarto cuando contiene estos cuatro archivos:

```
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
iasi.yml
```

Los cuatro forman la **firma de la publicación**.

`_quarto.yml` contiene la configuración común del proyecto Quarto.

`_quarto-html.yml` contiene la configuración específica del perfil HTML.

`_quarto-pdf.yml` contiene la configuración específica del perfil PDF.

`iasi.yml` declara cómo debe interpretar y preparar la publicación `iasi.quarto`.

La coexistencia de los cuatro archivos es intencionada. La presencia aislada de `_quarto.yml` únicamente identifica un proyecto Quarto. La presencia aislada de `iasi.yml` tampoco es suficiente. Solo la firma completa identifica una publicación IASI Quarto.

Una estructura mínima puede ser:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml  
  index.qmd  
  chapters/
```

La firma permite reconocer el proyecto sin analizar todavía toda su configuración ni recorrer su contenido. El reconocimiento es deliberadamente superficial: primero se determina qué existe; después se estudia qué contiene.

4.1.2 Una publicación no es necesariamente un libro

`iasi.quarto` utiliza el término **publicación** para referirse a la unidad que puede comprobarse, prepararse y construirse.

Actualmente, el caso principal es un libro Quarto:

```
project:  
  type: book
```

Sin embargo, la publicación y el tipo de proyecto son conceptos diferentes.

La publicación es la unidad gestionada por `iasi.quarto`.

El tipo indica cómo entiende Quarto ese proyecto:

```
book  
website  
manuscript  
article
```

Esta separación evita incorporar en la arquitectura una equivalencia innecesaria:

```
publicación libro
```

Un libro es un tipo posible de publicación. La publicación es el concepto más general.

4.1.3 Publicación individual

Cuando la carpeta indicada contiene directamente la firma completa, se considera una **publicación individual**.

Por ejemplo:

```
user-guide/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml  
  index.qmd  
  chapters/
```

En este caso, la carpeta `user-guide/` es la raíz de la publicación.

La construcción se realiza desde esa raíz y todos los archivos de configuración, contenido y salida se interpretan en relación con ella.

```
validate("user-guide")
```

También puede validarse desde el propio directorio:

```
validate()
```

4.1.4 Multiproyecto

Una carpeta puede actuar como contenedor de varias publicaciones independientes.

```
docs/  
  user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

```
index.qmd
  chapters/

developer-guide/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
  index.qmd
  chapters/
```

La carpeta `docs/` no es una publicación. Es un **multiproyecto** que contiene dos publicaciones:

```
user-guide
developer-guide
```

Cada una conserva su propia configuración, estrategia, contenido y perfiles de salida.

El multiproyecto permite construirlas conjuntamente sin convertirlas en partes de un único libro.

```
validate("docs")
```

La validación reconoce el contenedor y localiza las publicaciones que contiene.

La raíz del multiproyecto no necesita disponer de su propio `iasi.yml`. Su función es agrupar publicaciones completas, no comportarse como una publicación adicional.

4.1.5 La publicación actual tiene prioridad

Cuando la carpeta indicada contiene la firma completa, se interpreta como una publicación individual aunque existan otras firmas en directorios descendientes.

```
publication/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
  index.qmd
  chapters/
```

```
examples/  
  sample-book/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

En este caso, `publication/` es la publicación actual. La publicación situada bajo `examples/` no se incorpora automáticamente al mismo plan de construcción.

Esta regla evita que ejemplos, plantillas, pruebas o proyectos auxiliares se confundan con publicaciones hermanas.

El reconocimiento sigue este orden:

1. comprobar si la carpeta actual contiene la firma completa;
2. si la contiene, tratarla como una publicación individual;
3. si no la contiene, buscar publicaciones completas en sus descendientes;
4. si se encuentran, tratar la carpeta como multiproyecto;
5. si no se encuentra ninguna, concluir que no es un proyecto IASI Quarto.

4.1.6 Publicación y contenido

La raíz de la publicación no obliga a utilizar un directorio llamado `chapters`.

El directorio de contenido se declara en `iasi.yml`:

```
publication:  
  content-dir: chapters
```

Podría utilizarse otro nombre:

```
publication:  
  content-dir: content
```

Y la estructura correspondiente sería:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml
```

```
index.qmd
content/
```

La publicación es la raíz completa del proyecto. El directorio de contenido es solo una de sus partes.

Esta distinción permite que una publicación contenga, además del texto principal:

```
images/
resources/
bibliography/
extensions/
scripts/
outputs/
```

`iasi.quarto` no redefine toda la estructura de Quarto. Solo necesita conocer dónde se encuentra el contenido que debe organizar.

4.1.7 Archivos fuente y archivos derivados

Dentro de una publicación existen dos clases de archivos.

Los **archivos fuente** pertenecen al autor:

```
index.qmd
iasi.yml
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
chapters/01-intro/01-introduction.qmd
```

Los **archivos derivados** son generados por `iasi.quarto` durante la preparación:

```
_book-structure.yml
chapters/01-intro/index.qmd
```

La lista exacta depende de la estrategia de publicación.

La diferencia es importante:

- los archivos fuente expresan decisiones editoriales;
- los archivos derivados materializan esas decisiones para Quarto.

Un archivo derivado puede regenerarse. No debe convertirse accidentalmente en la única copia de contenido escrito por el autor.

4.1.8 Reconocer no significa construir

El reconocimiento de una publicación no implica que sea válida ni que pueda construirse.

Una carpeta puede contener la firma completa y, aun así, presentar problemas:

```
iasi.yml mal formado
estrategia desconocida
directorio de contenido inexistente
index.qmd raíz ausente
estructura incompatible con la estrategia
```

Por eso `validate()` responde a una pregunta limitada:

¿Existe aquí una publicación IASI Quarto o un multiproyecto reconocible?

No genera archivos, no modifica el proyecto y no ejecuta Quarto.

```
validate()
```

La construcción completa corresponde a:

```
build()
```

Esta separación permite inspeccionar un proyecto antes de intervenir sobre él.

4.1.9 La publicación como unidad de construcción

Una publicación reúne todo lo necesario para ejecutar una construcción independiente:

```
configuración común
configuración por formato
declaración IASI
contenido
artefactos derivados
salidas
```

En un multiproyecto, cada publicación mantiene esa independencia.

Esto permite construir:

- todas las publicaciones;
- una publicación concreta;
- todos los formatos;
- un formato determinado.

Por ejemplo:

```
build(  
  book = "user-guide",  
  format = "html"  
)
```

La selección pertenece al proceso de construcción. La publicación, por su parte, continúa siendo una unidad completa y autosuficiente.

Esa es la frontera fundamental del modelo:

Una publicación IASI Quarto es una unidad Quarto reconocible, configurable y construible de forma independiente.

4.2 Estructura de un proyecto

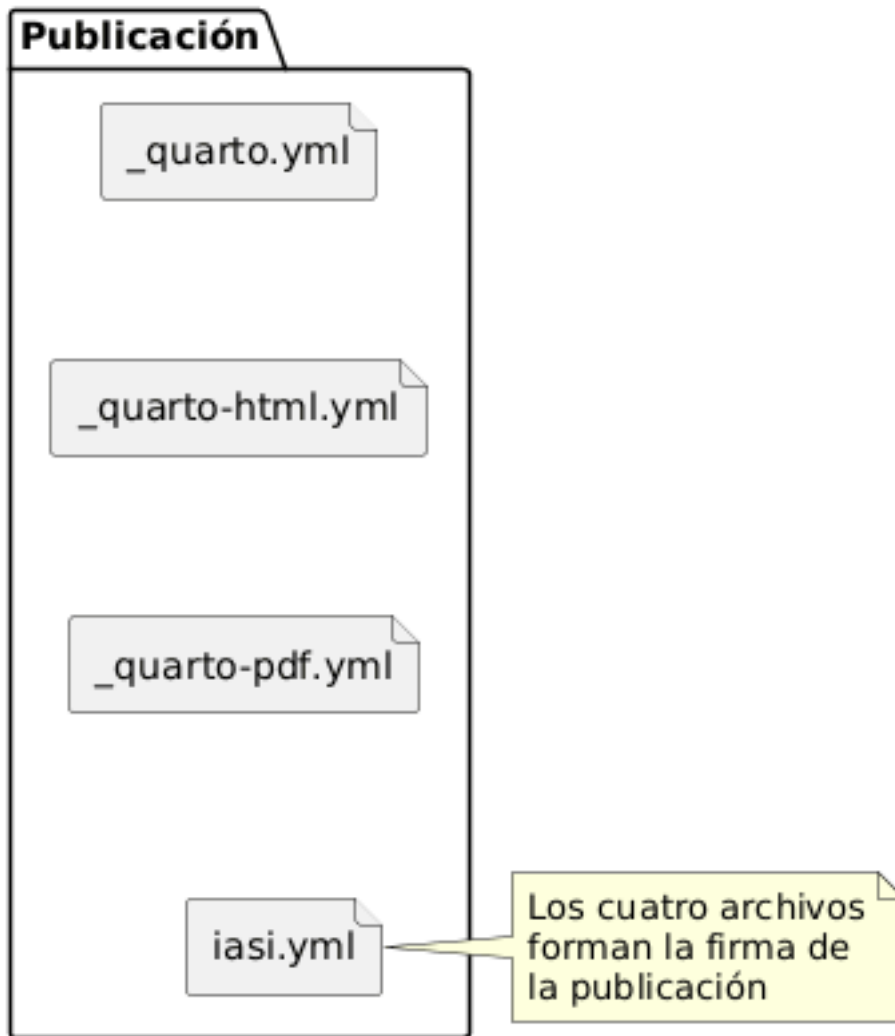
Una publicación IASI Quarto sigue siendo un proyecto Quarto.

`iasi.quarto` no sustituye su estructura ni introduce un contenedor documental propio. Añade una convención mínima que permite descubrir la publicación, comprobarla, preparar sus artefactos derivados y delegar finalmente el renderizado en Quarto.

La estructura mínima de una publicación es:

```
publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml
```

La presencia conjunta de estos cuatro archivos identifica una publicación IASI Quarto.



4.3 Configuración general y configuración por formato

El archivo `_quarto.yml` contiene la configuración común de Quarto:

```
project:
  type: book

profile:
  group:
    - [html, pdf]
```

```
metadata-files:
- _book-structure.yml
```

Las configuraciones específicas de cada salida se mantienen separadas:

```
_quarto-html.yml
  configuración de la publicación HTML

_quarto-pdf.yml
  configuración de la publicación PDF
```

Esta separación permite que una misma publicación produzca distintos formatos sin mezclar sus decisiones particulares.

Por ejemplo, el HTML puede utilizar una hoja de estilos CSS, mientras que el PDF puede declarar opciones específicas de LaTeX.

4.4 Configuración de `iasi.quarto`

El archivo `iasi.yml` describe cómo debe interpretar `iasi.quarto` el contenido de la publicación.

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Sus valores predeterminados son:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Por tanto, una publicación puede omitir las propiedades que no necesite modificar.

`iasi.yml` no contiene configuración de renderizado. Su responsabilidad es describir la organización editorial que `iasi.quarto` debe descubrir y preparar.

4.5 Directorio de contenido

La propiedad `content-dir` señala dónde se encuentran los documentos de la publicación.

```
publication:  
  content-dir: chapters
```

Con esta configuración, la estructura habitual es:

```
publication/  
  chapters/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

El nombre `chapters` es el valor predeterminado, pero no forma parte rígida del modelo.

También sería válida una publicación como:

```
publication/  
  content/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

si `iasi.yml` declara:

```
publication:  
  content-dir: content
```

4.6 Documentos de raíz

Los documentos situados en la raíz de la publicación pertenecen directamente al libro.

Por ejemplo:

```
publication/  
  index.qmd  
  manifesto.qmd  
  principles.qmd  
  licenses.qmd  
  chapters/
```

`iasi.quarto` conserva estos documentos y los incorpora antes o después del contenido organizado en carpetas, según su posición dentro de la estructura preparada.

El `index.qmd` de la raíz es siempre un documento del autor.

No debe confundirse con los archivos `index.qmd` que pueden existir dentro del directorio de contenido.

4.7 Carpetas de contenido

Cada carpeta inmediata de `content-dir` representa una unidad editorial.

```
chapters/  
  01-fundamentals/  
  02-installation/  
  03-usage/
```

El significado exacto de cada carpeta depende de la estrategia de publicación.

En una publicación **regular**, cada carpeta se convierte en un capítulo compuesto.

En una publicación **structured**, cada carpeta se convierte en una parte que contiene capítulos independientes.

La estructura física puede ser parecida, pero su interpretación editorial es distinta.

4.8 Estructura regular

Una carpeta **regular** contiene documentos parciales que se ensamblan en un único capítulo.

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

El archivo `front-matter.quarto` es opcional.

Cuando existe, contiene un front matter Quarto completo:

```
---  
title: "Fundamentals"  
title-block-style: none  
---
```

`iasi.quarto` lo copia literalmente al principio del `index.qmd` generado.

Los documentos `.qmd` contienen el cuerpo del capítulo:

```
## La publicación  
  
Contenido de la sección.
```

Durante la preparación, `iasi.quarto` genera:

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd  
    index.qmd
```

El resultado derivado tiene esta forma:

```
---  
title: "Fundamentals"  
title-block-style: none  
---  
  
<!-- Generated by iasi.quarto. Do not edit. -->
```

```
{{{< include 00-index.qmd >}}}
{{{< include 01-publication.qmd >}}}
{{{< include 02-project-structure.qmd >}}}
```

En esta estrategia:

```
el autor mantiene los documentos parciales
iasi.quarto mantiene el index.qmd de la carpeta
```

Si `front-matter.quarto` no existe, el capítulo se genera sin front matter. Quarto podrá mostrar `index.html` como nombre de navegación, haciendo visible que falta la metadata editorial.

4.9 Estructura structured

En una publicación **structured**, cada carpeta contiene capítulos independientes.

```
chapters/
  01-fundamentals/
    index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

El `index.qmd` de la carpeta pertenece al autor y actúa como introducción de la parte.

Los demás documentos son capítulos completos y comienzan normalmente con un encabezado de primer nivel:

```
# Estructura del proyecto
```

En esta estrategia, `iasi.quarto` no genera ni reemplaza el `index.qmd` de la carpeta.

```
regular:
  iasi.quarto mantiene el index.qmd

structured:
  el autor mantiene el index.qmd
```

4.10 Archivos fuente y archivos derivados

Una publicación contiene dos clases de archivos.

Los archivos fuente son mantenidos por el autor:

```
iasi.yml
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
front-matter.quarto
*.qmd
```

Los archivos derivados son producidos por `iasi.quarto`:

```
_book-structure.yml
index.qmd de las carpetas regular
```

Esta separación es importante.

Los archivos derivados pueden regenerarse en cualquier momento. No deben utilizarse como lugar de edición porque su contenido pertenece al proceso de preparación.



4.11 La estructura preparada

Quarto necesita una relación explícita de los capítulos que forman el libro.

`iasi.quarto` genera esa relación en:

```
_book-structure.yml
```

Por ejemplo:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-fundamentals/index.qmd"
    - "chapters/02-installation/index.qmd"
```

El archivo `_quarto.yml` incorpora esta estructura mediante:

```
metadata-files:  
  - _book-structure.yml
```

De esta forma, la configuración principal permanece estable mientras la estructura editorial puede regenerarse automáticamente.

4.12 Publicaciones múltiples

Un directorio superior puede contener varias publicaciones independientes:

```
workspace/  
  01-user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml  
  
  02-developer-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

El directorio `workspace` no necesita ser una publicación ni contener un `iasi.yml`.

`iasi.quarto` descubre recursivamente las carpetas que contienen la firma completa y trata cada una como una unidad de construcción independiente.

4.13 Una estructura explícita

La estructura de una publicación IASI Quarto no pretende ocultar Quarto.

Todos sus elementos continúan siendo visibles:

```
los documentos siguen siendo .qmd  
la configuración sigue siendo YAML  
los perfiles siguen perteneciendo a Quarto  
los formatos finales siguen siendo generados por Quarto
```

`iasi.quarto` añade una capa de organización y preparación, pero mantiene abiertas las piezas que forman la publicación.

La estructura no reemplaza el proyecto Quarto. Lo hace reconocible, comprobable y reproducible.

4.14 El archivo `iasi.yml`

`iasi.yml` describe cómo debe interpretar `iasi.quarto` una publicación.

No sustituye a `_quarto.yml` ni contiene opciones de renderizado. Su función es declarar la organización editorial que `iasi.quarto` debe descubrir, comprobar y preparar antes de delegar la generación final en Quarto.

La estructura actual es:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Las tres propiedades son opcionales. Cuando no se declaran, `iasi.quarto` utiliza sus valores predeterminados.

4.15 Valores predeterminados

La configuración mínima válida es:

```
publication:
```

También puede escribirse simplemente:

```
publication: {}
```

En ambos casos, el resultado efectivo es:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Por tanto, estas dos configuraciones son equivalentes:

```
publication: {}
```

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Los valores predeterminados permiten crear una publicación convencional sin repetir configuración que ya forma parte del modelo.

4.16 strategy

La propiedad **strategy** indica cómo debe interpretarse el contenido de cada carpeta inmediata del directorio de contenido.

```
publication:
  strategy: regular
```

Los valores admitidos son:

```
regular
structured
direct
```

4.16.1 regular

En la estrategia **regular**, cada carpeta representa un capítulo compuesto por varios documentos parciales.

```
chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

Durante la preparación, `iasi.quarto` genera:

```
chapters/01-fundamentals/index.qmd
```

Ese archivo reúne los documentos parciales mediante inclusiones Quarto.

```
{{< include 00-index.qmd >}}  
  
{{< include 01-publication.qmd >}}  
  
{{< include 02-project-structure.qmd >}}
```

En esta estrategia, el `index.qmd` de la carpeta es un artefacto derivado y pertenece a `iasi.quarto`.

4.16.2 structured

En la estrategia **structured**, cada carpeta representa una parte y sus documentos son capítulos independientes.

```
chapters/  
  01-fundamentals/  
    index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

El `index.qmd` de la carpeta pertenece al autor y actúa como introducción de la parte.

Los demás documentos se incorporan como capítulos independientes dentro de esa parte.

4.16.3 direct

La estrategia **direct** conserva una relación más directa entre los documentos encontrados y la estructura entregada a Quarto.

No genera capítulos compuestos y no asume que cada carpeta deba convertirse en una parte o en un capítulo ensamblado.

Se utiliza cuando la publicación ya controla explícitamente su organización o cuando no encaja en las convenciones de **regular** y **structured**.

4.17 content-dir

La propiedad `content-dir` indica dónde se encuentra el contenido organizado de la publicación.

```
publication:
  content-dir: chapters
```

El valor predeterminado es:

```
chapters
```

Con esa configuración, `iasi.quarto` busca las carpetas editoriales en:

```
publication/
  chapters/
```

Puede utilizarse otro nombre:

```
publication:
  content-dir: content
```

La publicación correspondiente podría ser:

```
publication/
  content/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
```

`content-dir` se interpreta como una ruta relativa a la raíz de la publicación.

No es una ruta relativa al directorio desde el que se ejecuta R ni al directorio de trabajo del usuario.

4.18 numbered

La propiedad `numbered` indica si las carpetas y los documentos fuente deben seguir una convención de prefijos numéricos.

```
publication:
  numbered: true
```

Cuando su valor es `true`, las carpetas deben comenzar por uno o más dígitos seguidos de un guion:

```
01-fundamentals
02-installation
10-reference
```

El patrón utilizado es:

```
^[0-9]+-
```

Los documentos fuente `.qmd` deben seguir la misma convención:

```
00-index.qmd
01-publication.qmd
02-project-structure.qmd
```

El patrón utilizado es:

```
^[0-9]+-.*\.qmd$
```

Los prefijos determinan el orden editorial porque el descubrimiento se apoya en el orden de los nombres.

Por ejemplo:

```
01-introduction.qmd
02-model.qmd
10-reference.qmd
```

se procesa en ese mismo orden.

4.18.1 `numbered: false`

Una publicación puede desactivar esta exigencia:

```
publication:
  numbered: false
```

En ese caso, `iasi.quarto` no exige prefijos numéricos en carpetas ni documentos.

```
chapters/
  fundamentals/
  installation/
  usage/
```

y:

```
introduction.qmd
publication.qmd
project-structure.qmd
```

siguen siendo nombres válidos.

Desactivar la validación no añade un mecanismo alternativo de ordenación. La publicación continúa dependiendo del orden lexicográfico de los nombres encontrados.

4.19 Configuración parcial

No es necesario declarar todas las propiedades.

Por ejemplo:

```
publication:
  strategy: structured
```

produce esta configuración efectiva:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

Del mismo modo:

```
publication:
  numbered: false
```

mantiene `regular` como estrategia y `chapters` como directorio de contenido.

`iasi.quarto` completa únicamente las propiedades ausentes. No sustituye valores que hayan sido declarados explícitamente.

4.20 Configuración válida completa

Una publicación regular convencional puede declarar:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
```

Una publicación estructurada puede declarar:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

Una publicación sin prefijos numéricos podría utilizar:

```
publication:
  strategy: regular
  content-dir: content
  numbered: false
```

4.21 YAML mal formado

`iasi.yml` debe ser un documento YAML válido.

Por ejemplo, esta configuración es incorrecta:

```
publication:
  strategy regular
```

Falta el separador `:` entre la propiedad y su valor.

Los errores sintácticos se detectan durante el descubrimiento, cuando `iasi.quarto` intenta leer el archivo.

La publicación no puede continuar hacia la comprobación ni hacia la preparación mientras el YAML no pueda interpretarse.

4.22 Valores semánticamente inválidos

Un documento puede ser YAML válido y, aun así, contener una configuración no admitida.

Por ejemplo:

```
publication:
  strategy: automatic
```

`automatic` es una cadena válida desde el punto de vista de YAML, pero no es una estrategia reconocida por `iasi.quarto`.

También es inválido:

```
publication:
  numbered: yes
```

Aunque algunas implementaciones históricas de YAML pueden interpretar `yes` como un valor lógico, el contrato de `iasi.quarto` espera una configuración inequívoca y debe expresarse como:

```
publication:
  numbered: true
```

o:

```
publication:
  numbered: false
```

Los valores semánticamente inválidos se rechazan durante la fase de comprobación.

4.23 Propiedades desconocidas

El contrato de `iasi.yml` es explícito.

Esta configuración contiene una propiedad desconocida:

```
publication:
  strategy: regular
  content-dir: chapters
  numbered: true
  language: es
```

`language` no pertenece al modelo de `iasi.quarto`.

El idioma de la publicación debe configurarse en Quarto:

```
lang: es
```

dentro de `_quarto.yml`.

La presencia de propiedades desconocidas se considera un error porque puede ocultar erratas o configuraciones que el autor cree activas cuando en realidad no tienen ningún efecto.

Por ejemplo:

```
publication:
  stratgy: regular
```

no debe ignorarse silenciosamente. La propiedad está mal escrita y la publicación debe informar del problema.

4.24 Separación de responsabilidades

Cada archivo de configuración tiene una responsabilidad distinta.

```
_quarto.yml
  configuración común de Quarto

_quarto-html.yml
  configuración específica de HTML

_quarto-pdf.yml
```

```
configuración específica de PDF

iasi.yml
  organización editorial interpretada por iasi.quarto
```

Por ejemplo, esta opción pertenece a `_quarto-html.yml`:

```
format:
  html:
    theme: cosmo
```

Esta pertenece a `_quarto-pdf.yml`:

```
format:
  pdf:
    toc: true
```

Y esta pertenece a `iasi.yml`:

```
publication:
  strategy: regular
```

Mantener esta separación evita que `iasi.quarto` se convierta en una segunda capa de configuración de Quarto.

4.25 Configuración y ciclo de construcción

`iasi.yml` participa en las primeras fases del proceso:

```
validate()
  reconoce la publicación por su firma

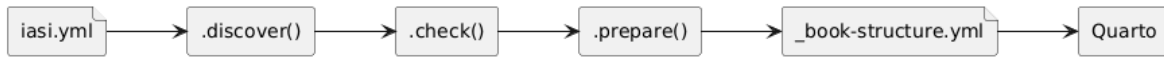
.discover()
  lee iasi.yml y aplica valores predeterminados

.check()
  comprueba propiedades, valores y estructura

.prepare()
  construye los artefactos derivados según la estrategia
```

La configuración no se interpreta durante el renderizado.

Cuando `.render()` se ejecuta, la publicación ya ha sido descubierta, comprobada y preparada.



4.26 Un contrato pequeño

`iasi.yml` contiene deliberadamente pocas propiedades.

No pretende describir toda la publicación ni duplicar la configuración disponible en Quarto.

Su contrato actual responde únicamente a tres preguntas:

```
¿Cómo se interpreta el contenido?  
  strategy  
  
¿Dónde se encuentra?  
  content-dir  
  
¿Debe seguir una convención numérica?  
  numbered
```

Esa limitación mantiene la configuración legible y permite que la mayor parte del proyecto siga siendo Quarto abierto y reconocible.

`iasi.yml` no describe cómo se renderiza una publicación. Describe cómo debe entenderse antes de renderizarla.

4.27 Estrategias de publicación

La estrategia de una publicación determina cómo interpreta `iasi.quarto` las carpetas y documentos situados dentro de `content-dir`.

Se declara en `iasi.yml`:

```
publication:  
  strategy: regular
```

El contrato actual admite tres estrategias:

```
regular
structured
direct
```

La estrategia se aplica a la publicación completa. No describe el formato de salida y no cambia entre HTML y PDF.

Su responsabilidad es editorial:

```
strategy
  determina cómo se transforma la estructura fuente
  en la estructura que Quarto debe renderizar
```

4.28 Una misma estructura física, distintas publicaciones

Dos publicaciones pueden contener carpetas y documentos parecidos, pero producir estructuras editoriales diferentes.

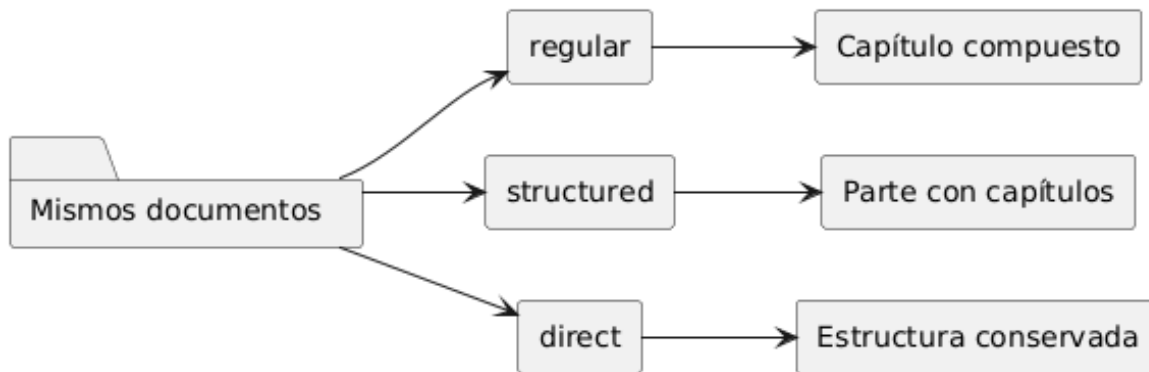
Por ejemplo:

```
chapters/
  01-fundamentals/
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

Con estrategia **regular**, esos documentos forman un único capítulo compuesto.

Con estrategia **structured**, cada documento puede convertirse en un capítulo independiente dentro de una parte.

La diferencia no reside únicamente en los nombres de archivo. Reside en quién controla la composición editorial y en qué artefactos debe preparar `iasi.quarto`.



4.29 Estrategia regular

`regular` es la estrategia predeterminada.

```
publication:  
  strategy: regular
```

Está pensada para redactar un capítulo mediante varios documentos parciales y pequeños. Cada carpeta inmediata de `content-dir` representa un capítulo.

```
chapters/  
  01-fundamentals/  
  02-installation/  
  03-usage/
```

Dentro de cada carpeta, los documentos `.qmd` representan secciones del capítulo:

```
chapters/  
  01-fundamentals/  
    front-matter.quarto  
    00-index.qmd  
    01-publication.qmd  
    02-project-structure.qmd  
    03-iasi-yml.qmd
```

Los documentos parciales comienzan normalmente en nivel dos:

```
## La publicación
```

```
Contenido de la sección.
```

No necesitan declarar un título de capítulo en nivel uno porque el capítulo se construye mediante el `index.qmd` generado.

4.29.1 Propiedad del `index.qmd`

En regular, el `index.qmd` de cada carpeta pertenece a `iasi.quarto`.

No debe mantenerse manualmente.

Durante `.prepare()`, el sistema genera un archivo con las inclusiones ordenadas:

```
<!-- Generated by iasi.quarto. Do not edit. -->

{{< include 00-index.qmd >}}

{{< include 01-publication.qmd >}}

{{< include 02-project-structure.qmd >}}

{{< include 03-iasi-yml.qmd >}}
```

El archivo generado es un artefacto derivado y puede recrearse en cualquier momento.

```
autor
  mantiene los documentos parciales

iasi.quarto
  mantiene el index.qmd compuesto
```

4.29.2 `front-matter.quarto`

Una carpeta regular puede contener:

```
front-matter.quarto
```

Este archivo es opcional y contiene un front matter Quarto completo:

```

---
title: "Fundamentals"
title-block-style: none
toc: true
---

```

iasi.quarto no interpreta cada propiedad por separado ni inventa valores que falten.

Copia el contenido literalmente al comienzo del `index.qmd` generado.

```

---
title: "Fundamentals"
title-block-style: none
toc: true
---

<!-- Generated by iasi.quarto. Do not edit. -->

{{< include 00-index.qmd >}}

```

Si el archivo no existe, el capítulo se genera sin front matter.

4.29.3 Estructura resultante

La estructura fuente:

```

chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd

```

se convierte durante la preparación en:

```

chapters/
  01-fundamentals/
    front-matter.quarto
    00-index.qmd
    01-publication.qmd
    02-project-structure.qmd
    index.qmd

```

Y `_book-structure.yml` incorpora únicamente el capítulo compuesto:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-fundamentals/index.qmd"
```

Los documentos parciales no aparecen como capítulos independientes en la estructura del libro.

4.29.4 Cuándo utilizar regular

regular es apropiada cuando:

```
un capítulo crece demasiado para mantenerse en un único archivo
se desea dividir la escritura sin dividir el capítulo editorial
las secciones deben compartir el mismo título y front matter
el autor prefiere documentos pequeños y ensamblables
```

El modelo permite separar la unidad de escritura de la unidad de publicación.

```
varios archivos fuente
  forman
un capítulo publicado
```

4.30 Estrategia structured

structured está pensada para libros en los que cada carpeta representa una parte y cada documento es un capítulo independiente.

```
publication:
  strategy: structured
```

La estructura habitual es:

```
chapters/
  01-fundamentals/
    index.qmd
    01-publication.qmd
    02-project-structure.qmd
    03-iasi-yml.qmd
```

En esta estrategia:

```
la carpeta
  representa una parte

index.qmd
  presenta la parte

cada otro .qmd
  representa un capítulo
```

4.30.1 Propiedad del index.qmd

En `structured`, el `index.qmd` de la carpeta pertenece al autor.

`iasi.quarto` no lo genera ni lo reemplaza.

El archivo puede contener la introducción de la parte:

```
# Fundamentals

Esta parte presenta el modelo fundamental de una publicación IASI Quarto.
```

Los demás documentos son capítulos completos y comienzan normalmente con un encabezado de nivel uno:

```
# La publicación

Contenido del capítulo.
```

La regla de propiedad es inversa a la estrategia `regular`:

```
regular
  iasi.quarto mantiene index.qmd

structured
  el autor mantiene index.qmd
```

4.30.2 Estructura resultante

La estructura fuente:

```
chapters/  
  01-fundamentals/  
    index.qmd  
    01-publication.qmd  
    02-project-structure.qmd
```

produce una estructura editorial equivalente a:

```
book:  
  chapters:  
    - "index.qmd"  
    - part: "chapters/01-fundamentals/index.qmd"  
      chapters:  
        - "chapters/01-fundamentals/01-publication.qmd"  
        - "chapters/01-fundamentals/02-project-structure.qmd"
```

La carpeta no se ensambla en un único documento.

Cada capítulo conserva su identidad dentro de la parte.

4.30.3 front-matter.quarto no participa

front-matter.quarto pertenece al modelo de composición de **regular**.

En **structured**, el autor controla directamente el front matter de cada documento:

```
---  
title: "La publicación"  
---  
  
# La publicación
```

No se necesita un archivo externo para construir el `index.qmd` porque ese archivo ya es fuente del autor.

4.30.4 Cuándo utilizar structured

structured es apropiada cuando:

```
el libro se organiza explícitamente en partes
cada documento debe aparecer como capítulo independiente
cada capítulo necesita su propio título o metadata
el autor quiere controlar el index.qmd introductorio de cada parte
```

El modelo coincide de forma directa con la estructura editorial clásica de un libro Quarto.

```
una carpeta
  contiene
una parte y sus capítulos
```

4.31 Estrategia direct

direct es la estrategia mínima.

```
publication:
  strategy: direct
```

Su objetivo es reducir al mínimo la transformación aplicada por `iasi.quarto`.

No construye capítulos compuestos como `regular` y no impone el modelo de partes de `structured`.

Los documentos descubiertos se conservan como unidades editoriales directas.

Por ejemplo:

```
chapters/
  01-introduction.qmd
  02-configuration.qmd
  03-reference.qmd
```

puede trasladarse a una estructura equivalente:

```
book:
  chapters:
    - "index.qmd"
    - "chapters/01-introduction.qmd"
    - "chapters/02-configuration.qmd"
    - "chapters/03-reference.qmd"
```

También puede utilizarse con carpetas cuando la publicación ya contiene documentos completos y no necesita que `iasi.quarto` genere índices de composición.

4.31.1 Qué no hace `direct`

La estrategia `direct` no debe entenderse como una estrategia que adivina la intención del autor.

No:

```
combina documentos parciales
crea títulos de capítulo
genera front matter
convierte carpetas automáticamente en partes
```

Su papel es conservar la estructura encontrada con la menor intervención posible.

4.31.2 Cuándo utilizar `direct`

`direct` es apropiada cuando:

```
la publicación ya posee documentos completos
la organización no encaja en regular ni structured
se desea utilizar iasi.quarto para validar y construir
sin introducir una composición editorial adicional
```

Es también una salida útil para publicaciones heredadas o estructuras deliberadamente simples.

4.32 Comparación

Las tres estrategias pueden resumirse así:

Estrategia	Carpeta inmediata	Documentos .qmd	index.qmd de carpeta	Preparación principal
regular	Capítulo compuesto	Secciones parciales	Generado por <code>iasi.quarto</code>	Ensamblar inclusiones
structured	Parte	Capítulos independientes	Mantenido por el autor	Construir parte y capítulos
direct	Contenedor opcional	Unidades directas	No se genera por convención	Conservar la estructura

La diferencia esencial es la unidad editorial que representa cada archivo.

```
regular
  varios archivos = un capítulo

structured
  varios archivos = varios capítulos dentro de una parte

direct
  cada archivo conserva su papel directo
```

4.33 Estrategia y numeración

La propiedad `numbered` es independiente de `strategy`.

Por ejemplo:

```
publication:
  strategy: regular
  numbered: false
```

permite:

```
chapters/
  fundamentals/
    introduction.qmd
    publication.qmd
    project-structure.qmd
```

Y:

```
publication:
  strategy: structured
  numbered: true
```

exige:

```
chapters/
  01-fundamentals/
    index.qmd
    01-publication.qmd
    02-project-structure.qmd
```

`strategy` define el significado editorial.

`numbered` define la convención de nombres y orden.

4.34 Estrategia y formatos de salida

La estrategia tampoco depende del formato final.

Una publicación puede construirse como HTML y PDF:

```
build()
```

o únicamente como HTML:

```
build(format = "html")
```

En ambos casos se utiliza la misma estructura preparada.

```
fuentes
-> estrategia
-> estructura editorial
-> HTML

fuentes
-> estrategia
-> estructura editorial
-> PDF
```

No existe una estrategia **regular** para HTML y otra **structured** para PDF.

La organización editorial se resuelve antes de renderizar los formatos.

4.35 Elección de estrategia

La elección puede reducirse a tres preguntas.

```
¿Varios archivos deben formar un solo capítulo?  
  regular  
  
¿Cada carpeta debe ser una parte con capítulos propios?  
  structured  
  
¿La estructura debe conservarse con mínima transformación?  
  direct
```

Cambiar de estrategia no es únicamente cambiar una opción.

Puede cambiar:

```
el significado de las carpetas  
el nivel de los encabezados  
la propiedad de los index.qmd  
la estructura generada para Quarto
```

Por eso la estrategia debe elegirse según la unidad editorial deseada, no según la comodidad momentánea de los nombres de archivo.

4.36 Una decisión editorial explícita

Las estrategias no intentan inferir qué quiso decir el autor.

Lo declaran.

```
publication:  
  strategy: regular
```

convierte una convención implícita en un contrato verificable.

`iasi.quarto` puede entonces descubrir la estructura, comprobarla y generar únicamente los artefactos que corresponden a esa decisión.

La estrategia no define dónde están los archivos. Define qué significan.

4.37 Ciclo de construcción

`iasi.quarto` organiza la construcción de una publicación como una secuencia de etapas explícitas.

La entrada pública principal es:

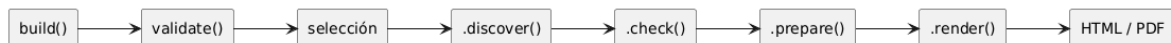
```
build()
```

Internamente, la construcción sigue este flujo:

```
build()
-> validate()
-> seleccionar publicaciones
-> .discover()
-> .check()
-> .prepare()
-> resolver formatos
-> .render()
```

Cada etapa tiene una responsabilidad concreta.

No todas modifican archivos y no todas necesitan conocer los mismos detalles de la publicación.



4.38 `build()` como orquestador

`build()` es la operación pública que coordina el proceso completo.

```
build()
```

Puede construir:

```
una publicación actual
varias publicaciones dentro de un multiproyecto
todos los formatos soportados
un formato seleccionado
```

Por ejemplo:

```
build(format = "html")
```

o:

```
build(  
  book = "user-guide",  
  format = "pdf"  
)
```

`build()` no concentra toda la lógica en una única función.

Delega cada decisión en la etapa que corresponde:

```
reconocer el espacio de trabajo  
  validate()  
  
leer la configuración  
  .discover()  
  
comprobar el contrato  
  .check()  
  
generar artefactos derivados  
  .prepare()  
  
invocar Quarto  
  .render()
```

Esto permite que el proceso falle en el punto correcto y que cada etapa pueda entenderse de forma independiente.

4.39 Primera etapa: `validate()`

`validate()` reconoce si una ruta contiene una publicación IASI Quarto o un multiproyecto que puede contener varias.

```
plan = validate(".")
```

La función busca la firma de publicación:

```
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
iasi.yml
```

Si los cuatro archivos existen en la ruta actual, esa ruta se considera una publicación.

Si no existen juntos, `validate()` puede reconocer la ruta como un contenedor desde el que después se localizarán publicaciones completas.

El resultado es un plan:

```
class(plan)
```

```
"iasi_quarto_plan"  
"list"
```

El plan contiene inicialmente:

```
plan$path  
plan$current  
plan$books
```

4.39.1 Publicación actual

Cuando la propia ruta validada contiene la firma completa:

```
plan$current = TRUE
```

La publicación actual tiene prioridad.

No se buscan publicaciones anidadas porque la ruta ya representa una unidad de construcción completa.

4.39.2 Multiproyecto

Cuando la ruta no es una publicación, pero contiene publicaciones descendientes:

```
plan$current = FALSE
```

El plan actúa como contenedor de las publicaciones que podrán seleccionarse y descubrirse.

`validate()` reconoce el terreno.

Todavía no interpreta `iasi.yml` ni genera artefactos.

4.40 Selección de publicaciones

En un multiproyecto, la selección ocurre después de `validate()` y antes de `.discover()`.

```
validate()  
  -> seleccionar publicaciones  
  -> .discover()
```

Este orden es importante.

Si el usuario solicita una publicación concreta:

```
build(book = "user-guide")
```

solo esa publicación debe continuar por el pipeline.

Una publicación no seleccionada no debe bloquear la construcción aunque contenga una configuración incompleta o inválida.

La selección admite:

```
nombre completo del directorio  
nombre sin prefijo numérico  
prefijo numérico
```

Por ejemplo, una carpeta:

```
01-user-guide
```

puede seleccionarse mediante:

```
build(book = "01-user-guide")
```

```
build(book = "user-guide")
```

```
build(book = "1")
```

En una publicación actual, el argumento `book` no es necesario porque la unidad de construcción ya está determinada.

4.41 Segunda etapa: `.discover()`

`.discover()` transforma el plan validado en un modelo enriquecido de las publicaciones seleccionadas.

```
plan = .discover(plan)
```

Durante esta fase se leen:

```
_quarto.yml  
iasi.yml
```

y se determina:

```
nombre de la publicación  
ruta  
tipo Quarto  
estrategia  
directorio de contenido  
uso de numeración  
configuración efectiva
```

Cada publicación descubierta se representa mediante un proyecto:

```
project$name  
project$path  
project$quarto_file  
project$iasi_file  
project$type
```

```
project$strategy
project$content_dir
project$content_path
project$numbered
project$quarto
project$iasi
```

La clase del objeto es:

```
iasiquarto_project
```

4.41.1 Aplicación de valores predeterminados

`.discover()` completa las propiedades ausentes de `iasi.yml`.

Por ejemplo:

```
publication:
  strategy: structured
```

se descubre como:

```
publication:
  strategy: structured
  content-dir: chapters
  numbered: true
```

La etapa distingue entre:

```
configuración ausente
  se completa con valores predeterminados
```

```
YAML ilegible
  detiene el descubrimiento
```

Un archivo que no puede interpretarse como YAML no puede convertirse en un proyecto descubierto.

4.42 Tercera etapa: `.check()`

`.check()` verifica que las publicaciones descubiertas respeten el contrato de `iasi.quarto`.

```
plan = .check(plan)
```

La comprobación se ocupa de aspectos semánticos y estructurales.

Por ejemplo:

```
estrategia soportada  
content-dir válido  
numbered lógico  
propiedades conocidas  
convenciones de nombres  
estructura coherente con la estrategia
```

Un YAML puede estar bien formado y, aun así, ser incorrecto:

```
publication:  
  strategy: automatic
```

`.discover()` puede leerlo, pero `.check()` debe rechazarlo porque `automatic` no es una estrategia soportada.

4.42.1 Resultado de la comprobación

Cada proyecto recibe un estado:

```
project$valid
```

El plan resume el resultado:

```
plan$valid
```

La construcción solo continúa si todas las publicaciones seleccionadas son válidas.

```
plan$valid = TRUE
-> continúa hacia .prepare()

plan$valid = FALSE
-> se detiene antes de generar artefactos
```

Esta separación evita que una publicación incorrecta llegue hasta Quarto y produzca errores más tardíos y menos precisos.

4.43 Cuarta etapa: `.prepare()`

`.prepare()` genera la estructura derivada que Quarto necesita para renderizar la publicación.

```
plan = .prepare(plan)
```

Esta es la primera etapa que puede escribir archivos.

Los artefactos generados dependen de la estrategia.

4.43.1 Preparación regular

En una publicación regular, `.prepare()` genera el `index.qmd` de cada carpeta editorial.

```
chapters/01-fundamentals/index.qmd
```

Ese archivo:

```
copia front-matter.quarto si existe
añade la cabecera de archivo generado
incluye los documentos parciales en orden
```

También genera:

```
_book-structure.yml
```

con los capítulos compuestos que Quarto debe procesar.

4.43.2 Preparación structured

En una publicación `structured`, `.prepare()` conserva los `index.qmd` mantenidos por el autor.

No los reemplaza.

Su trabajo principal consiste en construir `_book-structure.yml` con:

```
partes
introducciones de parte
capítulos independientes
```

4.43.3 Preparación direct

En una publicación `direct`, `.prepare()` aplica la mínima transformación necesaria.

No construye capítulos compuestos ni convierte automáticamente cada carpeta en una parte.

4.43.4 Artefactos

La publicación preparada mantiene información sobre los archivos derivados:

```
publication$chapters
publication$artifacts
publication$changed
```

`publication$changed` indica si la preparación tuvo que modificar algún artefacto.

4.44 Idempotencia de la preparación

La preparación debe ser idempotente.

Esto significa que ejecutar dos veces el mismo proceso sobre las mismas fuentes no debe reescribir continuamente los mismos archivos.

```
build()
build()
```

Si nada ha cambiado:

```
primera ejecución
  puede generar artefactos

segunda ejecución
  encuentra el mismo contenido
  no vuelve a escribirlo
```

La escritura se realiza únicamente cuando el contenido nuevo es distinto del existente.

Esto reduce:

```
cambios innecesarios en Git
marcas de tiempo artificiales
reconstrucciones provocadas por archivos idénticos
ruido en el proceso
```

La idempotencia convierte los artefactos derivados en resultados reproducibles, no en residuos de cada ejecución.

4.45 Resolución de formatos

Después de preparar la publicación, `build()` determina los formatos que deben renderizarse.

Los formatos soportados actualmente son:

```
html
pdf
```

Cuando no se indica ninguno:

```
build()
```

se construyen todos los formatos soportados.

También puede elegirse uno:

```
build(format = "html")
```

o:

```
build(format = "pdf")
```

La resolución de formatos pertenece al orquestador público.

`.render()` recibe un único formato ya resuelto y no vuelve a validarlo.

```
build()  
  decide qué formatos  
  
.render()  
  ejecuta un formato
```

4.46 Quinta etapa: `.render()`

`.render()` es una operación privada y mínima.

Recibe:

```
una publicación preparada  
un formato resuelto
```

Su contrato conceptual es:

```
.render(publication, format)
```

Por ejemplo:

```
publicación 01-user-guide  
formato html
```

representa una ejecución.

La misma publicación en PDF representa otra:

```
publicación 01-user-guide  
formato pdf
```

En un multiproyecto con dos publicaciones y dos formatos, `build()` coordina cuatro renderizados:

```
01-user-guide      -> html
01-user-guide      -> pdf
02-developer-guide -> html
02-developer-guide -> pdf
```

4.46.1 Delegación en Quarto

`.render()` no implementa un motor documental propio.

Invoca Quarto con el perfil y formato correspondientes:

```
quarto render --profile html --to html
```

o:

```
quarto render --profile pdf --to pdf
```

La publicación ya debe estar validada, descubierta, comprobada y preparada.

`.render()` no vuelve hacia atrás para resolver problemas de configuración o estructura.

4.46.2 Resultado

Después de un renderizado correcto, la publicación registra:

```
publication$rendered
publication$profiles
```

Por ejemplo:

```
publication$rendered = TRUE
publication$profiles = c("html", "pdf")
```

4.47 Fallar antes de renderizar

Una propiedad importante del ciclo es que los errores se detecten lo antes posible.

```
ruta no reconocible
  validate()

YAML ilegible
  .discover()

configuración o estructura inválida
  .check()

problema al generar artefactos
  .prepare()

fallo real de Quarto
  .render()
```

Cada problema pertenece a una etapa.

Esto evita utilizar el renderizado como mecanismo genérico de validación.

Quarto debe recibir una publicación ya preparada, no una colección de decisiones pendientes.

4.48 Ciclo de una publicación individual

Para una publicación actual:

```
publication/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
```

el flujo es:

```
build()
  reconoce la publicación
  descubre su configuración
  comprueba su estructura
  prepara sus artefactos
  renderiza HTML y PDF
```

No existe selección entre publicaciones porque la ruta actual ya es la unidad de construcción.

4.49 Ciclo de un multiproyecto

Para:

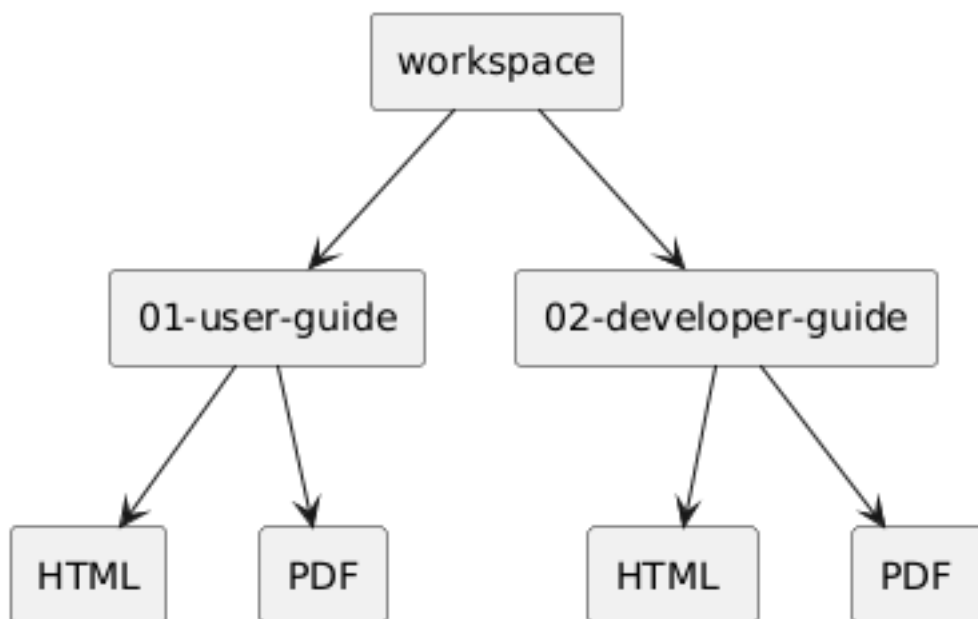
```
workspace/  
  01-user-guide/  
  02-developer-guide/
```

el flujo es:

```
build()  
  reconoce el workspace  
  selecciona publicaciones  
  descubre solo las seleccionadas  
  comprueba cada una  
  prepara cada una  
  renderiza cada combinación de publicación y formato
```

La unidad de trabajo de `.render()` sigue siendo una publicación y un formato.

El multiproyecto solo amplía la orquestación.



4.50 Estado acumulado

El plan se enriquece durante el proceso.

```
validate()  
  crea el plan  
  
.discover()  
  añade proyectos y configuración  
  
.check()  
  añade el resultado de comprobación  
  
.prepare()  
  añade publicaciones y artefactos  
  
.render()  
  añade formatos renderizados
```

El resultado final de `build()` no es únicamente un código de salida.

Es el plan completado, con información sobre lo que se descubrió, preparó y renderizó.

```
plan = build()
```

Aunque `build()` lo devuelve de forma invisible, el objeto puede conservarse para inspección.

4.51 Un pipeline deliberado

El ciclo de construcción separa reconocimiento, interpretación, comprobación, preparación y ejecución.

```
validate  
  ¿es un espacio IASI Quarto?  
  
discover  
  ¿qué contiene y cómo está configurado?  
  
check  
  ¿es coherente con el contrato?
```

```
prepare
  ¿qué artefactos necesita Quarto?

render
  ¿puede Quarto producir el formato solicitado?
```

Cada etapa reduce incertidumbre antes de pasar a la siguiente.

`build()` no es un único salto desde los fuentes hasta el PDF. Es una cadena de decisiones comprobables.

4.52 API pública

La API pública de `iasi.quarto` es deliberadamente pequeña.

Actualmente expone dos funciones:

```
validate()
build()
```

El resto del ciclo de construcción permanece como implementación interna:

```
.discover()
.check()
.prepare()
.render()
```

Esta separación establece una frontera clara entre:

```
lo que utiliza quien publica
lo que coordina internamente el paquete
```

La API pública no reproduce cada fase del pipeline como una operación independiente. Ofrece únicamente los puntos de entrada necesarios para reconocer una publicación y construirla.

4.53 `validate()`

`validate()` reconoce una publicación IASI Quarto o un multiproyecto.

```
plan = validate()
```

También puede recibir una ruta explícita:

```
plan = validate("path/to/workspace")
```

Su firma conceptual es:

```
validate(path = ".")
```

La función comprueba la presencia de la firma IASI Quarto:

```
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
iasi.yml
```

Cuando los cuatro archivos están presentes en la ruta indicada, esa ruta se reconoce como una publicación actual.

Cuando no están presentes juntos, la ruta puede reconocerse como un contenedor desde el que posteriormente se descubrirán publicaciones completas.

4.54 Resultado de `validate()`

Cuando la ruta corresponde a un espacio IASI Quarto, `validate()` devuelve invisiblemente un plan:

```
class(plan)
```

```
"iasi_quarto_plan"  
"list"
```

El plan inicial contiene:

```
plan$path  
plan$current  
plan$books
```

4.54.1 `plan$path`

Contiene la ruta normalizada utilizada como raíz del proceso.

```
plan$path
```

Representa la publicación actual o el multiproyecto validado.

4.54.2 `plan$current`

Indica si la propia ruta es una publicación completa:

```
plan$current
```

```
TRUE
```

```
la ruta actual contiene la firma completa
```

```
FALSE
```

```
la ruta actúa como contenedor de publicaciones
```

Cuando `current` es `TRUE`, la publicación actual tiene prioridad y no se buscan publicaciones anidadas.

4.54.3 `plan$books`

Contiene las publicaciones candidatas localizadas desde un multiproyecto.

En una publicación actual, la unidad de construcción ya está determinada y no es necesario seleccionar entre varios libros.

4.55 Ruta no reconocida

Cuando la ruta no parece una publicación ni un multiproyecto IASI Cuarto, `validate()` devuelve:

```
NULL
```

Por ejemplo:

```
plan = validate("empty-directory")  
  
is.null(plan)
```

```
TRUE
```

Esto permite utilizar `validate()` como operación de reconocimiento sin iniciar una construcción completa.

4.56 Qué hace `validate()`

`validate()` responde a una pregunta limitada:

¿La ruta parece contener un espacio de trabajo IASI Quarto?

No:

```
interpreta por completo iasi.yml  
comprueba toda la estructura editorial  
genera index.qmd  
genera _book-structure.yml  
invoca Quarto
```

Esas tareas pertenecen a etapas posteriores del pipeline.

La función valida la frontera exterior del espacio de trabajo, no toda su semántica interna.

4.57 Ejemplo con una publicación

Dada esta estructura:

```
user-guide/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml
```

la llamada:

```
plan = validate("user-guide")
```

produce conceptualmente:

```
plan$current = TRUE  
plan$path    = ".../user-guide"
```

4.58 Ejemplo con un multiproyecto

Dada esta estructura:

```
workspace/  
  01-user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml  
  
  02-developer-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

la llamada:

```
plan = validate("workspace")
```

produce un plan de multiproyecto:

```
plan$current = FALSE
```

y conserva las publicaciones candidatas para la selección posterior.

4.59 build()

build() es el punto de entrada principal del paquete.

```
build()
```

Su firma pública es:

```
build(  
    book = NULL,  
    format = NULL,  
    path = "."  
)
```

La función ejecuta el ciclo completo:

```
validate()  
selección de publicaciones  
.discover()  
.check()  
.prepare()  
.render()
```

`build()` puede operar sobre:

```
la publicación actual  
todas las publicaciones de un multiproyecto  
una publicación seleccionada  
todos los formatos soportados  
un formato concreto
```

4.60 Construcción predeterminada

La llamada más sencilla es:

```
build()
```

Cuando se ejecuta dentro de una publicación:

```
construye la publicación actual  
genera HTML  
genera PDF
```

Cuando se ejecuta en un multiproyecto:

```
construye todas las publicaciones descubiertas  
genera todos los formatos soportados
```

Los formatos soportados actualmente son:

```
html  
pdf
```

4.61 Argumento path

path indica la publicación o multiproyecto que debe construirse.

```
build(path = "path/to/publication")
```

Su valor predeterminado es:

```
path = "."
```

Por tanto, build() trabaja normalmente sobre el directorio actual.

Ejemplos:

```
build(path = "01-user-guide")
```

```
build(path = "workspace")
```

La ruta se valida antes de iniciar las demás etapas.

Si no corresponde a un espacio IASI Quarto, build() devuelve invisiblemente:

```
NULL
```

4.62 Argumento format

format selecciona el formato o formatos de salida.

Puede utilizarse:

```
build(format = "html")
```

```
build(format = "pdf")
```

También puede indicarse:

```
build(format = "all")
```

Los valores:

```
NULL
```

y:

```
"all"
```

seleccionan todos los formatos soportados.

Conceptualmente:

```
format = NULL  
  -> html y pdf  
  
format = "all"  
  -> html y pdf  
  
format = "html"  
  -> solo html  
  
format = "pdf"  
  -> solo pdf
```

Un formato no soportado produce un error antes del renderizado:

```
build(format = "docx")
```

```
Unsupported format: "docx".
```

La validación de formatos pertenece a `build()`.

`.render()` recibe siempre un formato ya resuelto.

4.63 Argumento book

book selecciona una o varias publicaciones dentro de un multiproyecto.

Por ejemplo:

```
build(book = "user-guide")
```

La selección admite tres formas.

4.63.1 Nombre completo

```
build(book = "01-user-guide")
```

4.63.2 Nombre sin prefijo numérico

```
build(book = "user-guide")
```

4.63.3 Prefijo numérico

```
build(book = "1")
```

Las tres expresiones pueden seleccionar:

```
01-user-guide
```

También pueden solicitarse varias publicaciones:

```
build(  
  book = c(  
    "user-guide",  
    "developer-guide"  
  )  
)
```

4.64 `book = NULL` y `book = "all"`

Cuando `book` no se indica:

```
build(book = NULL)
```

se construyen todas las publicaciones disponibles en el multiproyecto.

También puede expresarse:

```
build(book = "all")
```

En una publicación actual, la selección mediante `book` no es necesaria.

La ruta ya identifica la única publicación que debe construirse.

4.65 Selecciones no encontradas

Cuando una selección no existe, `build()` informa mediante una advertencia.

Por ejemplo:

```
build(  
  book = c(  
    "user-guide",  
    "missing"  
  )  
)
```

puede continuar construyendo `user-guide` y advertir:

```
Books not found: "missing".
```

Una publicación no encontrada no se inventa ni se sustituye por otra coincidencia aproximada.

4.66 Selección antes del descubrimiento

La selección se aplica antes de `.discover()`.

```
validate()  
  -> selección  
  -> .discover()
```

Esto tiene una consecuencia importante.

Una publicación no seleccionada no participa en:

```
lectura de iasi.yml  
comprobación  
preparación  
renderizado
```

Por tanto, una publicación experimental o incompleta no bloquea la construcción de otra publicación seleccionada.

4.67 Ejemplos de uso

4.67.1 Construir la publicación actual

```
build()
```

4.67.2 Construir solo HTML

```
build(format = "html")
```

4.67.3 Construir solo PDF

```
build(format = "pdf")
```

4.67.4 Construir un multiproyecto completo

```
build(path = "workspace")
```

4.67.5 Construir una publicación del multiproyecto

```
build(  
  path = "workspace",  
  book = "user-guide"  
)
```

4.67.6 Construir una publicación y un formato

```
build(  
  path = "workspace",  
  book = "user-guide",  
  format = "html"  
)
```

4.67.7 Construir varias publicaciones

```
build(  
  path = "workspace",  
  book = c(  
    "1",  
    "developer-guide"  
  ),  
  format = "pdf"  
)
```

4.68 Resultado de build()

build() devuelve invisiblemente el plan completado:

```
plan = build()
```

El objeto conserva información acumulada durante el pipeline.

Por ejemplo:

```
plan$path  
plan$current  
plan$projects  
plan$valid  
plan$formats  
plan$rendered  
plan$elapsed
```

Cada proyecto puede contener su publicación preparada:

```
plan$projects[[1]]$publication
```

y esta puede registrar:

```
publication$chapters  
publication$artifacts  
publication$changed  
publication$rendered  
publication$profiles
```

La devolución invisible permite dos formas de uso.

Uso habitual:

```
build()
```

Uso con inspección:

```
plan = build()  
  
plan$formats  
plan$elapsed
```

4.69 Mensajes de construcción

Aunque el plan se devuelve invisiblemente, `build()` informa del proceso mediante mensajes.

La salida puede mostrar:

```
publicaciones reconocidas
estado de comprobación
formatos contruidos
resultado final
tiempo empleado
```

Los mensajes están dirigidos a la persona que ejecuta la construcción.

El objeto devuelto está dirigido a la inspección programática.

4.70 API pública y funciones privadas

Las fases internas no forman parte del contrato público:

```
.discover()
.check()
.prepare()
.render()
```

El prefijo `.` expresa que son funciones privadas del paquete.

No deben utilizarse como puntos de entrada estables desde código externo.

Por ejemplo, esto pertenece a la implementación:

```
plan = .discover(plan)
```

La forma pública de ejecutar el proceso es:

```
build()
```

La diferencia no es meramente estética.

Las funciones privadas pueden cambiar su estructura interna mientras se conserva el contrato de:

```
validate()
build()
```

4.71 Por qué no se expone cada etapa

Una API con todas las fases públicas obligaría a quien la utiliza a conocer:

```
el orden correcto
los tipos intermedios
las precondiciones de cada función
los cambios internos del pipeline
```

Por ejemplo, no tendría sentido ejecutar `.prepare()` sobre un plan que todavía no ha pasado por `.discover()` y `.check()`.

`build()` conserva esas precondiciones dentro del paquete.

```
usuario
  llama a build()

build()
  garantiza el orden correcto
```

Esto reduce la superficie pública sin ocultar el modelo conceptual.

Las etapas siguen existiendo y pueden documentarse, pero no se convierten en obligaciones para quien utiliza el paquete.

4.72 `validate()` frente a `build()`

La elección entre las dos funciones públicas es sencilla.

```
validate()
  reconoce y devuelve un plan inicial

build()
  ejecuta la construcción completa
```

Utilice `validate()` cuando necesite comprobar qué representa una ruta:

```
plan = validate("workspace")
```

Utilice `build()` cuando necesite generar la publicación:

```
build("workspace")
```

Con argumentos nombrados, la llamada correcta es:

```
build(path = "workspace")
```

porque el primer argumento de `build()` es `book`, no `path`.

4.73 Llamadas con argumentos nombrados

Para evitar ambigüedades, es recomendable nombrar los argumentos cuando se proporciona una ruta:

```
build(path = "workspace")
```

y no:

```
build("workspace")
```

La segunda expresión se interpreta como:

```
book = "workspace"
```

porque la firma es:

```
build(  
  book = NULL,  
  format = NULL,  
  path = "."  
)
```

Del mismo modo:

```
build(  
  book = "user-guide",  
  format = "html",  
  path = "workspace"  
)
```

hace explícita cada decisión.

4.74 Contrato estable, implementación evolutiva

La API pública protege al usuario de la evolución interna del paquete.

El pipeline puede incorporar en el futuro:

```
nuevas comprobaciones  
nuevos artefactos  
nuevos formatos  
nuevas estrategias
```

sin exigir que el código externo coordine manualmente cada cambio.

La frontera se mantiene pequeña:

```
validate()  
build()
```

La API pública expresa la intención. El pipeline privado resuelve el procedimiento.

5 Instalación

5.1 Instalación

`iasi.quarto` se distribuye como un paquete de R.

Su instalación añade al entorno dos operaciones públicas:

```
validate()  
build()
```

La primera reconoce una publicación IASI Quarto. La segunda ejecuta su ciclo completo de construcción.

El paquete no sustituye a Quarto ni instala por sí mismo sus herramientas externas. Trabaja sobre un entorno en el que R y Quarto ya están disponibles.



La instalación se divide en tres pasos:

```
comprobar los requisitos  
instalar iasi.quarto  
verificar el entorno
```

En primer lugar, se comprueba que R y Quarto puedan ejecutarse correctamente.

Después se instala el paquete desde su repositorio.

Por último, se realiza una comprobación mínima para confirmar que `iasi.quarto` puede cargarse y reconocer una publicación.

Este capítulo no pretende explicar cómo instalar R o Quarto en cada sistema operativo. Su objetivo es dejar preparado el entorno específico que necesita `iasi.quarto`.

Al finalizar, debe ser posible ejecutar:

```
library(iasi.quarto)

validate()
```

y obtener un plan cuando el directorio actual contiene una publicación o un multiproyecto IASI Quarto.

La instalación prepara la herramienta. La estructura y la configuración de la publicación siguen perteneciendo al proyecto.

5.2 Requisitos

`iasi.quarto` necesita dos componentes para funcionar:

```
R
Quarto
```

R ejecuta el paquete.

Quarto procesa la publicación y genera los formatos finales.

Para construir PDF también se necesita una distribución de TeX disponible para Quarto.



5.3 R

`iasi.quarto` es un paquete de R y debe instalarse en una biblioteca accesible desde la sesión que ejecutará la construcción.

Puede comprobarse la instalación de R desde la consola:

```
R.version.string
```

El resultado debe mostrar la versión activa de R.

También conviene comprobar dónde se instalarán y buscarán los paquetes:

```
.libPaths()
```

No es necesario utilizar RStudio.

El paquete puede ejecutarse desde:

```
RStudio  
la consola de R  
Rscript  
un proceso automatizado  
un pipeline de integración continua
```

RStudio es un entorno de trabajo posible, no una dependencia de `iasi.quarto`.

5.4 Quarto

`iasi.quarto` delega el renderizado final en la línea de comandos de Quarto.

Por tanto, el ejecutable `quarto` debe estar instalado y disponible para el proceso de R.

Desde una terminal:

```
quarto --version
```

Desde R:

```
Sys.which("quarto")
```

El resultado de `Sys.which()` debe contener una ruta.

Por ejemplo:

```
quarto  
"C:\Program Files\Quarto\bin\quarto.exe"
```

Si devuelve una cadena vacía:

```
quarto  
""
```

R no puede localizar el ejecutable mediante la variable `PATH`.

También puede comprobarse la instalación completa desde una terminal:

```
quarto check
```

Esta orden informa sobre Quarto, Pandoc y los componentes auxiliares disponibles.

5.5 HTML

La construcción HTML necesita:

```
R  
iasi.quarto  
Quarto
```

No requiere una distribución de TeX.

Una vez instalado el paquete, el formato puede comprobarse con:

```
build(format = "html")
```

5.6 PDF

La construcción PDF añade una dependencia externa:

```
una distribución de TeX
```

Puede utilizarse, por ejemplo:

```
TinyTeX  
TeX Live  
MiKTeX
```

`iasi.quarto` no instala ni administra esa distribución.

Quarto debe poder localizarla y utilizarla durante el renderizado.

La comprobación práctica es:

```
build(format = "pdf")
```

Si HTML funciona y PDF no, el primer elemento que debe revisarse es el entorno TeX, no la estructura de `iasi.quarto`.

5.7 Git

Git no es una dependencia de ejecución.

`iasi.quarto` puede cargar y construir una publicación sin que Git esté instalado.

Sin embargo, Git resulta útil para:

```
obtener el código desde un repositorio
actualizar el paquete
versionar las publicaciones
trabajar con ramas y entregas
```

Su disponibilidad puede comprobarse desde una terminal:

```
git --version
```

La necesidad real de Git depende del método de instalación utilizado.

5.8 Acceso al repositorio

Mientras `iasi.quarto` se distribuya desde un repositorio de código, la instalación necesita acceso a ese repositorio.

El acceso puede realizarse mediante:

```
HTTPS
una copia local
un archivo del paquete
```

Una vez instalado, el paquete no necesita consultar el repositorio para ejecutar `validate()` o `build()`.

5.9 Permisos de escritura

Durante la preparación, `iasi.quarto` puede generar archivos dentro de la publicación:

```
_book-structure.yml  
index.qmd de las carpetas regular
```

La cuenta que ejecuta la construcción debe tener permiso de escritura sobre el proyecto.

También debe poder escribir en los directorios de salida y temporales utilizados por Quarto.

Una publicación de solo lectura puede validarse, pero no podrá completar una preparación que necesite crear o actualizar artefactos.

5.10 Red y servicios externos

La construcción básica de `iasi.quarto` no necesita un servicio remoto propio.

Sin embargo, una publicación concreta puede utilizar recursos externos:

```
repositorios de paquetes  
fuentes web  
bibliografía remota  
extensiones Quarto  
servidores de diagramas
```

Por ejemplo, el filtro PlantUML puede depender de un servidor configurado por la publicación.

Esas dependencias pertenecen al proyecto documental o a sus extensiones, no al núcleo de `iasi.quarto`.

5.11 Comprobación mínima

Antes de instalar el paquete, el entorno puede revisarse con:

```
R.version.string  
Sys.which("quarto")
```

Y desde una terminal:

```
quarto --version  
quarto check
```

Para construir ambos formatos, la comprobación mínima es:

```
R disponible  
Quarto disponible  
TeX disponible para PDF  
permisos de escritura sobre la publicación
```

No se necesita un entorno especial ni un editor concreto.

`iasi.quarto` organiza la construcción, pero utiliza las herramientas reales de R y Quarto que ya existen en el sistema.

5.12 Instalar `iasi.quarto`

`iasi.quarto` se distribuye actualmente desde su repositorio de GitHub:

```
iasi-org/iasi-quarto
```

El nombre del repositorio utiliza un guion:

```
iasi-quarto
```

El nombre del paquete de R utiliza un punto:

```
iasi.quarto
```

Esta diferencia aparece en la instalación y en el uso:

```
remotes::install_github("iasi-org/iasi-quarto")  
  
library(iasi.quarto)
```

5.13 Instalar remotes

La instalación desde GitHub puede realizarse con el paquete `remotes`.

Si todavía no está instalado:

```
install.packages("remotes")
```

Esta operación solo es necesaria una vez por biblioteca de R.

Puede comprobarse su disponibilidad con:

```
requireNamespace(  
  "remotes",  
  quietly = TRUE  
)
```

El resultado esperado es:

```
TRUE
```

5.14 Instalar la versión actual

Para instalar la versión disponible en la rama principal del repositorio:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

`remotes` descarga el código fuente, resuelve las dependencias declaradas por el paquete y ejecuta la instalación en la biblioteca activa de R.

Una vez terminada:

```
library(iasi.quarto)
```

La carga debe completarse sin errores.

5.15 Instalar una versión concreta

Una instalación reproducible debe indicar la versión que necesita.

Por ejemplo, para instalar la versión 0.5.0:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

La referencia situada después de @ puede ser:

```
una etiqueta  
una rama  
un commit
```

Una etiqueta publicada es la opción natural para una versión estable:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

Un commit permite fijar exactamente el código utilizado:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@<commit>"  
)
```

La instalación sin referencia:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

utiliza el estado actual de la rama predeterminada y puede cambiar con el tiempo.

5.16 Actualizar el paquete

Para actualizar una instalación existente, se ejecuta de nuevo la instalación:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

No es necesario desinstalar previamente el paquete.

Cuando `iasi.quarto` ya está cargado en la sesión, R puede mantener archivos del paquete en uso, especialmente en Windows.

En ese caso:

```
reinicie la sesión de R  
vuelva a ejecutar la instalación  
cargue de nuevo el paquete
```

En RStudio, la sesión puede reiniciarse desde:

```
Session  
-> Restart R
```

Después:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)  
  
library(iasi.quarto)
```

5.17 Comprobar la versión instalada

La versión activa puede consultarse con:

```
packageVersion("iasi.quarto")
```

Para la versión 0.5.0, el resultado será equivalente a:

```
[1] '0.5.0'
```

También puede consultarse la descripción completa del paquete:

```
packageDescription("iasi.quarto")
```

La ruta desde la que se ha cargado puede obtenerse con:

```
find.package("iasi.quarto")
```

Esta comprobación resulta útil cuando existen varias bibliotecas de R y no está claro cuál contiene la instalación activa.

5.18 Biblioteca de instalación

R instala el paquete en una de las rutas declaradas por:

```
.libPaths()
```

La primera ruta suele ser la biblioteca utilizada por defecto.

Puede consultarse antes de instalar:

```
.libPaths()
```

Y comprobar después dónde quedó instalado:

```
find.package("iasi.quarto")
```

Para instalar expresamente en una biblioteca concreta:

```
remotes::install_github(  
  "iasi-org/iasi-quarto",  
  lib = "path/to/library"  
)
```

La misma ruta debe estar disponible en `.libPaths()` cuando se cargue el paquete.

5.19 Dependencias

Las dependencias declaradas por `iasi.quarto` se instalan durante el proceso cuando no están disponibles.

La instalación no incorpora:

```
Quarto CLI
una distribución de TeX
servicios externos utilizados por la publicación
```

Esos componentes pertenecen al entorno de construcción y deben estar preparados por separado.

El paquete puede instalarse correctamente aunque todavía falte TeX. En ese caso, podrá cargarse y construir HTML, pero el renderizado PDF no estará completo hasta que Quarto disponga de una distribución de TeX.

5.20 Instalación desde una copia local

Cuando ya existe una copia local del repositorio, puede instalarse directamente desde su raíz.

Con `remotes`:

```
remotes::install_local(
  "path/to/iasi-quarto"
)
```

Durante el desarrollo del propio paquete también puede utilizarse:

```
devtools::install(
  "path/to/iasi-quarto"
)
```

La ruta debe señalar el directorio que contiene el archivo:

```
DESCRIPTION
```

La instalación local utiliza el estado actual de los archivos, incluidos los cambios que todavía no se hayan publicado en GitHub.

5.21 Cargar el paquete

La instalación coloca el paquete en una biblioteca de R.

Para utilizarlo en una sesión:

```
library(iasi.quarto)
```

Esto incorpora su API pública:

```
validate()  
build()
```

También puede llamarse una función sin cargar todo el paquete:

```
iasi.quarto::validate()
```

```
iasi.quarto::build()
```

Esta forma hace explícito el paquete al que pertenece cada función.

5.22 Instalación reproducible

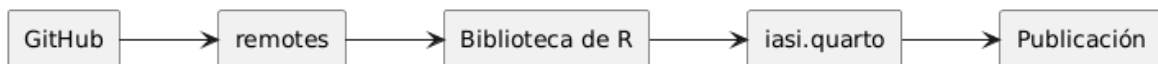
Para un entorno personal de desarrollo puede ser suficiente instalar la rama principal:

```
remotes::install_github(  
  "iasi-org/iasi-quarto"  
)
```

Para una construcción que deba repetirse en otro equipo o en integración continua, debe fijarse una versión:

```
remotes::install_github(  
  "iasi-org/iasi-quarto@v0.5.0"  
)
```

De este modo, el código de la publicación y la herramienta que interpreta su estructura evolucionan de forma controlada.



5.23 Instalación mínima

La secuencia mínima completa es:

```
install.packages("remotes")

remotes::install_github(
  "iasi-org/iasi-quarto@v0.5.0"
)

library(iasi.quarto)

packageVersion("iasi.quarto")
```

La instalación termina cuando R puede cargar el paquete desde su biblioteca.

La siguiente comprobación consiste en verificar que `iasi.quarto` reconoce una publicación real y puede ejecutar su ciclo de construcción.

5.24 Verificar la instalación

La instalación queda verificada cuando R puede cargar `iasi.quarto`, localizar Quarto y reconocer una publicación válida.

La comprobación puede realizarse en cuatro pasos:

```
comprobar el paquete
comprobar Quarto
validar una publicación
construir un formato
```

5.25 Comprobar el paquete

Cargue el paquete:

```
library(iasi.quarto)
```

La operación debe terminar sin errores.

Después, consulte la versión instalada:

```
packageVersion("iasi.quarto")
```

Para la versión 0.5.0, el resultado será equivalente a:

```
[1] '0.5.0'
```

También puede comprobarse la ruta desde la que R ha cargado el paquete:

```
find.package("iasi.quarto")
```

Esto resulta útil cuando existen varias bibliotecas de R o varias instalaciones del mismo paquete.

5.26 Comprobar Quarto

`iasi.quarto` necesita que el ejecutable de Quarto esté disponible para la sesión de R.

Compruébelo con:

```
Sys.which("quarto")
```

El resultado debe contener una ruta.

Por ejemplo:

```
quarto  
"C:\Program Files\Quarto\bin\quarto.exe"
```

Si el resultado es una cadena vacía:

```
quarto  
""
```

R no puede localizar Quarto mediante la variable `PATH`.

También puede comprobarse la versión desde R:

```
system2(  
  "quarto",  
  "--version"  
)
```

Y desde una terminal:

```
quarto --version
```

5.27 Situarse en una publicación

Para verificar `validate()`, la sesión debe trabajar sobre una publicación IASI Quarto o sobre un multiproyecto que contenga publicaciones completas.

Una publicación mínima contiene:

```
publication/  
_quarto.yml  
_quarto-html.yml  
_quarto-pdf.yml  
iasi.yml
```

Desde R puede comprobarse el directorio actual con:

```
getwd()
```

Cuando la publicación se encuentra en otra ruta:

```
setwd(  
  "path/to/publication"  
)
```

También puede evitarse el cambio de directorio pasando la ruta explícitamente:

```
plan = validate(  
  "path/to/publication"  
)
```

5.28 Validar la publicación

Ejecute:

```
plan = validate()
```

Si el directorio actual contiene una publicación o multiproyecto IASI Quarto, `validate()` devuelve invisiblemente un plan.

Puede comprobarse con:

```
is.null(plan)
```

El resultado esperado es:

```
FALSE
```

Y su clase:

```
class(plan)
```

debe incluir:

```
"iasi_quarto_plan"
```

También pueden inspeccionarse sus propiedades iniciales:

```
plan$path  
plan$current  
plan$books
```

5.28.1 Publicación actual

Cuando la ruta validada contiene la firma completa:

```
plan$current
```

debe devolver:

```
TRUE
```

5.28.2 Multiproyecto

Cuando la ruta actúa como contenedor de varias publicaciones:

```
plan$current
```

debe devolver:

```
FALSE
```

Las publicaciones candidatas quedan registradas en:

```
plan$books
```

5.29 Directorio no reconocido

Si la ruta no contiene una publicación ni un multiproyecto IASI Quarto:

```
plan = validate(  
  "path/to/ordinary-directory"  
)
```

el resultado es:

```
NULL
```

Esto no indica un fallo de instalación.

Indica que la ruta no cumple la firma de publicación esperada.

La diferencia es importante:

```
library(iasi.quarto) falla  
  problema de instalación del paquete  
  
Sys.which("quarto") devuelve ""  
  Quarto no está disponible para R  
  
validate() devuelve NULL  
  la ruta no es un espacio IASI Quarto
```

5.30 Construir HTML

La comprobación funcional mínima consiste en construir HTML:

```
build(  
  format = "html"  
)
```

HTML es la primera prueba recomendada porque no depende de una distribución de TeX.

La construcción completa recorre:

```
validate()  
.discover()  
.check()  
.prepare()  
.render()
```

Si termina correctamente, quedan verificados:

```
la carga del paquete  
la lectura de la configuración  
la estructura de la publicación  
la generación de artefactos  
la invocación de Quarto
```

5.31 Inspeccionar el resultado

`build()` devuelve invisiblemente el plan completado.

Puede conservarse:

```
plan = build(  
  format = "html"  
)
```

Después pueden inspeccionarse:

```
plan$formats  
plan$rendered  
plan$elapsed
```

El formato esperado es:

```
plan$formats
```

```
"html"
```

Y, después de un renderizado correcto:

```
plan$rendered
```

debe ser:

```
TRUE
```

Cada proyecto contiene también su publicación preparada:

```
publication = plan$projects[[1]]$publication
```

Pueden consultarse:

```
publication$chapters  
publication$artifacts  
publication$changed  
publication$rendered  
publication$profiles
```

5.32 Construir PDF

Una vez verificado HTML, puede comprobarse PDF:

```
build(  
  format = "pdf"  
)
```

Esta prueba añade la dependencia de TeX.

Si HTML funciona y PDF falla, el paquete y la estructura básica ya han superado la verificación principal.

En ese caso, debe revisarse:

```
la instalación de TeX
la integración de TeX con Quarto
las dependencias LaTeX de la publicación
```

La comprobación general puede realizarse desde una terminal:

```
quarto check
```

5.33 Verificación completa

La secuencia completa es:

```
library(iasiq.quarto)

packageVersion("iasiq.quarto")

Sys.which("quarto")

plan = validate()

stopifnot(
  !is.null(plan)
)

html = build(
  format = "html"
)

stopifnot(
  isTRUE(html$rendered)
)
```

Después, cuando el entorno PDF esté preparado:

```
pdf = build(
  format = "pdf"
)

stopifnot(
  isTRUE(pdf$rendered)
)
```

5.34 Diagnóstico rápido

Los fallos más habituales pueden localizarse por la etapa en la que aparecen.

Comprobación	Resultado	Interpretación
<code>library(iasi.quarto)</code>	error	El paquete no está instalado o no está disponible en <code>.libPaths()</code>
<code>Sys.which("quarto")</code> <code>validate()</code>	<code>""</code> NULL	R no puede localizar Quarto La ruta no contiene una publicación IASI Quarto
<code>build(format = "html")</code>	error antes de Quarto	La configuración o estructura no supera el pipeline
<code>build(format = "html")</code>	error de Quarto	Quarto no pudo renderizar la publicación
<code>build(format = "pdf")</code>	falla con HTML correcto	Debe revisarse el entorno TeX

```
PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:13:24 UTC]  
  
This version of PlantUML is 218 days old, so you should  
consider upgrading from https://plantuml.com/download  
  
[From string (line 4) ]  
  
@startuml  
top to bottom direction  
  
start  
Syntax Error? (Assumed diagram type: class)
```

5.35 Resultado esperado

La instalación puede considerarse correcta cuando:

```
R carga iasi.quarto  
Quarto está disponible  
validate() reconoce la publicación  
build(format = "html") termina correctamente
```

PDF puede verificarse después como una capacidad adicional del entorno.

La instalación no termina al copiar el paquete. Termina cuando una publicación real atraviesa el pipeline y llega a Quarto.

6 PlantUML

6.1 Instalación

`iasi.quarto` se distribuye como un paquete de R.

Su instalación añade al entorno dos operaciones públicas:

```
validate()  
build()
```

La primera reconoce una publicación IASI Quarto. La segunda ejecuta su ciclo completo de construcción.

El paquete no sustituye a Quarto ni instala por sí mismo sus herramientas externas. Trabaja sobre un entorno en el que R y Quarto ya están disponibles.



La instalación se divide en tres pasos:

```
comprobar los requisitos  
instalar iasi.quarto  
verificar el entorno
```

En primer lugar, se comprueba que R y Quarto puedan ejecutarse correctamente.

Después se instala el paquete desde su repositorio.

Por último, se realiza una comprobación mínima para confirmar que `iasi.quarto` puede cargarse y reconocer una publicación.

Este capítulo no pretende explicar cómo instalar R o Quarto en cada sistema operativo. Su objetivo es dejar preparado el entorno específico que necesita `iasi.quarto`.

Al finalizar, debe ser posible ejecutar:

```
library(iasi.quarto)

validate()
```

y obtener un plan cuando el directorio actual contiene una publicación o un multiproyecto IASI Quarto.

La instalación prepara la herramienta. La estructura y la configuración de la publicación siguen perteneciendo al proyecto.

6.1.1 ¿Por qué PlantUML?

Quarto admite distintas formas de incorporar diagramas en una publicación.

Mermaid es una alternativa especialmente cómoda para diagramas sencillos. Puede escribirse directamente dentro del documento y encaja bien cuando el objetivo principal es representar rápidamente un flujo, una secuencia o una relación entre elementos.

PlantUML introduce una decisión diferente.

En esta plataforma, el diagrama no se trata únicamente como una ilustración embebida en Markdown. Se trata como una fuente que atraviesa un compilador especializado antes de incorporarse al documento final.

```
fuelle PlantUML
-> estilos compartidos
-> extensión Lua
-> servidor PlantUML
-> imagen
-> caché
-> HTML o PDF
```

La elección no nace de una oposición entre herramientas.

Mermaid y PlantUML resuelven problemas parcialmente coincidentes, pero encajan de forma distinta en una arquitectura documental.

6.1.1.1 Mermaid como opción válida

Mermaid resulta apropiado cuando se busca:

```
escribir diagramas con poca infraestructura
mantener todo dentro del documento
crear representaciones rápidas
aprovechar una integración directa con Quarto
evitar un servidor externo
```

Para muchos proyectos, estas propiedades son suficientes.

Un diagrama sencillo puede escribirse cerca del texto al que pertenece y renderizarse como parte natural del documento.

La ventaja principal es la inmediatez.

```
fuelle Mermaid
-> Quarto
-> representación final
```

Este camino reduce componentes y simplifica la puesta en marcha.

6.1.1.2 Lo que necesita la plataforma

La plataforma persigue algo más amplio que insertar diagramas ocasionales.

Necesita que los diagramas puedan formar parte de una infraestructura común:

```
estilos reutilizables
servidor controlado
salida gráfica predecible
caché independiente
integración con HTML y PDF
arquitectura extensible
```

También necesita separar claramente:

```
la fuente del diagrama
la configuración del compilador
el servicio de renderizado
la imagen generada
la publicación final
```

PlantUML encaja mejor con esa separación.

6.1.1.3 Un lenguaje orientado a documentación técnica

PlantUML ofrece una familia amplia de diagramas útiles para documentación de sistemas.

Entre otros, permite representar:

```
secuencias  
componentes  
despliegues  
clases  
casos de uso  
estados  
actividades  
objetos  
arquitecturas
```

La importancia no está únicamente en la cantidad de tipos disponibles.

La misma herramienta puede acompañar diferentes niveles de documentación:

```
visión general del sistema  
interacción entre servicios  
estructura lógica  
despliegue físico  
flujo de una operación  
modelo de dominio
```

Esto reduce la necesidad de introducir una tecnología diferente para cada clase de diagrama.

6.1.2 Estilos compartidos

Una de las razones principales de la elección es la posibilidad de centralizar la presentación mediante archivos `.puml`.

Por ejemplo:

```
resources/plantuml/iasi.puml
```

Ese archivo puede contener reglas comunes para toda la publicación:

```
colores
tipografía
bordes
espaciado
apariencia de componentes
convenciones visuales
```

El documento describe el significado del diagrama.

El recurso compartido define su apariencia.

```
documento
-> estructura y relaciones

iasi.puml
-> lenguaje visual común
```

Esta separación evita copiar configuraciones visuales dentro de cada bloque.

También permite modificar la identidad gráfica de numerosos diagramas desde un único lugar.

6.1.3 Servidor independiente

PlantUML puede ejecutarse mediante un servidor separado del proceso de Quarto.

La extensión envía la fuente preparada y recibe una imagen.

```
extensión
-> petición HTTP
-> servidor PlantUML
-> imagen
```

Esta separación aporta varias posibilidades:

```
servidor local para una persona
servidor compartido para un equipo
contenedor dedicado
servicio interno dentro de una red
```

El motor de diagramación no queda acoplado al navegador ni al formato final del documento.

Puede evolucionar, actualizarse o desplegarse de forma independiente.

6.1.4 Una imagen común para HTML y PDF

La configuración recomendada genera PNG.

```
PlantUML -> PNG -> HTML  
PlantUML -> PNG -> PDF
```

Quarto recibe el mismo tipo de recurso para ambos formatos.

Esto evita depender de una representación distinta según el destino y reduce conversiones adicionales durante la construcción del PDF.

La decisión no significa que PNG sea siempre superior a SVG.

Significa que, para este pipeline concreto, una salida común y predecible simplifica el comportamiento general.

6.1.5 Caché antes del renderizado

La extensión puede calcular una identidad para cada diagrama a partir de:

```
fuelle preparada  
servidor  
formato  
versión de la extensión
```

Cuando esa identidad ya existe en la caché, no es necesario volver a solicitar la imagen al servidor.

```
diagrama sin cambios  
  -> reutilizar imagen  
  
diagrama modificado  
  -> compilar de nuevo
```

La caché pertenece a la infraestructura de la extensión, no al documento final.

Esta posición dentro del pipeline permite reutilizar diagramas antes de que Quarto procese HTML o PDF.

6.1.6 Integración con la arquitectura de extensiones

PlantUML es también el primer ejemplo completo del framework de extensiones de la plataforma.

La integración separa:

```
núcleo común
configuración
caché
sistema de archivos
mediabag
compilador específico
```

PlantUML aporta el compilador concreto.

El núcleo aporta el ciclo general.

```
bloque de código
-> configuración
-> preparación
-> identidad
-> caché
-> compilación
-> recurso para Pandoc
```

Esta arquitectura permite que futuras extensiones sigan el mismo modelo sin copiar toda la infraestructura.

PlantUML no es únicamente una herramienta de diagramación dentro del proyecto.

Es el primer ciudadano de una familia de compiladores documentales.

6.1.7 Comparación resumida

Aspecto	Mermaid	PlantUML en esta plataforma
Puesta en marcha	Muy directa	Requiere extensión y servidor
Infraestructura	Mínima	Explícita y configurable
Fuente en el documento	Sí	Sí
Estilos compartidos	Posibles, pero no son el centro del diseño adoptado	Archivo <code>.puml</code> común
Servidor propio	No es necesario	Recomendado

Aspecto	Mermaid	PlantUML en esta plataforma
Caché de la extensión	No forma parte de este diseño	Integrada antes de Quarto
Salida común HTML/PDF	Depende del procesamiento del documento	PNG generado previamente
Encaje con el framework Lua	No es la integración elegida	Compilador especializado
Uso recomendado	Diagramas rápidos y ligeros	Infraestructura documental técnica

La tabla no pretende establecer un vencedor universal.

Describe la razón por la que esta plataforma adopta PlantUML como opción principal.

6.1.8 Criterio de elección

La decisión puede resumirse así:

```

si el objetivo es dibujar rápidamente dentro del documento
  Mermaid puede ser suficiente

si el objetivo es integrar los diagramas en una infraestructura común
  PlantUML encaja mejor con esta plataforma

```

La elección se apoya en cuatro necesidades:

```

control
reutilización
reproducibilidad
extensibilidad

```

Control sobre el servicio y el formato generado.

Reutilización de estilos y convenciones visuales.

Reproducibilidad de la construcción mediante fuente, caché y compilador.

Extensibilidad mediante una arquitectura que puede incorporar otros motores.

No elegimos PlantUML porque Mermaid no sirva. Lo elegimos porque aquí el diagrama es una pieza del pipeline de ingeniería documental.

6.2 Servidor PlantUML

La extensión necesita un servidor PlantUML accesible mediante HTTP.

El servidor recibe la fuente preparada por la extensión y devuelve la imagen del diagrama.

```
extensión Lua  
-> petición HTTP  
-> servidor PlantUML  
-> imagen PNG
```

El servidor no tiene que ejecutarse dentro del proyecto Quarto ni dentro del proceso de R.

Puede estar:

```
en el mismo equipo  
en otra máquina de la red  
en una infraestructura compartida  
en un servicio público
```

La elección depende del entorno, pero esta guía recomienda utilizar un servidor controlado por el usuario o por el equipo.

6.3 Alternativas

6.3.1 Servidor público

Un servidor público permite comenzar sin instalar infraestructura propia.

```
ventajas  
  puesta en marcha inmediata  
  ninguna administración local  
  
inconvenientes  
  dependencia de Internet  
  disponibilidad fuera del control del proyecto  
  envío de la fuente del diagrama a un tercero
```

Puede resultar útil para una prueba rápida.

No es la opción recomendada para documentación interna, diagramas sensibles o construcciones que deban ser reproducibles dentro de un entorno controlado.

6.3.2 Servidor local

Un servidor local se ejecuta en el mismo equipo desde el que se construye la publicación.

```
publicación  
-> http://localhost:1025  
-> PlantUML
```

Sus principales ventajas son:

```
no depende de un servicio público  
mantiene la fuente dentro del equipo  
puede funcionar sin acceso a Internet  
es sencillo de iniciar y detener
```

Es la opción recomendada para desarrollo individual.

6.3.3 Servidor compartido

Un equipo puede mantener una única instancia accesible dentro de su red.

```
equipo A  
equipo B  > servidor PlantUML interno  
CI/CD
```

Esto permite compartir:

```
la misma versión del motor  
una dirección común  
una política de actualización  
un servicio disponible para integración continua
```

Es la opción recomendada cuando varias personas o procesos construyen las mismas publicaciones.

6.4 Instalación recomendada con Docker

La instalación de referencia utiliza la imagen oficial de PlantUML Server y publica su puerto interno 8080 como puerto 1025 del equipo anfitrión.

```
docker run -d \  
  --name plantuml-server \  
  --restart unless-stopped \  
  -p 1025:8080 \  
  plantuml/plantuml-server:jetty
```

En PowerShell puede escribirse en una sola línea:

```
docker run -d --name plantuml-server --restart unless-stopped -p 1025:8080 plantuml/plantuml-
```

La correspondencia de puertos es:

```
equipo anfitrión : 1025  
contenedor      : 8080
```

La extensión utilizará:

```
http://localhost:1025
```

El puerto 1025 es una convención de esta instalación, no una exigencia de PlantUML ni de iasi.quarto.

Puede sustituirse por otro puerto libre.

6.5 Instalación con Docker Compose

La misma instalación puede declararse mediante `compose.yml`:

```
services:  
  plantuml:  
    image: plantuml/plantuml-server:jetty  
    container_name: plantuml-server  
    restart: unless-stopped  
    ports:  
      - "1025:8080"
```

Para iniciarla:

```
docker compose up -d
```

Para comprobar su estado:

```
docker compose ps
```

Para detenerla:

```
docker compose down
```

La declaración Compose resulta especialmente útil cuando el servidor forma parte de un entorno de desarrollo reproducible.

6.6 Comprobar el servidor

Después de iniciar el contenedor, puede abrirse en un navegador:

```
http://localhost:1025
```

También puede comprobarse desde una terminal:

```
curl http://localhost:1025
```

La respuesta debe proceder del servidor PlantUML.

El contenedor puede inspeccionarse con:

```
docker ps --filter name=plantuml-server
```

Y sus mensajes con:

```
docker logs plantuml-server
```

6.7 Utilizarlo desde otra máquina

Cuando Quarto se ejecuta en otro equipo, `localhost` ya no representa la máquina que aloja PlantUML.

Debe utilizarse su nombre o dirección de red:

```
http://<host>:1025
```

Por ejemplo:

```
filter-options:  
  plantuml:  
    server: http://plantuml.local:1025
```

La máquina cliente debe poder resolver el nombre y acceder al puerto publicado.

La comprobación debe realizarse desde el mismo entorno que ejecutará Quarto:

```
curl http://plantuml.local:1025
```

Que el servidor funcione en su propia máquina no garantiza que sea accesible desde otro equipo, una máquina virtual o un contenedor.

6.8 Configuración local recomendada

Para una publicación construida en el mismo equipo:

```
filter-options:  
  plantuml:  
    enabled: true  
    server: http://localhost:1025  
    format: png  
    cache: true  
    styles:  
      - resources/plantuml/iasi.puml
```

Esta configuración combina:

```
servidor local controlado  
salida PNG  
caché activada  
estilo compartido
```

Es el punto de partida recomendado en esta guía.

6.9 Configuración para un equipo

Cuando el servidor es compartido, solo cambia su dirección:

```
filter-options:
  plantuml:
    enabled: true
    server: http://plantuml.local:1025
    format: png
    cache: true
    styles:
      - resources/plantuml/iasi.puml
```

La publicación no necesita conocer cómo se ejecuta internamente el servidor.

Solo necesita una URL compatible y accesible.

Esto permite cambiar su despliegue sin modificar los bloques PlantUML de los documentos.

6.10 Actualizar el servidor

Cuando se utiliza la etiqueta `jetty`, Docker puede obtener una versión más reciente de la imagen:

```
docker pull plantuml/plantuml-server:jetty
```

Después debe recrearse el contenedor.

Con Docker Compose:

```
docker compose pull
docker compose up -d
```

Para construcciones que necesiten reproducibilidad estricta, el equipo debe fijar una etiqueta concreta o un digest de imagen en lugar de depender de una etiqueta móvil.

La actualización del servidor puede cambiar el resultado gráfico de un diagrama aunque su fuente no haya cambiado.

Por eso debe tratarse como una actualización de infraestructura y comprobarse antes de adoptarla en una publicación estable.

6.11 Seguridad y privacidad

La fuente del diagrama se envía al servidor configurado.

Por tanto:

```
un servidor público recibe el contenido del diagrama
un servidor local lo mantiene en el equipo
un servidor interno lo mantiene dentro de la red controlada
```

Los diagramas pueden contener nombres de sistemas, relaciones, componentes o flujos internos.

La dirección del servidor no debe elegirse únicamente por comodidad.

Tampoco se recomienda publicar directamente en Internet una instancia propia sin revisar su exposición, configuración y política de seguridad.

Para uso individual, enlazarla únicamente al entorno necesario reduce la superficie accesible.

Para uso compartido, el servidor debe incorporarse a las medidas normales de red, actualización y control de acceso del equipo.

6.12 Disponibilidad y caché

El servidor es necesario cuando la extensión debe compilar un diagrama nuevo.

```
diagrama nuevo o modificado
-> necesita servidor

diagrama presente en caché
-> puede reutilizar la imagen
```

La caché mejora el rendimiento y reduce la dependencia durante construcciones repetidas, pero no debe confundirse con una garantía de disponibilidad permanente.

Una construcción limpia necesita poder acceder al servidor para regenerar todos los diagramas.

6.13 La instalación de referencia

La recomendación de esta guía puede resumirse así:

```
PlantUML Server oficial
ejecutado en Docker
puerto del anfitrión 1025
acceso local o mediante red interna
salida PNG
caché activada
estilos compartidos
```

No es la única instalación posible.

Es una configuración pequeña, comprensible y suficiente para desarrollar publicaciones, compartir el servicio dentro de un equipo y trasladarlo posteriormente a una infraestructura más estable.

El contrato de la extensión es una URL PlantUML accesible. Docker y el puerto 1025 son la implementación de referencia.

6.14 Configurar la extensión

La extensión PlantUML se incorpora a la publicación desde la configuración de Quarto.

La configuración recomendada se declara en `_quarto.yml`:

```
filters:
  - extensions/plantuml/plantuml.lua

filter-options:
  plantuml:
    enabled: true
    server: http://localhost:1025
    format: png
    cache: true
    styles:
      - resources/plantuml/iasi.puml
```

Esta declaración tiene dos partes:

```
filters
  carga el filtro Lua

filter-options
  configura su comportamiento
```

El filtro debe cargarse antes de que sus opciones puedan aplicarse.

6.15 Ruta del filtro

La entrada:

```
filters:  
  - extensions/plantuml/plantuml.lua
```

indica la ubicación del filtro respecto a la raíz de la publicación.

La estructura esperada puede ser:

```
publication/  
  _quarto.yml  
  extensions/  
    core/  
    plantuml/  
      plantuml.lua  
      compiler.lua  
      defaults.lua  
  resources/  
    plantuml/  
      iasi.puml
```

La extensión PlantUML utiliza módulos comunes situados en:

```
extensions/core/
```

Por tanto, no debe copiarse únicamente `plantuml.lua`.

Debe conservarse la estructura completa de la extensión y de su núcleo compartido.

6.16 enabled

La propiedad:

```
enabled: true
```

activa el procesamiento de bloques PlantUML.

Cuando su valor es **false**, la extensión no debe compilar los bloques.

```
filter-options:  
  plantuml:  
    enabled: false
```

Esta opción permite conservar la configuración y desactivar temporalmente la integración sin retirar el filtro de `_quarto.yml`.

La configuración habitual es:

```
enabled: true
```

6.17 server

La propiedad `server` contiene la URL base del servidor PlantUML.

```
server: http://localhost:1025
```

No debe incluirse el segmento de formato:

```
/png  
/svg
```

La extensión lo añade al construir la petición.

La URL debe señalar únicamente la raíz del servicio:

```
correcto  
  http://localhost:1025  
  
incorrecto  
  http://localhost:1025/png
```

Para un servidor compartido:

```
server: http://plantuml.local:1025
```

La dirección debe ser accesible desde el proceso que ejecuta Quarto.

Esto es especialmente importante cuando la construcción ocurre dentro de:

```
una máquina virtual
un contenedor
un agente de integración continua
otro equipo de la red
```

En esos entornos, `localhost` representa el propio entorno de ejecución, no necesariamente la máquina anfitriona.

6.18 format

La configuración recomendada utiliza:

```
format: png
```

El flujo resultante es:

```
PlantUML
-> PNG
-> mediabag de Pandoc
-> HTML o PDF
```

La salida PNG evita que la construcción PDF dependa de una conversión posterior desde SVG.

Aunque la arquitectura conserva la validación de formatos admitidos por el compilador, la implementación actual de la plataforma normaliza la salida efectiva a PNG.

Por tanto, la configuración de usuario debe expresar también:

```
format: png
```

Así, la configuración declarada coincide con el comportamiento real del pipeline.

6.19 cache

La propiedad:

```
cache: true
```

activa la reutilización de diagramas ya compilados.

Cuando la caché está activa:

```
misma fuente preparada  
misma configuración relevante  
misma versión de la extensión  
-> misma entrada de caché
```

El servidor solo vuelve a utilizarse cuando cambia la identidad del diagrama o cuando la entrada almacenada ya no está disponible.

Para desactivar temporalmente la caché:

```
cache: false
```

Esto puede resultar útil durante el diagnóstico de un servidor o de una modificación del compilador.

No es la configuración recomendada para el trabajo habitual.

6.20 styles

La propiedad **styles** declara los archivos PlantUML que deben incorporarse a cada diagrama.

```
styles:  
- resources/plantuml/iasi.puml
```

Puede contener más de un archivo:

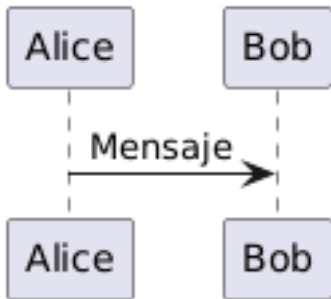
```
styles:  
- resources/plantuml/base.puml  
- resources/plantuml/architecture.puml
```

Los archivos se procesan en el orden declarado.

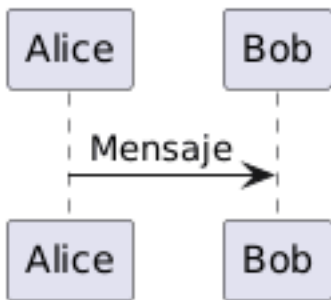
```
base.puml  
-> architecture.puml  
-> fuente del diagrama
```

La extensión lee esos recursos y los inserta después de la declaración `@startuml`.

Una fuente como:



se prepara conceptualmente como:



El archivo indicado debe existir y ser legible durante la construcción.

La ruta se interpreta dentro del contexto de la publicación, por lo que conviene mantener los estilos bajo una ubicación estable:

```
resources/plantuml/
```

6.21 Configuración mínima

La configuración mínima necesita el filtro y un servidor:

```
filters:
- extensions/plantuml/plantuml.lua

filter-options:
  plantuml:
    enabled: true
```

```
server: http://localhost:1025
format: png
```

Sin estilos:

```
el diagrama conserva la presentación predeterminada de PlantUML
```

Sin caché explícita:

```
se aplica el valor definido por la extensión
```

Para que el proyecto sea legible y reproducible, es preferible declarar las opciones importantes de forma explícita.

6.22 Configuración recomendada

La configuración completa recomendada es:

```
filters:
  - extensions/plantuml/plantuml.lua

filter-options:
  plantuml:
    enabled: true
    server: http://localhost:1025
    format: png
    cache: true
    styles:
      - resources/plantuml/iasi.puml
```

Esta combinación expresa todas las decisiones relevantes:

```
la extensión está activa
el servidor es local
la salida es PNG
la caché está habilitada
la publicación comparte un estilo
```

6.23 Configuración por entorno

La URL del servidor puede variar entre equipos.

Por ejemplo:

```
desarrollo local
  http://localhost:1025

red interna
  http://plantuml.local:1025

integración continua
  http://plantuml:8080
```

No conviene fijar en una guía reutilizable el nombre concreto de una máquina personal.

La configuración debe describir el servicio desde el punto de vista del entorno que ejecuta la publicación.

Cuando varios perfiles de Quarto necesitan direcciones diferentes, la opción puede colocarse en la configuración correspondiente al entorno.

La configuración común debe mantenerse en `_quarto.yml`, y únicamente las diferencias deben desplazarse a perfiles específicos.

6.24 Opciones de bloque

La infraestructura permite que determinados atributos del bloque reemplacen opciones globales para un diagrama concreto.

Las opciones reconocidas por el motor son:

```
server
format
cache
```

Un bloque puede declarar, por ejemplo:

```
```{.plantuml cache="false"}
@startuml
Alice -> Bob: Prueba sin caché
@enduml
```
```

Esta capacidad debe utilizarse con moderación.

La configuración global mantiene una publicación coherente. Las excepciones por bloque son apropiadas para pruebas o necesidades muy localizadas, no para sustituir una configuración común bien definida.

En la implementación actual, la salida efectiva continúa normalizándose a PNG.

6.25 Configuración HTML y PDF

La extensión se ejecuta antes de que Quarto genere el formato final.

Por eso puede utilizarse la misma configuración para ambos perfiles:

```
_quarto.yml
-> extensión PlantUML
-> imagen PNG

_quarto-html.yml
-> HTML

_quarto-pdf.yml
-> PDF
```

No es necesario declarar un servidor PlantUML diferente para HTML y PDF cuando ambos se construyen en el mismo entorno.

Tampoco es necesario mantener dos copias del estilo.

La configuración compartida pertenece a `_quarto.yml`.

6.26 Comprobar la configuración

Después de guardar `_quarto.yml`, deben comprobarse tres elementos.

Primero, que el servidor responde:

```
curl http://localhost:1025
```

Segundo, que las rutas existen:

```
extensions/plantuml/plantuml.lua
resources/plantuml/iasi.puml
```

Tercero, que un documento contiene al menos un bloque PlantUML válido:

```
```.plantuml
@startuml
Alice -> Bob: Verificación
@enduml
```
```

Después puede construirse HTML:

```
build(
  format = "html"
)
```

Y, una vez verificado:

```
build(
  format = "pdf"
)
```

6.27 Errores de configuración habituales

6.27.1 El filtro no existe

```
extensions/plantuml/plantuml.lua
```

debe coincidir con la ruta real.

Un cambio de nombre o una copia incompleta impide cargar la extensión.

6.27.2 El servidor incluye /png

La opción debe ser:

```
server: http://localhost:1025
```

No:

```
server: http://localhost:1025/png
```

La extensión construye la ruta final.

6.27.3 El estilo no puede leerse

Una ruta como:

```
styles:  
- resources/plantuml/iasi.puml
```

debe señalar un archivo existente.

La extensión detiene la compilación cuando no puede leer un estilo declarado.

6.27.4 localhost apunta al lugar equivocado

Desde un contenedor o una máquina virtual, `localhost` puede no ser el equipo donde se ejecuta PlantUML.

Debe utilizarse una dirección accesible desde el entorno de construcción.

6.27.5 La configuración declara SVG

La plataforma utiliza actualmente PNG como salida efectiva.

La configuración recomendada debe mantener:

```
format: png
```

para evitar una discrepancia entre lo declarado y lo generado.

6.28 Contrato de configuración

La extensión necesita conocer:

```
si debe ejecutarse
dónde está el servidor
qué formato debe producir
si puede reutilizar la caché
qué estilos debe incorporar
```

Estas decisiones pertenecen a la publicación, no a cada documento individual.

`_quarto.yml` conecta el contenido PlantUML con la infraestructura que lo convierte en una imagen reproducible.

6.29 Escribir diagramas

Los diagramas PlantUML se escriben directamente dentro de los documentos `.qmd`.

La forma básica es un bloque de código con la clase `plantuml`:

```
```{.plantuml}
@startuml
Alice -> Bob: Mensaje
@enduml
```
```

La extensión reconoce el bloque, envía su contenido al servidor y sustituye el código por la imagen generada.

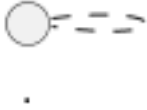
```
bloque .plantuml
-> fuente PlantUML
-> compilación
-> imagen dentro del documento
```

El diagrama aparece en la misma posición en la que se declaró el bloque.

6.30 Estructura mínima

Cada bloque debe contener una fuente PlantUML válida.

La estructura mínima es:



`@startuml` abre el diagrama.

`@enduml` lo cierra.

Entre ambas declaraciones se escribe su contenido.

Por ejemplo:

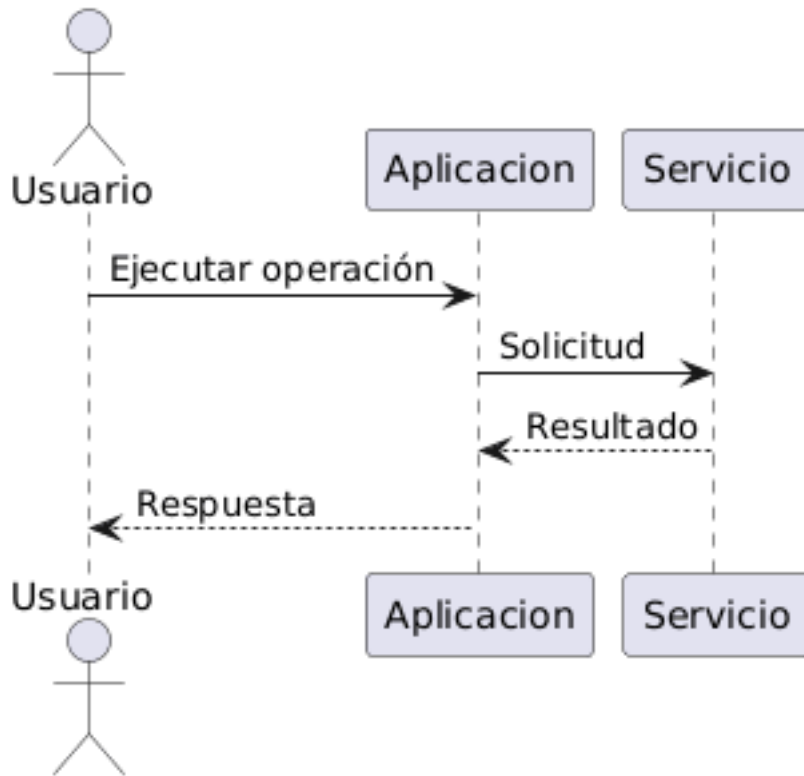
```
```.plantuml
@startuml
Cliente -> Servicio: Solicitud
Servicio --> Cliente: Respuesta
@enduml
```
```

La extensión no inventa ni corrige la sintaxis PlantUML.

Cuando el servidor no puede interpretar la fuente, la construcción debe detenerse con el error correspondiente.

6.31 Primer diagrama

Este es un ejemplo completo:



Su fuente es:

```

```{.plantuml}
@startuml
actor Usuario
participant Aplicacion
participant Servicio

Usuario -> Aplicacion: Ejecutar operación
Aplicacion -> Servicio: Solicitud
Servicio --> Aplicacion: Resultado
Aplicacion --> Usuario: Respuesta
@enduml
```

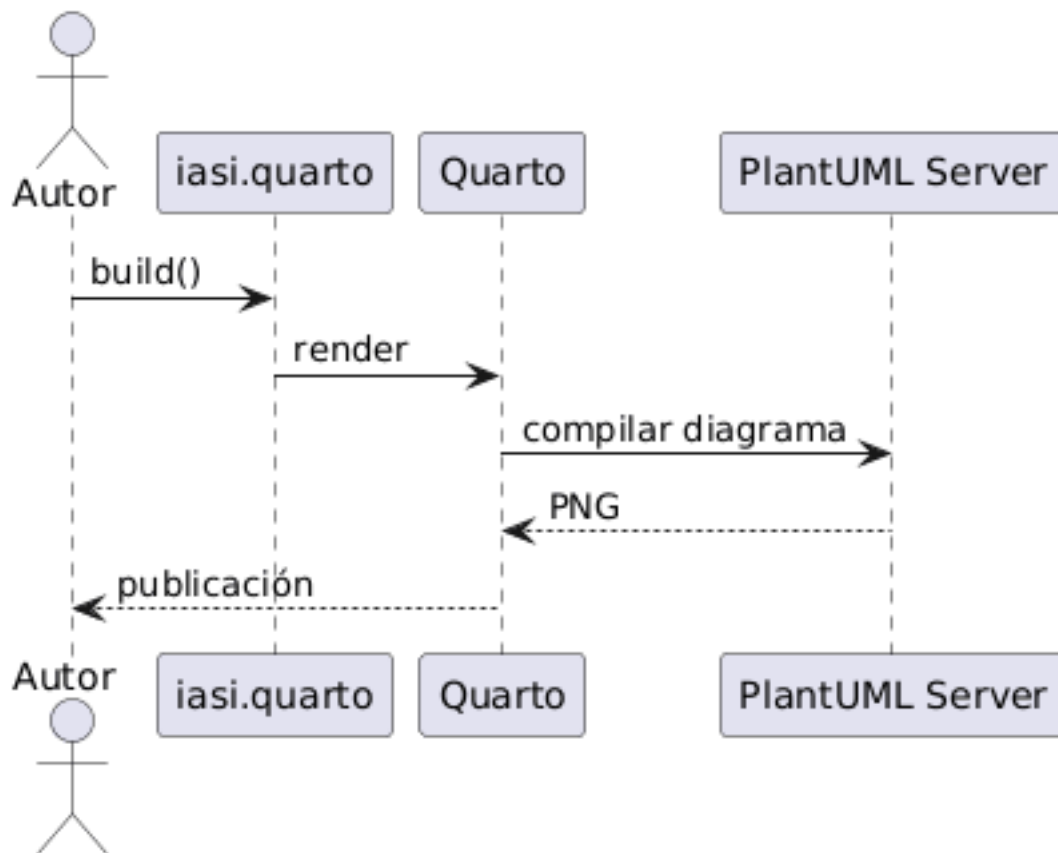
```

El mismo bloque se utiliza para HTML y PDF.

No es necesario mantener una versión distinta para cada formato.

6.32 Diagramas de secuencia

Los diagramas de secuencia representan interacciones ordenadas en el tiempo.



La fuente:

```
```.plantuml
@startuml
actor Autor
participant "iasi.quarto" as IASI
participant Quarto
participant "PlantUML Server" as PlantUML

Autor -> IASI: build()
IASI -> Quarto: render
Quarto -> PlantUML: compilar diagrama
PlantUML --> Quarto: PNG
Quarto --> Autor: publicación
```

```
@enduml
...
```

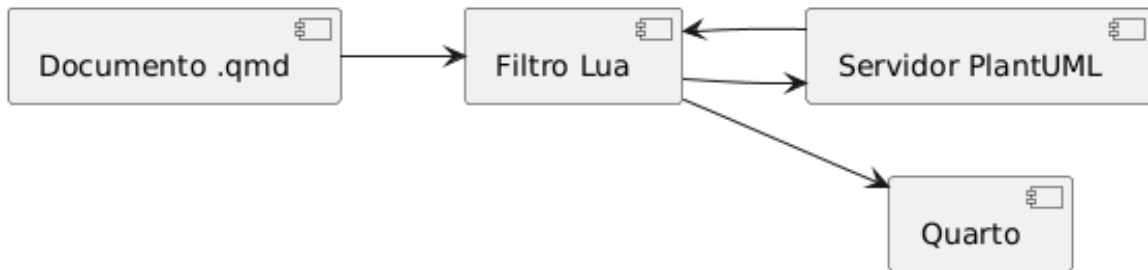
Pueden utilizarse:

```
actores
participantes
mensajes síncronos
respuestas
activaciones
fragmentos condicionales
bucles
notas
```

Conviene mantener los nombres coherentes con el vocabulario utilizado en el texto.

## 6.33 Diagramas de componentes

Los diagramas de componentes muestran piezas de un sistema y sus dependencias.



La fuente:

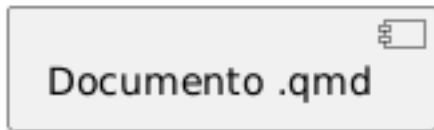
```
```.plantuml
@startuml
left to right direction

component "Documento .qmd" as Document
component "Filtro Lua" as Filter
component "Servidor PlantUML" as Server
component "Quarto" as Quarto

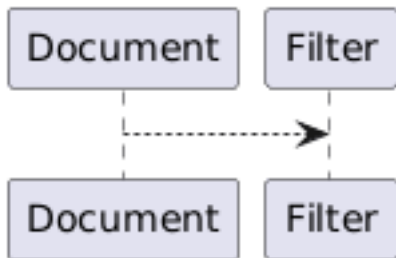
Document --> Filter
```

```
Filter --> Server
Server --> Filter
Filter --> Quarto
@enduml
```
```

Los alias:



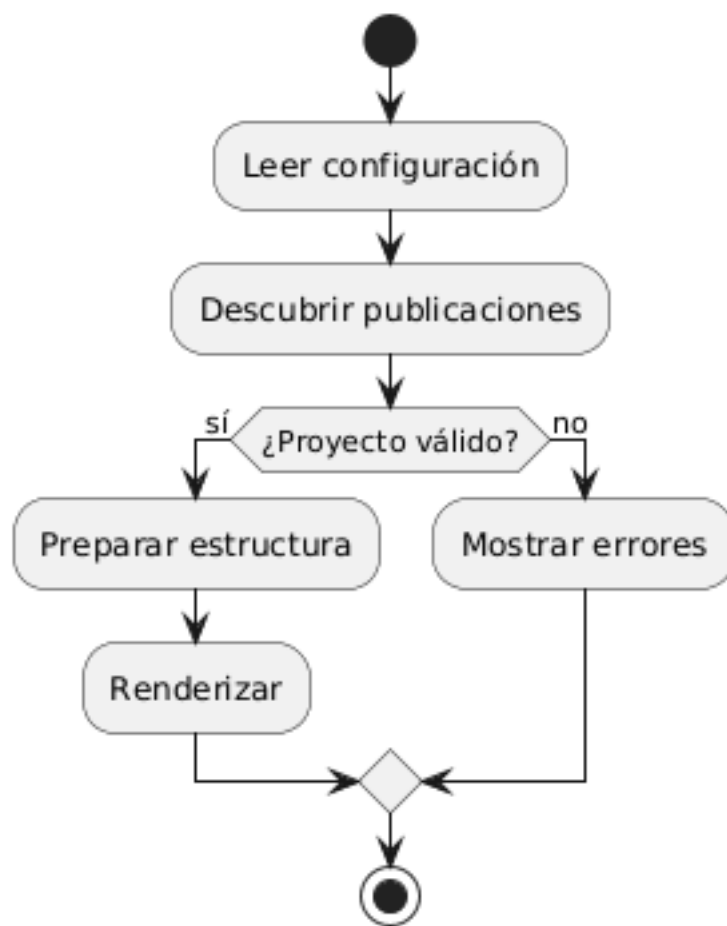
permiten utilizar una etiqueta legible en el diagrama y un identificador breve en las relaciones.



Esta práctica mejora la legibilidad cuando los nombres contienen espacios o son demasiado largos.

## 6.34 Diagramas de actividad

Los diagramas de actividad son útiles para describir procesos y decisiones.



La fuente:

```
```.plantuml
@startuml
start

:Leer configuración;
:Descubrir publicaciones;

if (¿Proyecto válido?) then (sí)
  :Preparar estructura;
  :Renderizar;
else (no)
  :Mostrar errores;
end
```

```
endif  
  
stop  
@enduml  
~~~
```

Este tipo de diagrama resulta apropiado para:

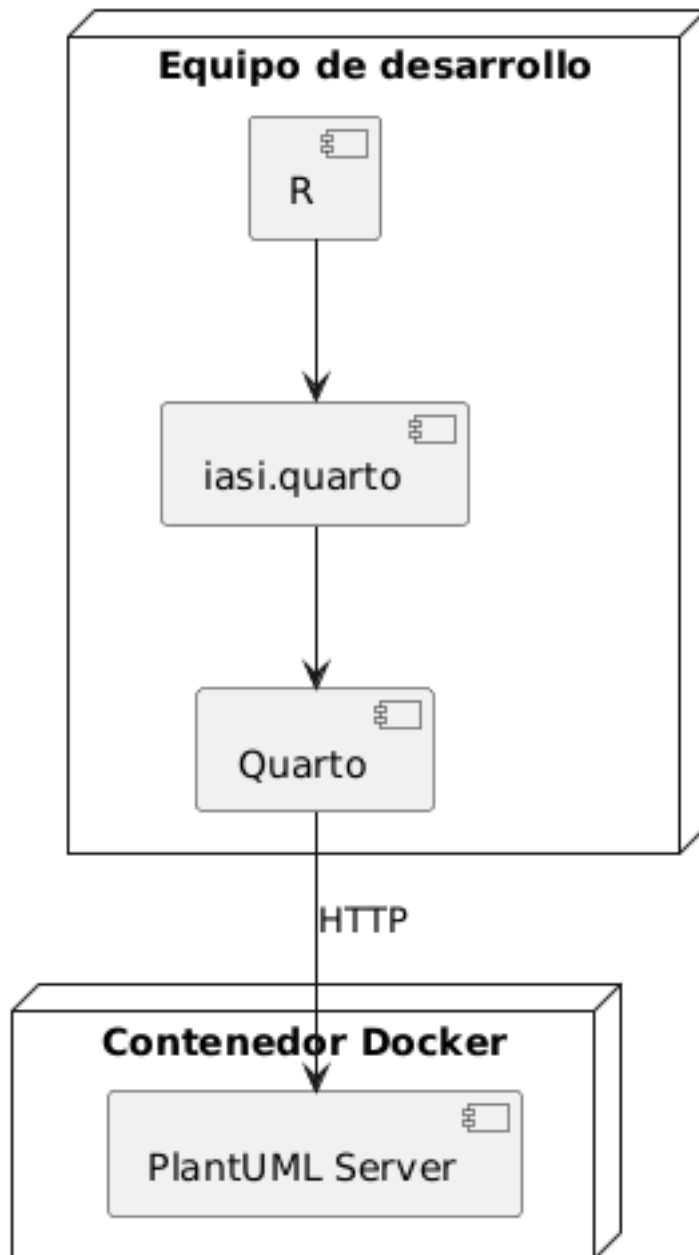
```
pipelines  
procesos de validación  
flujos de decisión  
procedimientos operativos  
ciclos de construcción
```

No debe utilizarse para sustituir una explicación necesaria.

El diagrama resume el flujo. El texto explica su significado y sus condiciones.

6.35 Diagramas de despliegue

Los diagramas de despliegue representan dónde se ejecutan los componentes.



La fuente:

```
```{.plantuml}
@startuml
node "Equipo de desarrollo" {
```

```

component R
component Quarto
component "iasi.quarto" as IASI
}

node "Contenedor Docker" {
 component "PlantUML Server" as Server
}

R --> IASI
IASI --> Quarto
Quarto --> Server : HTTP
@enduml
```

```

Este tipo de representación permite distinguir:

```

componentes lógicos
procesos
máquinas
contenedores
servicios de red
dependencias externas

```

6.36 Orientación

PlantUML elige automáticamente una disposición, pero puede indicarse una orientación general.

Horizontal:

Vertical:

Ejemplo:

```

```{.plantuml}
@startuml
left to right direction

rectangle Fuente
rectangle Compilador
rectangle Imagen

Fuente --> Compilador
Compilador --> Imagen
@enduml
```

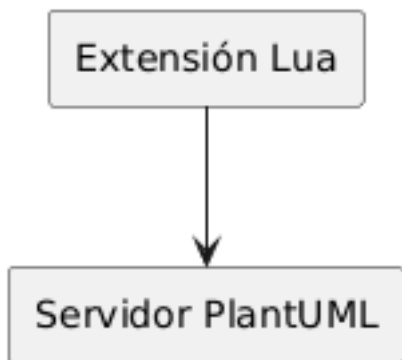
```

La orientación debe elegirse según la estructura del diagrama y el espacio disponible en la publicación.

Para PDF, los diagramas demasiado anchos pueden perder legibilidad al ajustarse al ancho de página.

6.37 Alias y nombres visibles

Los elementos pueden tener un nombre visible y un alias interno.



Esto evita repetir etiquetas largas:

```

nombre visible
  Servidor PlantUML

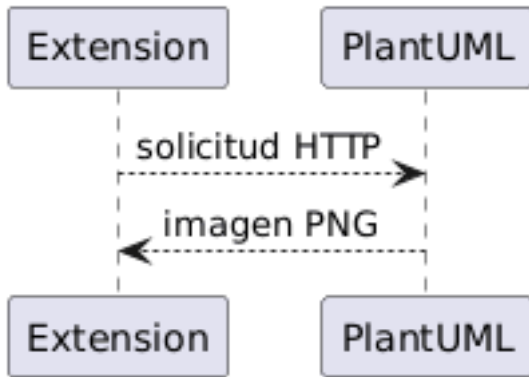
alias utilizado en la fuente
  PlantUML

```

Los alias también ayudan a mantener estable la fuente cuando cambia una etiqueta editorial.

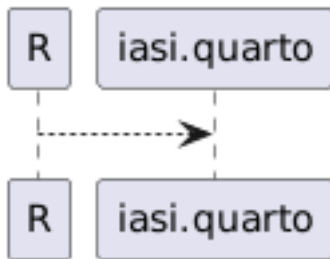
6.38 Etiquetas en las relaciones

Las relaciones pueden describirse:

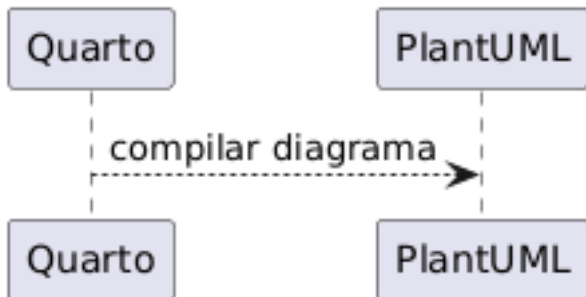


Las etiquetas deben aportar información que no resulte obvia por los nombres de los elementos.

Una relación sin contenido adicional puede mantenerse sin texto:



Una relación con significado operacional debería nombrarlo:



6.39 Notas

PlantUML permite añadir notas:



Ejemplo completo:

```
```.plantuml
@startuml
rectangle Quarto
rectangle PlantUML

Quarto --> PlantUML : fuente

note right of PlantUML
 La instancia puede ser
 local o compartida
end note
@enduml
```
```

Las notas son útiles para aclaraciones breves.

Cuando una explicación necesita varios párrafos, debe permanecer en el texto del documento y no dentro del diagrama.

6.40 Separar significado y apariencia

Los bloques deben concentrarse en:

```
elementos
relaciones
orden
agrupaciones
mensajes
decisiones
```

La apariencia común debe vivir en los archivos de estilo configurados para la publicación.

No conviene repetir en cada bloque:

```
colores corporativos
tipografías
grosos de borde
configuración visual general
```

La fuente del diagrama describe qué representa.

El estilo compartido decide cómo se presenta.

6.41 Un diagrama por bloque

Cada bloque `.plantuml` debe representar un único diagrama.

```
un bloque
-> una fuente
-> una imagen
-> una entrada de caché
```

No debe utilizarse un único bloque para producir varios diagramas independientes.

Separarlos permite:

```
colocarlos junto al texto correspondiente
reutilizar la caché de forma precisa
detectar qué fuente produjo cada imagen
mantener cambios pequeños y revisables
```

6.42 Ubicación dentro del documento

El diagrama debe situarse cerca del texto que explica.

Una secuencia natural es:

```
introducción del concepto  
diagrama  
explicación de sus elementos  
consecuencias o detalles
```

Evite acumular todos los diagramas al final de un documento.

El objetivo no es crear una galería, sino integrar cada representación en el argumento técnico.

6.43 Tamaño y complejidad

Un diagrama legible no intenta contener todo el sistema.

Cuando el número de elementos crece demasiado, conviene dividirlo por perspectiva:

```
visión general  
componentes principales  
flujo de una operación  
despliegue  
detalle de un subsistema
```

Una publicación puede contener varios diagramas relacionados, cada uno con una pregunta clara.

```
¿qué componentes existen?  
¿cómo se comunican?  
¿dónde se ejecutan?  
¿qué ocurre durante una operación?
```

Cada pregunta puede requerir una representación diferente.

6.44 Nombres estables

Los nombres utilizados en los diagramas deben coincidir con los nombres empleados en:

```
el texto  
la configuración  
el código  
otros diagramas
```

No conviene llamar al mismo elemento:

```
PlantUML Server  
servidor de diagramas  
motor gráfico  
servicio UML
```

sin una razón explícita.

La coherencia terminológica convierte los diagramas en parte de la documentación, no en ilustraciones aisladas.

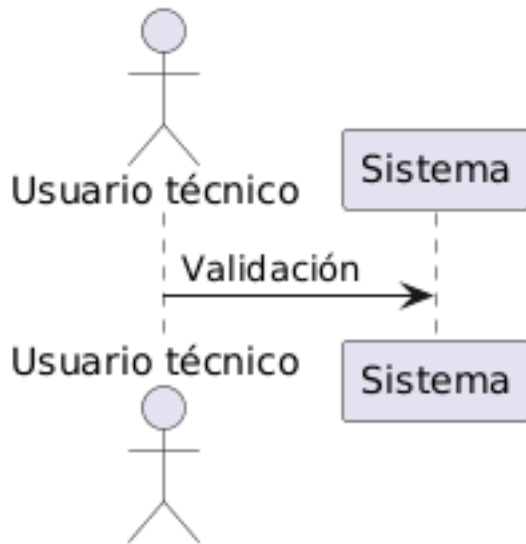
6.45 Caracteres y codificación

Los documentos y los archivos de estilo deben guardarse en UTF-8.

Esto permite utilizar correctamente:

```
acentos  
eñes  
símbolos  
etiquetas en castellano
```

Por ejemplo:



Cuando aparecen caracteres incorrectos, deben revisarse:

```
la codificación del archivo .qmd
la codificación del archivo .puml
la respuesta del servidor
la fuente utilizada por PlantUML
```

6.46 Opciones por bloque

Un bloque puede incluir atributos reconocidos por la extensión.

Por ejemplo, para evitar temporalmente la caché:

```
```{.plantuml cache="false"}
@startuml
Alice -> Bob: Compilación nueva
@enduml
```
```

Las opciones globales continúan viniendo de `_quarto.yml`.

Las opciones por bloque deben utilizarse únicamente cuando un diagrama necesita una excepción concreta.

El capítulo dedicado a la caché explica con más detalle su comportamiento.

6.47 Prueba mínima

Después de configurar la extensión, cree un documento con:

```
## Prueba de PlantUML

```{.plantuml}
@startuml
Alice -> Bob: Funciona
Bob --> Alice: PNG generado
@enduml
```
```

Construya HTML:

```
build(
  format = "html"
)
```

Después construya PDF:

```
build(
  format = "pdf"
)
```

El mismo diagrama debe aparecer en ambos resultados.

6.48 Criterio práctico

Un bloque PlantUML correcto debe cumplir cuatro condiciones:

```
la fuente es válida
el diagrama responde a una pregunta concreta
los nombres coinciden con el resto de la documentación
la complejidad permite leerlo en HTML y PDF
```

Un buen diagrama no contiene más información. Contiene la información adecuada, colocada donde el texto la necesita.

6.49 Estilos compartidos

La extensión puede incorporar uno o varios archivos PlantUML a todos los diagramas de una publicación.

La configuración recomendada utiliza:

```
filter-options:
  plantuml:
    styles:
      - resources/plantuml/iasi.puml
```

El archivo:

```
resources/plantuml/iasi.puml
```

centraliza la apariencia común de los diagramas.

```
documentos .qmd
  -> estructura y significado

iasi.puml
  -> apariencia compartida
```

Esta separación evita repetir reglas visuales dentro de cada bloque.

6.50 Estructura recomendada

Una publicación puede organizar los recursos PlantUML así:

```
publication/
  _quarto.yml
  resources/
    plantuml/
      iasi.puml
  chapters/
    01-intro/
    02-fundamentals/
```

La ruta declarada en `_quarto.yml` debe coincidir con la ubicación real:

```
styles:  
- resources/plantuml/iasi.puml
```

El archivo debe estar disponible durante la construcción y guardado en UTF-8.

6.51 Qué pertenece al estilo

Un archivo compartido puede definir:

```
colores  
tipografía  
bordes  
fondos  
sombras  
espaciado  
apariencia de flechas  
estilo de actores  
estilo de componentes  
estilo de nodos  
convenciones visuales
```

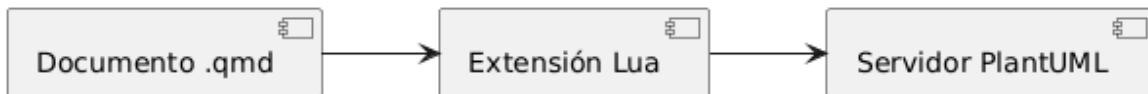
Por ejemplo:

Este contenido es únicamente un ejemplo de estructura.

Los colores, fuentes y convenciones definitivos pertenecen a la identidad visual de cada publicación.

6.52 Qué debe permanecer en el diagrama

El bloque `.plantuml` debe describir el contenido concreto:



No debería repetir reglas generales como:

Cuando esas reglas aparecen en cada documento:

```
aumenta la duplicación
los diagramas pueden divergir
los cambios visuales exigen editar muchos archivos
la fuente pierde legibilidad
```

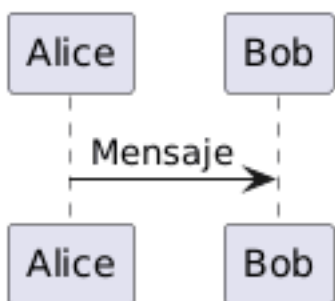
El diagrama debe concentrarse en elementos y relaciones.

6.53 Cómo se aplica el estilo

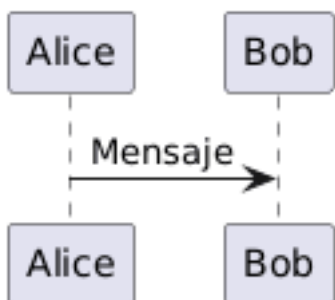
Antes de compilar un diagrama, la extensión lee los archivos configurados.

Después inserta su contenido inmediatamente después de la línea `@startuml`.

Una fuente original:



se prepara conceptualmente como:



El servidor recibe la fuente ya preparada.

Por tanto, el estilo forma parte de la compilación y también influye en la identidad utilizada por la caché.

```
cambia el estilo
-> cambia la fuente preparada
-> cambia la imagen generada
```

6.54 Varios archivos de estilo

La opción `styles` admite más de un archivo:

```
styles:
- resources/plantuml/base.puml
- resources/plantuml/iasi.puml
- resources/plantuml/architecture.puml
```

La extensión los lee en el orden declarado.

```
base.puml
-> iasi.puml
-> architecture.puml
-> contenido del diagrama
```

Este orden permite construir estilos por capas.

Por ejemplo:

```
base.puml
  reglas generales

iasi.puml
  identidad visual de la publicación

architecture.puml
  convenciones específicas para diagramas técnicos
```

Cuando dos archivos modifican la misma propiedad, la regla aplicada posteriormente puede reemplazar a la anterior según el comportamiento de PlantUML.

El orden debe ser deliberado y estable.

6.55 Un único archivo o varios

Para una publicación pequeña, un solo archivo suele ser suficiente:

```
resources/plantuml/iasi.puml
```

Cuando las reglas crecen, pueden separarse por responsabilidad:

```
resources/plantuml/  
  base.puml  
  sequence.puml  
  components.puml  
  deployment.puml
```

La separación resulta útil cuando:

```
las reglas tienen ámbitos claros  
varios libros comparten una base  
existen convenciones por tipo de diagrama  
los archivos siguen siendo fáciles de localizar
```

No conviene fragmentar el estilo sin una necesidad real.

Demasiados archivos convierten una decisión visual sencilla en una pequeña arqueología de includes.

6.56 Estilo base recomendado

Un punto de partida sencillo podría ser:

El objetivo no es decorar cada elemento.

El objetivo es conseguir que todos los diagramas pertenezcan visualmente a la misma publicación.

6.57 Estilos por tipo de elemento

PlantUML permite configurar familias concretas.

6.57.1 Rectángulos

6.57.2 Componentes

6.57.3 Secuencias

6.57.4 Actividades

No todas las publicaciones necesitan configurar todos los tipos.

Deben añadirse reglas cuando existe un diagrama real que las utiliza.

6.58 Colores semánticos

Un estilo puede definir convenciones visuales con significado.

Por ejemplo:

```
componente interno
  tono neutro

sistema externo
  tono distinto

error
  tono de alerta

artefacto generado
  borde discontinuo
```

Sin embargo, estas convenciones deben aplicarse de forma consistente.

Un mismo color no debería significar:

```
servicio externo en un diagrama
error en otro
base de datos en un tercero
```

La identidad gráfica sirve para reducir esfuerzo cognitivo, no para fabricar un acertijo cromático.

6.59 Macros y convenciones reutilizables

El archivo `.puml` también puede definir macros para elementos frecuentes.

Por ejemplo:

Welcome to PlantUML!

You can start with a simple UML Diagram like:

Bob->Alice: Hello

Or

class Example

You will find more information about PlantUML syntax on <https://plantuml.com>

(Details by typing `license` keyword)



```
PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:13:24 UTC]
```

```
This version of PlantUML is 218 days old, so you should
consider upgrading from https://plantuml.com/download
```

```
[From string (line 4) ]
```

```
@startuml
```

```
Empty description (Assumed diagram type: sequence)
```

Y después:

PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:13:24 UTC]

This version of PlantUML is 218 days old, so you should consider upgrading from <https://plantuml.com/download>

[From string (line 2)]

@startuml

INTERNAL_COMPONENT(API, Api)

Syntax Error? (Assumed diagram type: sequence)

Las macros pueden reducir repetición cuando representan una convención real del proyecto.

Deben utilizarse con moderación.

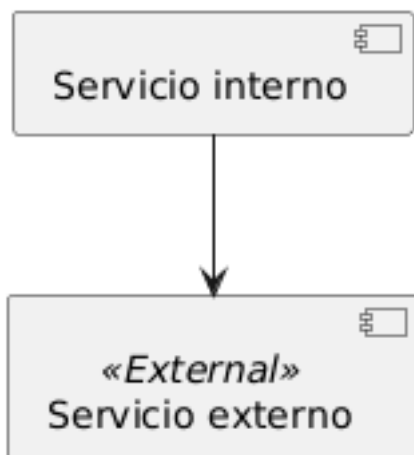
Una macro demasiado abstracta puede ocultar la sintaxis y dificultar que otro autor comprenda el diagrama.

6.60 Estereotipos

Los estereotipos permiten aplicar estilos diferenciados.

Por ejemplo, en el archivo compartido:

Y en el documento:



Esta técnica mantiene el significado dentro de la fuente:

```
<<External>>
```

y la apariencia dentro del estilo compartido.

6.61 Fuentes

La fuente configurada debe existir en el entorno donde PlantUML genera la imagen.

Un nombre como:

puede funcionar en un equipo y no estar disponible dentro de otro servidor o contenedor.

Para una construcción reproducible deben utilizarse fuentes disponibles en la infraestructura elegida.

Cuando la fuente no existe, PlantUML puede sustituirla por otra.

Eso puede alterar:

```
anchos  
saltos de línea  
tamaño de cajas  
distribución del diagrama
```

La tipografía también forma parte del entorno de renderizado.

6.62 Transparencia y fondo

Para integrar los diagramas en HTML y PDF puede utilizarse un fondo transparente:

Esto permite que la imagen adopte el fondo visual del documento.

Sin embargo, todos los textos, líneas y rellenos deben mantener contraste suficiente tanto en pantalla como en impresión.

Un diagrama diseñado únicamente para un fondo oscuro puede resultar ilegible en PDF.

La configuración común debe probarse en ambos formatos.

6.63 Cambiar el estilo

Cuando se modifica `iasi.puml`, los diagramas deben recompilarse con la nueva fuente preparada.

La extensión calcula la caché después de incorporar el estilo.

Por tanto, un cambio real en el contenido del archivo debe producir una identidad diferente.

El flujo es:

```
modificar iasi.puml
-> preparar de nuevo la fuente
-> generar otra identidad
-> solicitar un PNG nuevo
```

Si durante el desarrollo se sospecha que se está reutilizando una imagen antigua, puede limpiarse únicamente la caché PlantUML:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

No es necesario eliminar imágenes SVG o PNG de forma indiscriminada por todo el proyecto.

6.64 Estilo y caché

El estilo no es una decoración aplicada después de compilar.

Forma parte de la fuente enviada al servidor.

Eso significa que dos diagramas con el mismo cuerpo, pero con estilos diferentes, no representan la misma compilación.

```
mismo diagrama
estilo A
-> imagen A

mismo diagrama
estilo B
-> imagen B
```

La caché debe distinguir ambos resultados.

El capítulo siguiente desarrolla su funcionamiento completo.

6.65 Probar el estilo

Una prueba mínima puede incluir varios tipos de elementos:

```
```.plantuml
@startuml
left to right direction

actor Usuario
component "iasi.quarto" as IASI
rectangle Quarto
database Cache

Usuario --> IASI
IASI --> Quarto
IASI --> Cache
@enduml
```
```

Después deben construirse ambos formatos:

```
build(
  format = "html"
)

build(
  format = "pdf"
)
```

La revisión debe comprobar:

```
legibilidad de textos
contraste
consistencia de colores
tamaño de los elementos
anchura en PDF
apariencia con fondo HTML
```

6.66 Criterio de diseño

Un buen estilo compartido cumple tres funciones:

```
unifica
  los diagramas parecen pertenecer al mismo libro

simplifica
  los bloques contienen menos ruido visual

estabiliza
  una decisión gráfica se cambia en un único lugar
```

El documento explica el sistema. El archivo de estilo evita que cada diagrama tenga que inventar de nuevo su propio idioma visual.

6.67 Caché de diagramas

La extensión PlantUML puede conservar los diagramas compilados y reutilizarlos en construcciones posteriores.

La configuración recomendada activa esta capacidad:

```
filter-options:
  plantuml:
    cache: true
```

La caché evita solicitar al servidor una imagen que ya fue generada con la misma fuente y la misma configuración relevante.

```
diagrama sin cambios
  -> reutilizar imagen

diagrama modificado
  -> compilar de nuevo
```

No sustituye al servidor PlantUML ni a la fuente del diagrama.

Es un almacén local de artefactos derivados.

6.68 Por qué existe

Sin caché, cada construcción tendría que enviar de nuevo todos los diagramas al servidor.

```
10 diagramas  
-> 10 peticiones en cada build
```

Con caché:

```
10 diagramas sin cambios  
-> 0 compilaciones nuevas  
  
1 diagrama modificado  
-> 1 compilación nueva
```

Esto aporta:

```
construcciones más rápidas  
menos tráfico hacia el servidor  
menos trabajo repetido  
mayor estabilidad durante el desarrollo
```

La mejora resulta especialmente visible en publicaciones grandes o cuando el servidor se encuentra en otra máquina.

6.69 Ubicación

La caché de PlantUML se guarda dentro del espacio de trabajo de Quarto:

```
.quarto/plantuml/
```

Su contenido pertenece a la construcción local.

No forma parte del contenido editorial de la publicación.

```
chapters/  
resources/  
_quarto.yml  
-> fuentes del proyecto  
  
.quarto/plantuml/  
-> artefactos derivados
```

La carpeta puede eliminarse y regenerarse.

6.70 Identidad de un diagrama

La extensión calcula una identidad para cada compilación.

En la implementación actual intervienen:

```
fuelle preparada  
servidor configurado  
formato de salida  
versión de la extensión
```

La fuente preparada incluye el contenido de los estilos compartidos.

Por tanto:

```
cambia el bloque  
  -> cambia la identidad  
  
cambia iasi.puml  
  -> cambia la identidad  
  
cambia el servidor  
  -> cambia la identidad  
  
cambia el formato  
  -> cambia la identidad  
  
cambia la versión de la extensión  
  -> cambia la identidad
```

La identidad se calcula mediante un resumen criptográfico del contenido relevante.

El nombre final no intenta ser legible para una persona.

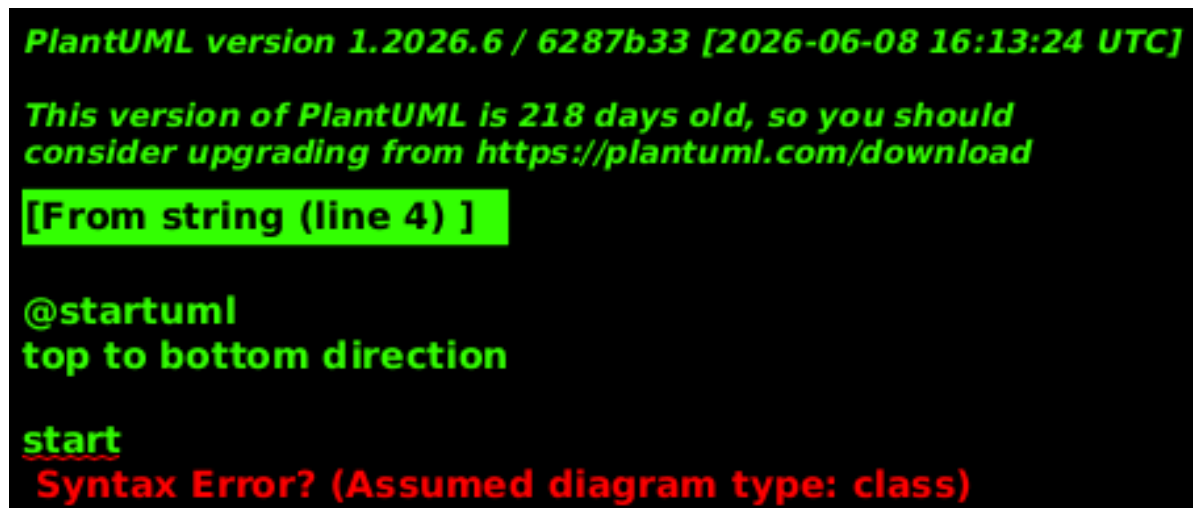
Su función es representar de forma estable una compilación concreta.

6.71 Flujo de lectura

Cuando la extensión encuentra un bloque PlantUML:

1. lee la configuración
2. incorpora los estilos
3. calcula la identidad
4. consulta la caché
5. reutiliza o compila
6. entrega la imagen a Pandoc

El flujo puede representarse así:



```
PlantUML version 1.2026.6 / 6287b33 [2026-06-08 16:13:24 UTC]

This version of PlantUML is 218 days old, so you should
consider upgrading from https://plantuml.com/download

[From string (line 4) ]

@startuml
top to bottom direction

start
Syntax Error? (Assumed diagram type: class)
```

6.72 Acierto de caché

Existe un acierto de caché cuando la identidad calculada ya tiene una imagen almacenada.

```
cache hit
-> no se llama al servidor
-> se reutiliza la imagen
```

Esto ocurre normalmente cuando:

```
el bloque no ha cambiado
los estilos no han cambiado
la configuración relevante no ha cambiado
la entrada sigue presente
```

El resultado debe ser idéntico al de la compilación original.

6.73 Fallo de caché

Existe un fallo de caché cuando no se encuentra una entrada válida.

```
cache miss
-> se llama al servidor
-> se recibe una imagen
-> se almacena
```

Un fallo de caché no es un error.

Es el comportamiento normal para:

```
un diagrama nuevo
un diagrama modificado
un estilo modificado
una caché recién eliminada
una versión nueva de la extensión
```

6.74 Caché activada

La configuración habitual es:

```
cache: true
```

Con ella, las construcciones repetidas reutilizan resultados.

Esta opción es apropiada para:

```
desarrollo local
documentación extensa
servidores compartidos
integración continua con caché persistente
```

La caché debe considerarse una optimización segura, no una fuente de verdad.

6.75 Caché desactivada

Puede desactivarse globalmente:

```
filter-options:  
  plantuml:  
    cache: false
```

En ese modo, cada diagrama se solicita de nuevo al servidor.

```
cada build  
  -> compilar todos los diagramas
```

Esto puede ser útil para:

```
probar el servidor  
diagnosticar una modificación del compilador  
comparar resultados  
confirmar que una imagen se regenera
```

No es recomendable como configuración permanente cuando la publicación contiene numerosos diagramas.

6.76 Desactivar la caché en un bloque

La infraestructura admite una excepción local:

```
```.plantuml cache="false"  
@startuml
Alice -> Bob: Compilación forzada
@enduml
```
```

Esto permite probar un diagrama concreto sin cambiar la configuración de toda la publicación.

Debe utilizarse de forma temporal.

Una publicación no debería mezclar arbitrariamente diagramas con y sin caché sin una razón documentada.

6.77 Limpiar la caché

Cuando es necesario regenerar todos los diagramas, debe eliminarse únicamente la caché de PlantUML.

En PowerShell:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

Después, la siguiente construcción vuelve a solicitar las imágenes al servidor.

```
build(  
  format = "html"  
)
```

No es necesario eliminar:

```
todos los archivos PNG del proyecto  
todos los archivos SVG  
la carpeta .quarto completa  
los directorios de salida HTML o PDF
```

Una limpieza quirúrgica evita destruir otros artefactos útiles.

6.78 Cuándo limpiar

La limpieza manual puede ser apropiada cuando:

```
se ha modificado el compilador  
se sospecha que una entrada está dañada  
se ha cambiado la política de formato  
se está probando una versión nueva del servidor  
se necesita reproducir una construcción limpia
```

No debería ser el primer paso ante cualquier error.

Antes conviene identificar si el problema pertenece a:

```
la fuente PlantUML
la configuración
el servidor
la caché
Quarto
```

Borrar la caché puede ocultar temporalmente un problema sin explicar su causa.

6.79 Cambios de estilo

Los archivos de estilo se incorporan antes de calcular la identidad.

Por tanto, una modificación real en:

```
resources/plantuml/iasi.puml
```

debe producir nuevas imágenes.

```
estilo anterior
-> identidad A
-> imagen A

estilo nuevo
-> identidad B
-> imagen B
```

En condiciones normales no es necesario limpiar manualmente la caché al cambiar el estilo.

La limpieza solo resulta necesaria cuando se sospecha una incoherencia, una entrada dañada o un problema durante el desarrollo de la propia extensión.

6.80 Cambio de formato

La configuración recomendada utiliza:

```
format: png
```

El formato participa en la identidad y en el nombre del recurso generado.

```
misma fuente + PNG
-> entrada PNG

misma fuente + SVG
-> entrada SVG
```

La implementación actual normaliza la salida efectiva a PNG.

Por ello, la configuración y la caché deben mantener esa misma decisión.

Cuando se cambia la política de formato de la extensión, conviene limpiar:

```
.quarto/plantuml/
```

para evitar conservar artefactos pertenecientes a una etapa anterior del diseño.

6.81 Cambio de servidor

La dirección del servidor también participa en la identidad.

```
server: http://localhost:1025
```

y:

```
server: http://plantuml.local:1025
```

producen identidades distintas aunque la fuente sea igual.

Esta decisión evita asumir que dos servidores diferentes generan necesariamente el mismo resultado.

Pueden utilizar:

```
versiones distintas de PlantUML
fuentes distintas
configuraciones distintas
dependencias distintas
```

El servidor forma parte del entorno de compilación.

6.82 Versión de la extensión

La versión de la extensión participa en la identidad.

Cuando cambia el comportamiento del compilador, una versión nueva evita reutilizar resultados generados por una implementación anterior.

```
extensión 0.1.0  
  -> caché A  
  
extensión 0.2.0  
  -> caché B
```

Esto es importante cuando la extensión modifica:

```
la preparación de la fuente  
la construcción de la URL  
el formato  
el MIME  
la incorporación de estilos
```

La versión no es únicamente información descriptiva.

También protege la coherencia de los artefactos derivados.

6.83 Caché y disponibilidad del servidor

Un diagrama presente en caché puede reutilizarse aunque el servidor no responda.

```
entrada existente  
  -> construcción posible  
  
entrada ausente  
  -> se necesita el servidor
```

Esto mejora la tolerancia durante el desarrollo, pero tiene un límite claro.

Una construcción limpia necesita acceso al servidor para regenerar todos los diagramas.

Por tanto, la caché no debe utilizarse como sustituto de una infraestructura PlantUML disponible.

6.84 Caché en integración continua

En un pipeline de integración continua pueden adoptarse dos estrategias.

6.84.1 Construcción limpia

```
cada ejecución comienza sin caché
```

Ventajas:

```
máxima independencia de artefactos previos  
verificación completa del servidor  
comportamiento fácil de reproducir
```

Coste:

```
todos los diagramas se compilan de nuevo
```

6.84.2 Caché persistente

```
el pipeline conserva .quarto/plantuml
```

Ventajas:

```
construcciones más rápidas  
menos solicitudes al servidor
```

Riesgos:

```
la clave externa de caché debe estar bien diseñada  
la carpeta puede crecer  
debe poder invalidarse
```

La estrategia elegida debe ser explícita.

La publicación no debe depender de una caché persistente para ser construible.

6.85 Versionado

La carpeta:

```
.quarto/plantuml/
```

normalmente no debe almacenarse en Git.

La fuente verdadera está en:

```
documentos .qmd  
archivos .puml  
configuración YAML  
código de la extensión
```

La caché puede regenerarse a partir de esas fuentes y del servidor.

Por eso suele incluirse `.quarto/` en `.gitignore`.

El repositorio debe conservar la receta, no cada plato que la cocina produjo ayer.

6.86 Crecimiento de la caché

Cada identidad distinta puede crear una nueva entrada.

La carpeta puede crecer cuando se realizan numerosos cambios en:

```
diagramas  
estilos  
servidores  
versiones  
formatos
```

La extensión no necesita borrar automáticamente todas las entradas antiguas durante cada construcción.

Una limpieza periódica puede hacerse cuando:

```
el tamaño deja de ser razonable  
se termina una etapa de desarrollo  
se cambia de versión principal  
se prepara una construcción limpia
```

El comando continúa siendo:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

6.87 Diagnóstico

Cuando un diagrama parece antiguo, conviene seguir este orden:

1. comprobar que el archivo .qmd fue guardado
2. comprobar que el estilo .puml fue guardado
3. revisar la configuración efectiva
4. confirmar qué compiler.lua está cargando Quarto
5. limpiar .quarto/plantuml
6. construir de nuevo

Si la URL del error contiene:

```
/svg/
```

cuando se espera PNG, el problema no debe atribuirse automáticamente a la caché.

Puede indicar:

```
otra copia de la extensión  
una configuración distinta  
un compiler.lua no actualizado  
una normalización de formato que no se está ejecutando
```

La propia URL de PlantUML muestra el formato efectivo utilizado por el compilador.

6.88 Prueba de funcionamiento

Puede comprobarse la caché con un diagrama pequeño.

Primera construcción:

```
build(  
  format = "html"  
)
```

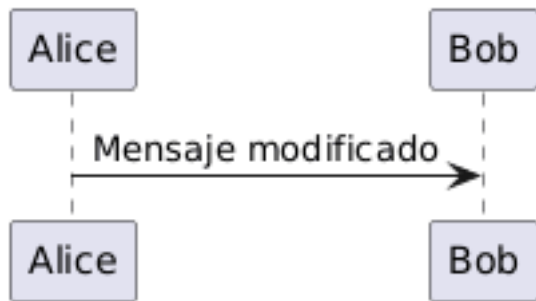
El servidor debe recibir la compilación.

Segunda construcción, sin cambios:

```
build(  
  format = "html"  
)
```

La imagen debe reutilizarse.

Después, cambie el texto:



La siguiente construcción debe generar una entrada nueva.

Finalmente, restaure el contenido original.

La extensión podrá reutilizar la entrada anterior si todavía existe.

6.89 Contrato de la caché

La caché debe cumplir cuatro propiedades:

```
determinista  
  la misma entrada produce la misma identidad  
  
invalidable  
  los cambios relevantes generan otra identidad  
  
desechable  
  puede eliminarse sin perder fuentes  
  
localizada  
  puede limpiarse sin borrar todo el proyecto
```

La caché acelera el pipeline, pero nunca debe convertirse en la única memoria de cómo se construyó un diagrama.

6.90 Parámetros de los bloques PlantUML

Un bloque PlantUML puede contener parámetros que modifican su compilación y atributos que controlan cómo se presenta la imagen dentro de Quarto.

No pertenecen al mismo nivel.

```
parámetros de compilación
  modifican cómo se obtiene la imagen

atributos de presentación
  modifican cómo Quarto coloca la imagen
```

Esta distinción evita recompilar un diagrama únicamente porque cambia su tamaño, alineación o pie de figura.

6.91 Forma general

Los parámetros se declaran en la cabecera del bloque:

```
```{.plantuml width="80%" fig-cap="Pipeline de construcción"}
@startuml
Fuente -> Compilador
Compilador -> Imagen
@enduml
```
```

La clase:

```
.plantuml
```

identifica el bloque que debe procesar la extensión.

Los demás valores se escriben como atributos:

```
nombre="valor"
```

6.92 Parámetros de compilación

Los parámetros de compilación modifican el proceso utilizado para obtener la imagen.

La extensión reconoce:

```
server  
format  
cache
```

Estos parámetros pueden declararse globalmente en `_quarto.yml` o reemplazarse para un bloque concreto.

6.93 server

`server` selecciona el servidor PlantUML utilizado por el bloque.

Configuración global:

```
filter-options:  
  plantuml:  
    server: http://localhost:1025
```

Excepción por bloque:

```
```{.plantuml server="http://plantuml.local:1025"}  
@startuml
Alice -> Bob: Mensaje
@enduml
```
```

La URL debe señalar la raíz del servidor.

No debe incluir:

```
/png  
/svg
```

La extensión añade el formato al construir la petición.

El uso por bloque debe reservarse para situaciones concretas. Una publicación normal debería utilizar un servidor común.

6.94 format

`format` indica el formato gráfico solicitado al servidor.

La configuración recomendada es:

```
format: png
```

También puede declararse en el bloque:

```
```{.plantuml format="png"}
@startuml
Alice -> Bob: Mensaje
@enduml
```
```

La plataforma utiliza actualmente PNG como salida efectiva común para HTML y PDF.

Por tanto, la configuración y los bloques deberían mantener:

```
format="png"
```

El formato participa en la identidad de la caché porque modifica el recurso generado.

6.95 cache

`cache` controla si el bloque puede reutilizar una compilación anterior.

Configuración global:

```
cache: true
```

Excepción por bloque:

```
```{.plantuml cache="false"}
@startuml
Alice -> Bob: Compilación nueva
@enduml
```
```

Valores:

```
true
  reutiliza la caché cuando existe

false
  solicita una compilación nueva
```

Desactivar la caché por bloque resulta útil durante pruebas y diagnóstico.

No debería utilizarse de forma permanente sin una razón concreta.

6.96 Atributos de presentación

Los atributos de presentación no modifican la fuente enviada a PlantUML.

Controlan la imagen que Quarto incorpora al documento.

Los principales son:

```
width
height
fig-cap
fig-alt
fig-align
identificador
```

Estos atributos no deberían participar en la identidad de compilación.

```
mismo diagrama + width="50%"
mismo diagrama + width="100%"
  -> una única imagen compilada
  -> dos presentaciones diferentes
```

6.97 width

`width` establece el ancho con el que Quarto presenta la imagen.

Porcentaje:

```
```{.plantuml width="80%"}
@startuml
Fuente -> Imagen
@enduml
```
```

Unidad física:

```
```{.plantuml width="14cm"}
@startuml
Fuente -> Imagen
@enduml
```
```

Otros ejemplos válidos dependen del formato de salida y de las unidades admitidas por Pandoc y Quarto:

```
600px
12cm
5in
75%
```

Para publicaciones HTML y PDF, los porcentajes y las unidades físicas pueden producir resultados diferentes.

Debe comprobarse el resultado en ambos formatos.

6.98 height

`height` establece la altura de presentación.

```
```{.plantuml height="8cm"}
@startuml
Fuente -> Imagen
@enduml
```
```

También puede combinarse con `width`:

```
```{.plantuml width="14cm" height="8cm"}
@startuml
Fuente -> Imagen
@enduml
```
```

Sin embargo, declarar ambas dimensiones puede deformar la imagen cuando la relación de aspecto no coincide con el PNG generado.

La práctica recomendada es indicar únicamente `width` y permitir que Quarto conserve la proporción original.

```
recomendado
width="80%"

utilizar con cuidado
width="14cm" height="8cm"
```

6.99 fig-cap

`fig-cap` añade un pie de figura.

```
```{.plantuml fig-cap="Pipeline de construcción de la publicación"}
@startuml
Fuente -> Preparación
Preparación -> Quarto
@enduml
```
```

El texto se procesa como contenido Markdown.

Puede combinarse con un identificador para crear referencias cruzadas.

6.100 Identificador

El identificador se declara con `#`.

```

```{.plantuml #fig-build-pipeline fig-cap="Pipeline de construcción"}
@startuml
Fuente -> Preparación
Preparación -> Quarto
@enduml
```

```

Después puede referenciarse desde el texto:

Como muestra la @fig-build-pipeline, la preparación ocurre antes del renderizado.

Para que Quarto lo trate como figura referenciable, el identificador debe seguir la convención:

```
#fig-
```

Ejemplo:

```

#fig-plantuml-server
#fig-build-pipeline
#fig-publication-flow

```

6.101 fig-alt

fig-alt proporciona una descripción alternativa para accesibilidad.

```

```{.plantuml
#fig-plantuml-flow
fig-cap="Flujo de compilación PlantUML"
fig-alt="Un documento Quarto pasa por la extensión Lua, el servidor PlantUML y termina conver
}
@startuml
Documento -> Extension
Extension -> Servidor
Servidor -> PNG
@enduml
```

```

El texto alternativo debe explicar la información principal del diagrama.

No debe limitarse a repetir el pie de figura.

```
pie
  nombre editorial de la figura

texto alternativo
  contenido y relaciones importantes
```

6.102 fig-align

fig-align controla la alineación de la figura.

```
```{.plantuml fig-align="center" width="75%"}
@startuml
Fuente -> Imagen
@enduml
```
```

Valores habituales:

```
left
center
right
```

La configuración más común para diagramas es:

```
fig-align="center"
```

La alineación pertenece a la presentación y no modifica la compilación.

6.103 caption

La extensión puede admitir `caption` como alias de `fig-cap`.

```
```{.plantuml caption="Diagrama de secuencia"}
@startuml
Alice -> Bob: Mensaje
@enduml
```
```

Para mantener coherencia con Quarto, la forma recomendada es:

```
fig-cap
```

6.104 label

Puede admitirse `label` como alternativa al identificador del bloque.

Sin embargo, la sintaxis natural de Quarto es:

```
```{.plantuml #fig-example}  
...
```
```

Por claridad editorial, debe preferirse el identificador `#fig-...`

6.105 Combinación recomendada

Un bloque completo puede escribirse así:

```
```{.plantuml  
#fig-publication-pipeline
width="80%"
fig-align="center"
fig-cap="Pipeline de una publicación IASI Quarto"
fig-alt="La fuente documental pasa por iasi.quarto, Quarto y el servidor PlantUML antes de p
}
@startuml
left to right direction

rectangle "Fuente" as Source
rectangle "iasi.quarto" as IASI
rectangle "Quarto" as Quarto
rectangle "PlantUML" as PlantUML
rectangle "HTML / PDF" as Output

Source --> IASI
IASI --> Quarto
Quarto --> PlantUML
PlantUML --> Quarto
```
```

```
Quarto --> Output
@enduml
...
```

En este ejemplo:

```
#fig-publication-pipeline
  identifica la figura

width
  controla el tamaño

fig-align
  controla la posición

fig-cap
  proporciona el pie

fig-alt
  describe el contenido

la fuente PlantUML
  define elementos y relaciones
```

6.106 Qué parámetros modifican la caché

Deben participar en la identidad de compilación:

```
fuentes preparada
server
format
versión de la extensión
estilos incorporados
```

No deben participar:

```
width
height
fig-cap
fig-alt
```

```
fig-align
identificador
```

La razón es sencilla:

```
compilación
  decide qué imagen se genera

presentación
  decide cómo se utiliza esa imagen
```

Cambiar el pie de figura no debería llamar al servidor PlantUML.

Cambiar el ancho tampoco.

Cambiar la fuente o el estilo sí debe generar una imagen nueva.

6.107 Parámetros globales y parámetros de bloque

No todas las opciones globales tienen sentido dentro de un bloque.

Configuración global:

```
enabled
styles
server
format
cache
```

Configuración por bloque:

```
server
format
cache
width
height
fig-cap
fig-alt
fig-align
identificador
```

`enabled` controla la extensión completa.

`styles` define una política común de la publicación.

Por tanto, no deberían modificarse arbitrariamente en cada diagrama.

6.108 Tabla de referencia

| Parámetro | Ámbito | Función | Afecta a la compilación |
|------------------------|-----------------|------------------------------------|---|
| <code>enabled</code> | Global | Activa o desactiva la extensión | Sí, detiene el procesamiento |
| <code>styles</code> | Global | Incorpora estilos compartidos | Sí |
| <code>server</code> | Global o bloque | Selecciona el servidor PlantUML | Sí |
| <code>format</code> | Global o bloque | Selecciona el formato gráfico | Sí |
| <code>cache</code> | Global o bloque | Activa o evita la reutilización | No cambia la imagen, pero cambia el proceso |
| <code>width</code> | Bloque | Controla el ancho de presentación | No |
| <code>height</code> | Bloque | Controla la altura de presentación | No |
| <code>fig-cap</code> | Bloque | Añade el pie de figura | No |
| <code>fig-alt</code> | Bloque | Añade descripción alternativa | No |
| <code>fig-align</code> | Bloque | Controla la alineación | No |
| <code>#fig-...</code> | Bloque | Identifica la figura | No |

6.109 Criterio de uso

La regla práctica es:

`si cambia la imagen que debe producir PlantUML`
`es un parámetro de compilación`

`si cambia cómo Quarto presenta la misma imagen`
`es un atributo de presentación`

PlantUML genera la imagen. Quarto decide cuánto ocupa, dónde aparece y cómo se referencia.

6.110 Resolución de problemas

Los errores de PlantUML pueden aparecer en distintas etapas del pipeline.

```
carga de la extensión  
lectura de configuración  
preparación de la fuente  
acceso al servidor  
compilación  
caché  
integración con Quarto
```

El mensaje final no siempre identifica por sí solo la causa real.

La forma más eficaz de diagnosticar consiste en localizar primero qué componente ha fallado.

6.111 Orden de diagnóstico

Siga este orden:

1. comprobar que el filtro cargado es el correcto
2. comprobar la configuración efectiva
3. comprobar que el servidor responde
4. comprobar la fuente PlantUML
5. comprobar los archivos de estilo
6. comprobar la caché
7. revisar la integración con Quarto

No conviene comenzar borrando archivos al azar.

Cada etapa deja señales distintas.

6.112 El filtro no se carga

La publicación debe declarar:

```
filters:  
- extensions/plantuml/plantuml.lua
```

Y debe existir:

```
extensions/plantuml/plantuml.lua
```

La extensión también depende del núcleo común:

```
extensions/core/
```

Una copia incompleta puede contener `plantuml.lua`, pero carecer de:

```
engine.lua  
config.lua  
cache.lua  
mediabag.lua  
filesystem.lua  
metadata.lua
```

La estructura debe conservarse completa.

Una referencia relativa incorrecta dentro de `plantuml.lua` también impide cargar módulos como:

```
compiler.lua  
defaults.lua  
engine.lua
```

Cuando el error menciona `require`, `dofile` o una ruta Lua, debe comprobarse qué archivo exacto está intentando cargar Quarto.

6.113 Se está editando otra copia

Es posible tener varias copias de la extensión:

```
dentro de la publicación  
en otro proyecto  
en recursos globales de Quarto  
en una instalación anterior
```

El error suele mostrar la ruta real del archivo ejecutado.

Por ejemplo:

```
...\resources\quarto\extensions\plantuml\compiler.lua
```

Esa ruta es la que debe revisarse.

Modificar otro `compiler.lua` no cambia el comportamiento de la copia que Quarto está cargando.

La primera pregunta debe ser:

```
¿Qué archivo exacto aparece en el error?
```

6.114 Error de sintaxis Lua

Un error como:

```
'end' expected near <eof>
```

indica que Lua ha llegado al final del archivo sin encontrar el cierre esperado.

Las causas habituales son:

```
una función sin end  
un if sin end  
un bucle sin end  
un archivo pegado dos veces  
una edición truncada
```

Cuando el mensaje indica:

```
to close 'function' at line 134
```

debe revisarse esa función y el final del archivo.

Un `compiler.lua` válido debe terminar con:

```
return Compiler
```

La solución no consiste en añadir un `end` al azar.

Debe comprobarse si el archivo contiene contenido duplicado o incompleto.

6.115 El servidor no responde

Compruebe primero la URL base:

```
curl http://localhost:1025
```

Después revise el contenedor:

```
docker ps --filter name=plantuml-server
```

Y sus mensajes:

```
docker logs plantuml-server
```

Si el contenedor no está en ejecución:

```
docker start plantuml-server
```

Con Docker Compose:

```
docker compose up -d
```

Cuando el servidor está en otra máquina:

```
curl http://plantuml.local:1025
```

La prueba debe ejecutarse desde el mismo entorno que construye Quarto.

6.116 localhost apunta al lugar equivocado

Dentro de:

```
una máquina virtual  
un contenedor  
un agente CI  
otro equipo
```

localhost representa ese entorno.

No representa automáticamente la máquina anfitriona.

Si PlantUML se ejecuta fuera del entorno de construcción, configure una dirección accesible:

```
server: http://plantuml.local:1025
```

También deben comprobarse:

```
resolución del nombre  
puerto publicado  
reglas de red  
firewall  
ruta entre ambos entornos
```

6.117 La URL contiene /svg/

La URL solicitada al servidor revela el formato efectivo.

```
http://localhost:1025/svg/...
```

significa que el compilador está solicitando SVG.

```
http://localhost:1025/png/...
```

significa que está solicitando PNG.

Si la configuración declara:

```
format: png
```

pero el error muestra `/svg/`, las causas probables son:

```
se está cargando otra copia del compilador
la modificación no fue guardada
la normalización del formato no se ejecuta
otra configuración reemplaza el valor
se está utilizando código antiguo
```

No debe atribuirse automáticamente a Quarto.

La ruta pertenece a la petición construida por el compilador PlantUML.

6.118 Forzar temporalmente una comprobación

Puede añadirse temporalmente dentro de `Compiler.compile()`:

```
error(
  "DEBUG compiler: "
  .. toString(config.format)
)
```

La construcción se detendrá mostrando el valor efectivo.

Después de comprobarlo, debe eliminarse el diagnóstico.

No conviene dejar errores deliberados dentro de la extensión.

6.119 SVG y construcción PDF

Un flujo basado en SVG puede hacer que Quarto intente convertir la imagen para incorporarla al PDF.

El error puede mencionar una herramienta como:

```
rsvg-convert
```

Esto indica que el pipeline ha generado SVG y necesita una conversión adicional.

La configuración recomendada evita esa dependencia utilizando:

```
format: png
```

El flujo esperado es:

```
PlantUML  
-> PNG  
-> PDF
```

Si el error continúa mencionando SVG, debe revisarse la URL efectiva y el archivo `compiler.lua` realmente cargado.

6.120 La respuesta está vacía

La extensión puede detenerse con un mensaje equivalente a:

```
PlantUML devolvió una respuesta vacía
```

Debe comprobarse:

```
que el servidor está disponible  
que la URL generada es válida  
que el servidor admite el formato solicitado  
que la fuente PlantUML es aceptable  
que no existe un proxy o error de red
```

La URL incluida en el error permite reproducir la petición fuera de Quarto.

No debe ocultarse ese dato durante el diagnóstico.

6.121 Error de fuente PlantUML

Una fuente incorrecta puede contener:

```
@startuml ausente  
@enduml ausente  
elementos sin cerrar  
sintaxis no reconocida  
macros inexistentes  
estereotipos mal escritos
```

Reduzca temporalmente el bloque a:

```
```.plantuml
@startuml
Alice -> Bob: Prueba
@enduml
```
```

Si este ejemplo funciona, el problema está en la fuente original o en alguno de sus recursos.

Después, reincorpore los elementos de forma gradual.

6.122 El estilo no puede leerse

La configuración:

```
styles:
- resources/plantuml/iasi.puml
```

exige que el archivo exista y sea legible.

La extensión puede mostrar un error equivalente a:

```
No se pudo leer el estilo PlantUML
```

Compruebe:

```
la ruta
el nombre exacto
la extensión .puml
los permisos
la codificación
el directorio desde el que se resuelve
```

En Windows, una diferencia de mayúsculas puede pasar inadvertida localmente y fallar después en Linux.

Conviene mantener nombres exactos y consistentes.

6.123 El estilo produce un error

Un archivo `.puml` también puede contener sintaxis inválida.

Para aislarlo:

```
styles: []
```

o retire temporalmente la opción `styles`.

Después construya un diagrama mínimo.

Si funciona sin el estilo:

```
el servidor responde
el filtro funciona
el problema está en el archivo .puml
```

Revise sus últimas modificaciones antes de volver a activarlo.

6.124 El diagrama no cambia

Primero compruebe que fueron guardados:

```
el archivo .qmd
el archivo .puml
_quarto.yml
compiler.lua
```

Después limpie únicamente la caché PlantUML:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

Y vuelva a construir:

```
build(
  format = "html"
)
```

Si la imagen sigue sin cambiar, compruebe qué copia de la extensión se está ejecutando.

La caché no explica una modificación realizada en un archivo que Quarto nunca carga.

6.125 El diagrama funciona en HTML pero no en PDF

Cuando HTML funciona, ya están verificados:

```
la carga del filtro
la configuración
el servidor
la sintaxis PlantUML
la generación de la imagen
```

El fallo PDF puede estar en:

```
el formato de imagen
la conversión de recursos
la instalación TeX
la anchura del diagrama
la integración de Quarto con PDF
```

Compruebe primero que la URL PlantUML utiliza `/png/`.

Después revise:

```
quarto check
```

Si el error procede de LaTeX, debe tratarse como un problema del entorno PDF, no del servidor PlantUML.

6.126 El diagrama es demasiado ancho

Un diagrama puede verse correctamente en HTML y resultar pequeño o ilegible en PDF.

Las soluciones preferibles son:

```
reducir el número de elementos
dividir el diagrama
cambiar la orientación
acortar etiquetas
crear una vista general y otra de detalle
```

Puede probarse:

en lugar de:

No conviene resolver toda la complejidad reduciendo la tipografía hasta convertir el diagrama en polvo técnico.

6.127 La tipografía cambia

El servidor PlantUML utiliza las fuentes disponibles en su propio entorno.

Un contenedor puede no tener la misma tipografía que el equipo anfitrión.

Si el estilo declara:

pero Arial no existe dentro del servidor, PlantUML puede sustituirla.

Esto puede cambiar:

```
anchos  
saltos de línea  
tamaños  
disposición
```

La solución reproducible consiste en utilizar fuentes disponibles dentro de la infraestructura elegida o preparar una imagen de servidor que las incluya.

6.128 Caracteres incorrectos

Los archivos deben guardarse en UTF-8:

```
documentos .qmd  
estilos .puml  
archivos Lua  
configuración YAML
```

Cuando aparecen caracteres extraños, revise:

```
codificación del documento  
codificación del estilo  
fuentes instaladas  
respuesta del servidor
```

No debe corregirse sustituyendo acentos por texto sin tildes.

La publicación debe preservar correctamente su idioma.

6.129 Error al descargar la imagen

La extensión utiliza `pandoc.mediabag.fetch` para obtener la respuesta del servidor.

Un error en esta etapa puede deberse a:

```
URL inaccesible  
servidor detenido  
error HTTP  
respuesta incompatible  
problema de red
```

El mensaje debe incluir la URL solicitada y el detalle devuelto por Pandoc.

Pruebe esa misma URL desde el entorno de construcción.

6.130 MIME inesperado

El compilador espera que el servidor devuelva un tipo compatible con el formato solicitado.

Para PNG:

```
image/png
```

Para SVG:

```
image/svg+xml
```

Si el servidor devuelve una página HTML de error, una redirección o una respuesta intermedia, el contenido puede no ser una imagen válida.

Compruebe:

```
la URL completa
el código HTTP
el contenido recibido
la configuración del servidor o proxy
```

Una respuesta HTTP correcta no garantiza por sí sola que el cuerpo sea el diagrama esperado.

6.131 Caché dañada

Una entrada puede quedar incompleta por una interrupción durante la escritura o por una modificación del desarrollo.

Limpie:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

Después construya de nuevo.

Si el problema reaparece en una caché limpia, la causa no es una entrada antigua.

Debe revisarse la compilación o la escritura del artefacto.

6.132 Configuración no aplicada

La configuración PlantUML debe estar bajo:

```
filter-options:
  plantuml:
```

No en una clave arbitraria.

Ejemplo correcto:

```
filters:
  - extensions/plantuml/plantuml.lua

filter-options:
  plantuml:
    enabled: true
    server: http://localhost:1025
    format: png
    cache: true
```

Una indentación YAML incorrecta puede hacer que las opciones no pertenezcan a `plantuml`.
Revise la estructura, no únicamente los valores.

6.133 Una publicación utiliza otro `_quarto.yml`

En un multiproyecto puede haber:

```
un _quarto.yml en la raíz
un _quarto.yml en cada publicación
perfiles HTML y PDF
```

La configuración efectiva depende de la publicación que se construye.

Compruebe:

```
qué libro fue seleccionado
desde qué ruta se ejecutó build()
qué perfiles se aplicaron
dónde está declarado el filtro
```

No debe asumirse que todos los libros comparten automáticamente la misma extensión.

6.134 Limpieza mínima

La limpieza recomendada para PlantUML es:

```
Remove-Item -Recurse -Force .quarto\plantuml -ErrorAction SilentlyContinue
```

No se recomienda comenzar con:

```
borrar todos los PNG
borrar todos los SVG
borrar toda la carpeta .quarto
borrar la salida completa del libro
reinstalar Quarto
```

La limpieza debe corresponder al componente sospechoso.

6.135 Diagnóstico completo

Una secuencia práctica es:

1. guardar todos los archivos
2. leer la ruta exacta del error
3. confirmar el `compiler.lua` cargado
4. comprobar `_quarto.yml`
5. abrir el servidor en el navegador
6. ejecutar `curl` contra el servidor
7. probar un diagrama mínimo
8. desactivar temporalmente los estilos
9. limpiar `.quarto/plantuml`
10. construir solo HTML
11. construir PDF

Cada paso reduce el espacio de búsqueda.

6.136 Tabla rápida

| Síntoma | Causa probable | Comprobación |
|---|--|--|
| No se encuentra <code>plantuml.lua</code> | Ruta incorrecta | Revisar <code>filters</code> y estructura |
| Error <code>require</code> o <code>dofile</code> | Copia incompleta o ruta Lua incorrecta | Revisar <code>extensions/core</code> |
| <code>end expected near <eof></code> | Archivo Lua truncado o duplicado | Revisar la línea indicada y el final |
| No conecta con el servidor | Contenedor detenido o URL inaccesible | <code>curl</code> , <code>docker ps</code> , <code>docker logs</code> |
| URL con <code>/svg/</code> | Formato efectivo SVG | Revisar <code>compiler.lua</code> cargado |
| Error <code>rsvg-convert</code> | Quarto intenta convertir SVG para PDF | Forzar y verificar PNG |
| No se puede leer <code>.puml</code>
Diagrama antiguo | Ruta o permisos incorrectos
Caché o copia de extensión equivocada | Revisar <code>styles</code>
Guardar, verificar ruta y limpiar caché |
| HTML funciona, PDF falla | Entorno PDF o formato de imagen | Revisar PNG, TeX y <code>quarto check</code> |
| Tipografía distinta | Fuente ausente en el servidor | Revisar fuentes del contenedor |

6.137 Información útil al informar de un error

Un informe útil debe incluir:

```
mensaje completo  
ruta exacta del archivo Lua  
URL PlantUML mostrada  
formato solicitado  
configuración filter-options  
resultado de curl  
resultado de docker ps  
si falla HTML, PDF o ambos  
si ocurre con un diagrama mínimo
```

También conviene indicar:

```
sistema operativo  
entorno de ejecución  
versión de Quarto  
versión de iasi.quarto  
versión de la extensión
```

No es necesario adjuntar toda la publicación para comenzar el diagnóstico.

Una evidencia pequeña y precisa suele abrir la cerradura más rápido que una carretilla de logs.

El objetivo no es probar cosas hasta que funcione. Es identificar qué etapa dejó de cumplir su contrato.